DS6400 Portfolio

Stephen Olsen

Table of contents

Preface 5

I 01: Performance Measures 6

1 In class exploration and discussion 7
- 1.0.1 Exercise . . . 7
- 1.0.2 Exercise . . . 8
- 1.0.3 Exercise . . . 9
- 1.0.4 Exercise [incomplete] . . . 11

2 Week 1: Steyerberg et al. (2001) 12
- 2.1 Student exploration . . . 12

3 Week 2: Efron 1997 26
- 3.1 Student exploration . . . 26
  - 3.1.1 Key Ideas . . . 26
  - 3.1.2 Simulation Plan . . . 29
  - 3.1.3 Definitions . . . 30
  - 3.1.4 Equations . . . 32
  - 3.1.5 Simulation . . . 33

II 02: Visualizing the Model 43

4 In class exploration and discussion 44

5 Partial Dependence Plots (PDP) 45
- 5.1 Student Exploration . . . 45
- 5.2 Motivating Idea for the PDP . . . 45
- 5.3 Exploration . . . 46

III 03: 4 models 52

6 In class exploration and discussion 53
6.0.1 Background notes . . . 53
6.0.2 K-Nearest Neighbors (KNN) . . . 53
6.0.3 Random Forests . . . 54
6.0.4 Neural Networks . . . 55

7 Neural Net 56
7.0.1 Motivating ideas and chronology . . . 56
7.0.2 Math behind the model . . . 56
7.0.3 Computational Exploration . . . 60
7.0.4 Sources: . . . 69
7.0.5 Future Reference: . . . 69

IV 04: Linear Algebra Review 70

8 Appendix A 71
8.1 Self-assessment . . . 71
8.2 Overall assessment: . . . 72

9 Appendix B 73
9.1 Self-assessment . . . 73
9.2 Overall assessment: . . . 74

V 05: Nonlinearity 75

10 Explainer 76
10.1 The Linear Model . . . 76
10.2 Nonlinearity . . . 77
10.2.1 Basis expansion . . . 78
10.3 Make and read in some data . . . 78

VI 06: Regularization 82

11 Regularization 83
11.1 Homework Problems . . . 83
11.1.1 Preliminary: install required packages . . . 83
11.1.2 Setup . . . 84
11.1.3 Problem 0 . . . 84
11.1.4 Problem 1: The Problem . . . 84

11.1.5 Problem 2: Ridge . . . 85
11.1.6 Problem 3: Ridge . . . 86
11.1.7 Problem 4: Lasso . . . 86
11.1.8 Problem 5: Elastic Net . . . 88
11.1.9 Problem 6: PCR . . . 89
11.1.10 Problem 7: Comparison . . . 91

12 Paper: Lasso's Victory Lap **92**
12.1 1. Sparsity . . . 92
12.1.1 Extended applications (Table 1) . . . 93
12.2 2. The ℓ_1 penalty, convexity, and the contrast with ridge . . . 94
12.3 3. Why do the relaxed lasso? The 'S' stands for screening, not selection . . . 96
12.4 4. Critiques . . . 98
12.4.1 Bayesian (Holmes) . . . 98
12.4.2 Correlated predictors . . . 98
12.5 5. Response: the known gap is inference . . . 99
12.6 General notes . . . 100
12.6.1 History, and why it took a while to catch on . . . 100
12.6.2 Computational advances . . . 100

13 Explainer: Regularization **101**
13.1 Why regularize at all? . . . 101
13.1.1 The general recipe . . . 102
13.1.2 Two housekeeping rules: standardize and center . . . 102
13.2 The data . . . 103
13.3 Ridge regression (ℓ_2 penalty) . . . 104
13.3.1 The math . . . 104
13.3.2 Implementation from scratch . . . 105
13.3.3 Implementation with `glmnet` . . . 107
13.4 The lasso (ℓ_1 penalty) . . . 108
13.4.1 The math . . . 108
13.4.2 Implementation from scratch: coordinate descent . . . 112
13.4.3 The lasso path and choosing λ . . . 114
13.5 Elastic net (a mix of ℓ_1 and ℓ_2) . . . 117
13.5.1 The math . . . 117
13.5.2 Implementation . . . 117
13.6 Principal component regression (PCR) . . . 120
13.6.1 The math . . . 120
13.6.2 Implementation from scratch . . . 121
13.6.3 Implementation with `pls` . . . 122
13.7 Putting it together: test-set comparison on the diabetes data . . . 123
13.8 Summary . . . 125
13.9 References . . . 126

Preface

This is a Quarto book for capturing notes associated with DS6400 Advanced ML 1.

Original class notes are copied from https://tgstewart.cloud/ml1/ to retain formulas and original exercises.

This is a book created from markdown and executable code. It contains topics on machine learning, estimation, and various mathematical concepts.

It will be continually populated over the course of the semester.

Part I

01: Performance Measures

1 In class exploration and discussion

1.0.1 Exercise

Consider two generic prediction models f_1 and f_2 for univariate, continuous outcome $Y \in \mathbb{R}$ with multivariate predictors $X \in \mathbb{R}^d$. Without knowing anything about how the prediction models were created, describe how you might evaluate the performance of f_1 and f_2 if you know the true relationship between X and Y. Try to be precise in your description, using mathematical notation if helpful.

> In class we discussed X as a matrix D, for data, and Y as an output vector or single number P, for predicitons. In between D and P there may be a D' matrix where the data is transformed. The prediction is related to some target quantity that we care about.
>
> One way we might validate the model, is to run X through f to get $\hat{Y}$, and then we can compare $\hat{Y}$ to Y. The specific formula from class is below:
>
> $$f : D -> D' -> P \qquad f(x_0) = \hat{y_0} \qquad P \in \mathbb{R}$$
>
> Then, we will have to consider what is the method of comparison between Y and $\hat{Y}$?
>
> Some options include the following one-number summary values:
>
> 1. Mean Squared Error:
> $$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$
> 2. Root Mean Squared Error:
> $$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$
> 3. Mean Absolute Error:
> $$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Alternatively, we might use a visualization to evaluate this. We would plot Y vs. $\hat{Y}$.

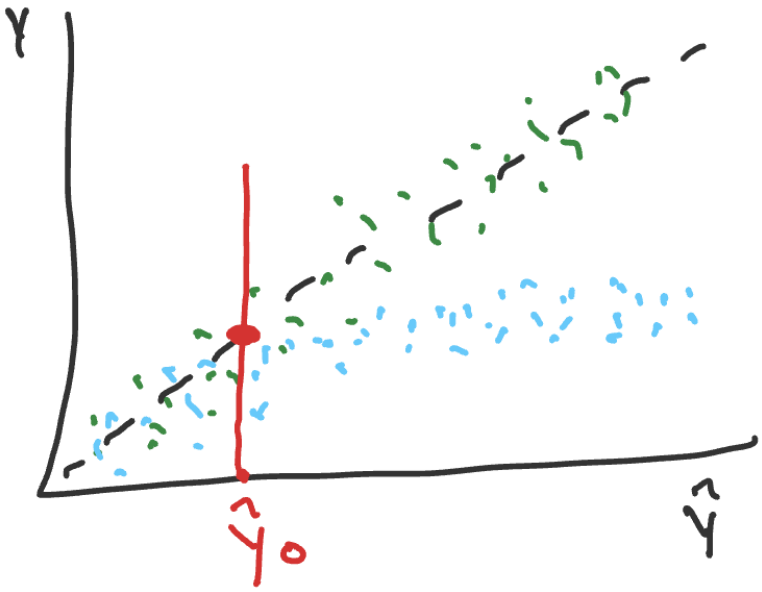


Figure 1.1: Calibration Curve

The plot above is the calibration curve. It is more descriptive than the available one number-summaries listed.

Q: What is the mathematical definition for "well-calibrated"? in the calibration curve? A: The expected value at y given a specific $\hat{y}_0$ is equal to $\hat{y}_0$.

$$\mathbb{E}[y \mid \hat{y}_0] = \hat{y}_0$$

OR

$$\mathbb{E}[y - \hat{y}_0 \mid \hat{y}_0] = 0$$

Calibration is all about the average or expectation, NOT about the spread around the line. Discrimination is another metric that looks at tightness around the line. These measures will be overly optimistic if we use the same data that built the model. More robust definitions and exploration of these concepts are continued below.

1.0.2 Exercise

Read RMS 5.1.2. Describe/define/develop an example of the following terms

1. Discrimination (rank based and variation based)

   Rank based discrimination is an assessment of whether predicted values are ordered correctly. From RMS 5.1.2, it is a rank correlation of $\hat{Y}$ and Y, with examples including Spearman's ρ, Kendall's τ, and Somers' D_{xy}. Variation based discrimination is an assessment of whether the predicted values differ or spread across predictions. From RMS 5.1.2, it is a measure based on variation in $\hat{Y}$, with examples including the regression sum of squares and the g–Index.

2. Calibration

   Calibration is a measure of the average predictive accuracy for f. From RMS 5.1.2, these measures assess absolute prediction accuracy. *Calibration–in–the–large* compares the average $\hat{Y}$ with the average Y. Calibration curves are a part of the overall calibration measures definition.

3. Calibration curve

   This is a plots Y values (actuals) versus $\hat{Y}$ values (predictions), which shows you the relationship between them, and can help assess performance at individual predicted values of $\hat{y}_0$. From RMS 5.1.2, a high-resolution calibration curve or *calibration–in–the–small* assesses the absolute forecast accuracy of predictions at individual levels of $\hat{Y}$. From class and as we can see below, this can help you observe calibration and discrimination.

   From Claude's assistance: * Discrimination = "does it sort people correctly (and by how much)?" * Calibration = "are its actual numbers trustworthy?"

1.0.3 Exercise

1. Sketch a calibration curve that represents poor discrimination but good calibration

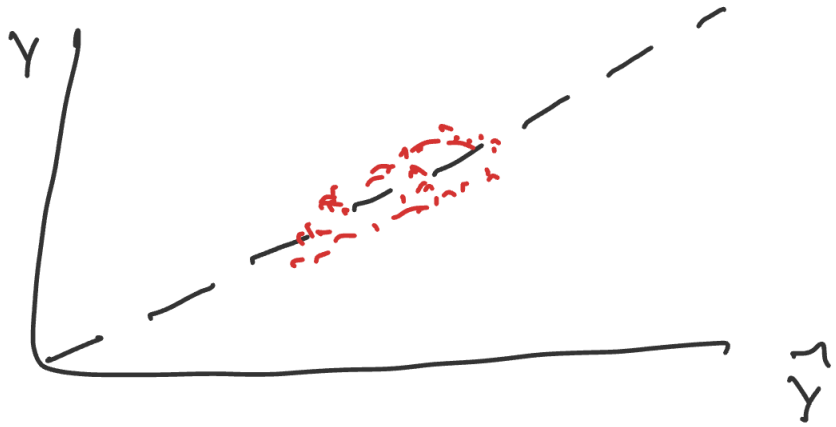


Figure 1.2: Good Calibration, Bad Discrimination

This plot is clustered around the line (good calibration), but it is not spread on $\hat{Y}$ (poor discrimination).

2. Sketch a calibration curve that represents good discrimination but poor calibration

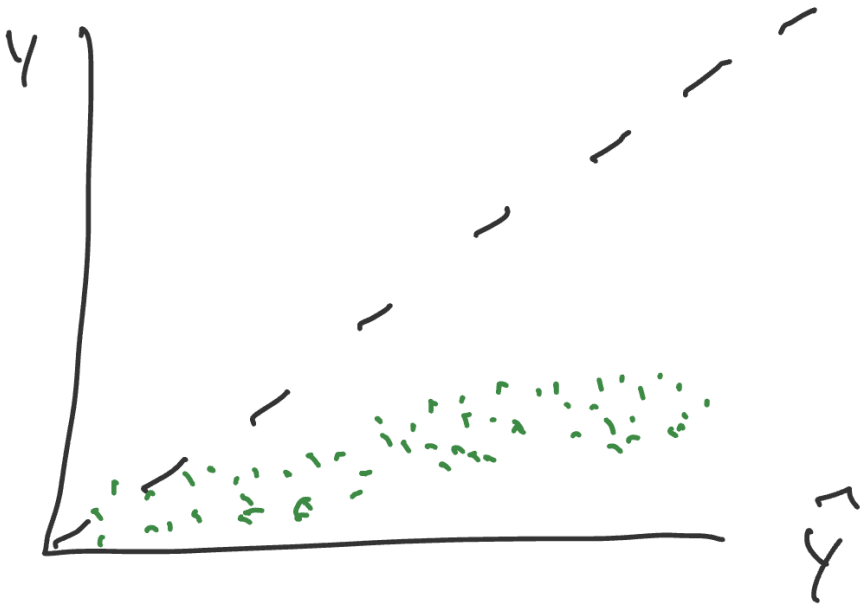


Figure 1.3: Bad Calibration, Good Discrimination

This plot is spread on $\hat{Y}$ (good discrimination) but not following the line (poor calibration).

1.0.4 Exercise [incomplete]

```
# may need to be installed first pak::pak("thomasgstewart/ds6021b")
require(ds6021b)
```

Consider the three predictor setting: $X \in \mathbb{R}^3$. Let

$$X_1, X_2 \sim N(0,1) \qquad X_3 \sim \text{BIN}(n=1, p=0.4)$$

$$Y \sim N(\mu(\mathbf{X}), 1) \qquad \mu(\mathbf{X}) = \sqrt{5}\,\text{sigmoid}(X_1^2 + X_3) + \sqrt{5}\,\text{sigmoid}(X_2)\,X_3$$

1. Simulate data from the model.
2. Create the calibration curves for f_1 to f_5, which are in the `ds6021b` package.
3. Calculate a global measure of model performance.
4. Add a loess curve to the calibration curve as a measure of local performance.

2 Week 1: Steyerberg et al. (2001)

2.1 Student exploration

1. Select one of the papers from the list below

   Selected Paper: Steyerberg et al. (2001) - Internal validation of predictive models: efficiency of some procedures for logistic regression analysis

2. For the initial exploration, do the following:

   - Prepare a summary of the key ideas. Be prepared to describe/illustrate the concepts in a concrete example.
   - Review the instructions for the student presentation and come with an idea/sketch of the simulation you'll perform.

   Key Ideas

   1. From the opening line of the abstract: *"The performance of a predictive model is overestimated when simply determined on the sample of subjects that was used to construct the model."* The authors expand on this idea of optimism. When the model is trained and evaluated on only data within the sample, the estimated performance will be optimistic compared with the real performance on new data that is completely outside the sample. This difference is between apparent performance and test performance.
   2. From the closing paragraph of the introduction: *"In this study we compare the efficiency of internal validation procedures for predictive logistic regression models."* (also basically the title lol) The authors walked through a comparison of the split-sample approach (test-train splits), cross-validation, and bootstrapping. Each one is an internal validation procedure that is commonly used for evaluating models.
   3. From the opening line of section 2.4: "We evaluated several internal validation procedures that aim to test performance more accurately than is achieved by the apparent performance estimate." The authors set out to determine which validation procedure of those listed above is best for the logistic regression model in question. They used the GUSTO-I dataset, detailed below, and fit multiple regression models for prediction.

Dataset: GUSTO-I; n=40,830; 2,851 deaths Working with a dataset of 30-day mortality after an acute myocardial infarction. What does that mean? They're looking at the % of patients who die from any cause within 30 days of a heart attack.

What they want us to do: Stop doing CV and split-sample (test-train splits), and start doing regular bootstraping instead for estimation of accuracy!

How they showed it: Logistic regression was fitted in each sample with a previously specified set of eight dichotomous predictors (including shock, age>65, diabetes, etc). The apparent performance of the model fitted to the full GUSTO-I data was the reference point from which they compared the other performance measures. *"The large size of the total GUSTO-I data set makes sure that the optimism in this performance estimate is negligible."* As mentioned above, they compared variants of split-sample, cross-validation, and bootstrap (including the .632 and .632+ methods) specifically at EPV=10 to analyze estimates of model performance.

Some results:

- The optimism and the variability of the model performance estimates reduced as sample size increased.
- "*The regular bootstrap resulted in the most accurate estimates of model performance: the bias was close to zero and the variability similar to that of the apparent performance."*
- Bootstrap resampling resulted in stable and nearly unbiased estimates of performance.
- Relation to some other papers: *"a puzzling observation was that the .632 and the .632+ bootstrap variants were not superior to the regular bootstrap."*

Definitions

- *Events Per Variable (EPV)*: The number of outcome events (i.e. number of deaths) divided by the number of predictor variables in the model. Recommended to be at least 10 to provide an adequate predictive model.
- *Internal Validation*: All methods of assessing performance using only the dataset you were provided.
- *External Validation*: Assessing performance using a TRULY NEW dataset. Something from another clinic, a separate experiment, real world data, etc.
- *Apparent Performance*: Predictive performance estimated on the training sample. Optimistically biased, because the same data is used to fit and evaluate the model.
- *Test Performance*: Predictive performance estimated on the test data (held-out sample), an independent test set consisting of all patients from GUSTO-I except the patients in the random sample.

- *Optimism*: The difference between apparent performance and a model's true performance on new data.
- *Split-sample approach*: Randomly split training data in two parts: one to develop the model and another to measure its performance.
- *Cross-validation*: Model is developed on one part of the data and tested on the other part, while these samples are randomly taken repeatedly and the average is taken as an estimate of the performance.
- *Bootstrap*: Replicates the process of sample generation from an underlying population by drawing samples with replacement from the original data set, of the same size as the original data set.
- *Discrimination*: The ability to distinguish high-risk subjects from low-risk subjects. Commonly quantified by a measure of concordance, the c statistic.
- *Calibration*: Whether the predicted probabilities agree with the observed probabilities. Well-calibrated models have a slope of 1, while models providing too extreme predictions have a slope less than 1. Well-calibrated models also have an intercept of 0, while models providing predictions that are too high or too low on average have an intercept different from 0.

Sketch of the simulation: Replicate a simulation from the paper

The goal is to demonstrate that the regular bootstrap returns the optimal estimates of internal validity.

- Start with the GUSTO-I dataset (from R or Python package)
- Generate sample and independent test sets for EPV = 10
- Complete split sample and regular boostrap on the sample dataset
- Compare Predictive performance (c statistic, calibration slope, Brier score, D(=scaledχ^2), R^2)

3. For the student presentation, do one of the following and come prepared to present it to the class:

- **Replicate a simulation or example from the paper** [SELECTION]
- Develop your own simulation study which explores a concept discussed in the paper
- Complete the simulation study described below.

Replicating the example from the paper

The following sections replicating the simulation were assisted by Claude. I am still working to develop a full understanding of the code, formulas, and methods completed within. Formulas and comments will be added progressively.

Dataset: GUSTO-I; n=40,830; 2,851 deaths Available in full anonymized version via `predtools` package in R.

Figures: Replicating a version of Figure 2 from the paper

In the paper, figure 2 shows bias and variability for a few of the procedures for estimating model performance in independent data for EPV = 10, including apparent performance, split-sample validation on 50% or 33% of the sample, cross-validation on 50% or 10% of the sample, and regular bootstrapping. In this simulation, I will show apparent performance, split-sample validation on 33% of the sample, and regular bootstrapping. I will also only show metrics for calibration and discrimination (c statistic, calibration slope, Brier score).

Methods: Replicating the steps of the paper

1. Load in the dataset
2. Define logistic regression model with 8 predictors from the paper
3. Generate a sub-sample for EPV = 10 (set a seed to keep results steady)
4. [Not needed for this simulation] Use model to compute the benchmark performance on the full dataset
5. Use model to compute apparent performance on sub-sample dataset [simulated many times to get bias and variability]
6. Use model to compute split-sample performance on sub-sample dataset [" "]
7. Use model to compute bootstrap performance on sub-sample dataset [" "]
8. Build figure 2 to compare performance!

```
## Loading in Gusto-I dataset from predtools

#install.packages("predtools")
library(predtools)
data(gusto)

str(gusto)
```

```
'data.frame':   40830 obs. of  29 variables:
 $ day30   : int  0 0 0 0 0 0 0 0 0 0 ...
 $ sho     : int  0 0 0 0 0 0 0 0 0 0 ...
 $ hig     : int  1 1 0 0 1 0 0 0 1 0 ...
 $ dia     : 'labelled' int  0 0 0 0 0 0 0 0 0 0 ...
  ..- attr(*, "label")= chr "Diabetes"
 $ hyp     : int  0 0 0 0 0 0 0 0 0 0 ...
 $ hrt     : int  0 1 0 0 1 1 1 0 0 1 ...
 $ ttr     : 'labelled' int  1 1 0 1 0 1 0 1 1 1 ...
  ..- attr(*, "label")= chr "Time to Relief of Chest Pain > 1h"
 $ sex     : Factor w/ 2 levels "male","female": 1 1 1 1 1 1 1 1 1 1 ...
  ..- attr(*, "label")= chr "Sex"
```

```
$ Killip  : Factor w/ 4 levels "I","II","III",..: 1 2 1 1 1 1 1 1 1 1 ...
 ..- attr(*, "label")= chr "Killip Class"
$ age     : num  19 19 20.3 20.8 21 ...
$ ste     : 'labelled' int  4 6 5 8 5 1 0 4 8 6 ...
 ..- attr(*, "label")= chr "Number of ECG Leads with ST Elevation"
$ pulse   : 'labelled' int  60 90 77 75 85 90 90 50 70 95 ...
 ..- attr(*, "label")= chr "Heart Rate"
 ..- attr(*, "units")= chr "beats/min"
$ sysbp   : 'labelled' int  130 150 106 124 135 129 130 120 157 130 ...
 ..- attr(*, "label")= chr "Systolic Blood Pressure"
 ..- attr(*, "units")= chr "mmHg"
$ ant     : int  1 1 0 0 1 0 0 0 1 0 ...
$ miloc   : Factor w/ 3 levels "Inferior","Other",..: 3 3 1 1 3 1 2 1 3 1 ...
 ..- attr(*, "label")= chr "MI Location"
$ height  : 'labelled' num  170 161 171 180 188 ...
 ..- attr(*, "label")= chr "Height"
 ..- attr(*, "units")= chr "cm"
$ weight  : 'labelled' num  60 72 72.5 69 107 115 60 80 65 76 ...
 ..- attr(*, "label")= chr "Weight"
 ..- attr(*, "units")= chr "kg"
$ pmi     : Factor w/ 2 levels "no","yes": 1 2 1 1 1 1 1 1 1 1 ...
 ..- attr(*, "label")= chr "Previous MI"
$ htn     : 'labelled' int  1 2 1 1 1 1 1 1 1 1 ...
 ..- attr(*, "label")= chr "History of Hypertension"
$ smk     : Factor w/ 3 levels "never","quit",..: 1 1 3 1 2 1 1 2 1 1 ...
 ..- attr(*, "label")= chr "Smoking"
$ pan     : int  1 2 1 2 1 1 2 1 2 1 ...
$ fam     : 'labelled' int  2 2 1 2 2 2 2 1 2 1 ...
 ..- attr(*, "label")= chr "Family History of MI"
$ prevcvd : int  1 1 1 1 1 1 1 1 1 1 ...
$ prevcabg: int  1 1 1 1 1 1 1 1 1 1 ...
$ regl    : int  6 8 7 7 2 10 2 3 5 8 ...
$ grpl    : int  17 24 18 19 6 30 5 9 15 24 ...
$ grps    : int  98 53 40 42 95 67 13 19 32 54 ...
$ tpa     : int  0 NA NA 1 0 1 NA NA 0 NA ...
$ tx      : Factor w/ 3 levels "SK+tPA","SK",..: 2 1 1 3 2 3 1 1 2 1 ...
 ..- attr(*, "label")= Named chr "Tx in 3 groups"
 .. ..- attr(*, "names")= chr "TX"
```

From the paper: *"A logistic regression model was fitted in each sample consisting of a previously specified set of eight dichotomous predictors: shock, age>65 years, high risk (anterior infarct location or previous MI), diabetes, hypotension (systolic blood pressure <100 mmHg), tachycardia (pulse>80), relief of chest pain>1 hr, female*

gender"

Variables we want are:

- Outcome: `day30`
- Predictors:
- Shock: `sho`
- Age > 65 years: `age` (needs transform to binary)
- High Risk: `ant` or `pmi` (needs transform to binary)
- Diabetes: `dia`
- Hypotension: `sysbp` (needs transform to binary)
- Tachycardia: `pulse` (needs transform to binary)
- Relief of chest pain > 1hr: `ttr`
- Female gender: `sex`

```
gusto_df <- data.frame(
   day30 = as.integer(gusto$day30),
   shock = as.integer(gusto$sho),
   age65 = as.integer(gusto$age > 65),
   highrisk = as.integer(gusto$ant == 1 | gusto$pmi == "yes"),
   diabetes = as.integer(gusto$dia),
   hypotension = as.integer(gusto$sysbp < 100),
   tachycardia = as.integer(gusto$pulse > 80),
   ttr = as.integer(gusto$ttr),
   female = as.integer(gusto$sex == "female")
)

head(gusto_df)
```

```
  day30 shock age65 highrisk diabetes hypotension tachycardia ttr female
1     0     0     0        1        0           0           0   1      0
2     0     0     0        1        0           0           1   1      0
3     0     0     0        0        0           0           0   0      0
4     0     0     0        0        0           0           0   1      0
5     0     0     0        1        0           0           1   0      0
6     0     0     0        0        0           0           1   1      0
```

All variables are now transformed to match the "eight dichotomous predictors" from the paper. Below, the logistic regression model is defined in R code for the upcoming simulations.

```r
model_formula <- day30 ~ shock + age65 + highrisk + diabetes + hypotension + tachycardia + t
```

From the paper: *"For example, for EPV=10 and 7.0% mortality, all training and test samples consisted of 1145 and 39,685 patients, with 80 and 2771 deaths, respectively."*

```r
## set seed for reproducibility
set.seed(17)

## Creating ID column for tracking which patients are in what samples
gusto_df$row_id <- seq_len(nrow(gusto_df))

## Function to draw the training sample for each time this needs to be run
draw_dev_sample <- function(data, sample_total = 1145, target_events = 80){

   ## EPV = 10 with 8 predictors requires 80 events (deaths)
   target_events <- target_events
   target_non_events <- sample_total-target_events

   ## Stratified sampling to ensure desired number of events
   # Split into events and non-events
   events_pool     <- gusto_df[gusto_df$day30 == 1, ]
   nonevents_pool  <- gusto_df[gusto_df$day30 == 0, ]

   # Draw the stratified subsample
   dev_events    <- events_pool[sample(nrow(events_pool), target_events), ]
   dev_nonevents <- nonevents_pool[sample(nrow(nonevents_pool), target_non_events), ]

   dev_sample <- rbind(dev_events, dev_nonevents)

   # Shuffle row order so events aren't grouped together
   dev_sample <- dev_sample[sample(nrow(dev_sample)), ]
}

## Function to draw the testing sample (all the rows that are not in the training sample)
draw_test_set <- function(data, dev_sample) {
  data[!(data$row_id %in% dev_sample$row_id), ]
}
```

Building reusable functions for computing desired metrics:

Metric definitions

C-statistic (discrimination) — the probability that a randomly chosen patient who died is assigned a higher predicted risk than a randomly chosen patient who survived:

$$c = P(\hat{p}_i > \hat{p}_j \mid Y_i = 1,\ Y_j = 0)$$

Estimated empirically as the proportion of all (event, non-event) pairs correctly ordered by the model, with ties counted as half-correct:

$$\hat{c} = \frac{1}{n_1 n_0} \sum_{i:\, Y_i=1} \sum_{j:\, Y_j=0} \left[\mathbb{1}(\hat{p}_i > \hat{p}_j) + 0.5 \cdot \mathbb{1}(\hat{p}_i = \hat{p}_j)\right]$$

where n_1 is the number of events and n_0 is the number of non-events. In this case from the paper, since we have a binary outcome, c is identical to the area under the receiver operating characteristic (ROC) curve; c varies between 0.5 and 1.0 for sensible models (the higher the better).

Calibration slope — the coefficient β from an auxiliary logistic regression of the observed outcome on the linear predictor (logit of the model's predicted probability):

$$\text{logit}(P(Y = 1)) = \alpha + \beta \cdot \text{linear predictor}$$

A well-calibrated model has $\beta = 1$. $\beta < 1$ indicates predictions that are too extreme (overfitting); $\beta > 1$ indicates predictions that are too conservative.

Brier score — mean squared error between predicted probability and observed outcome:

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^{n} (\hat{p}_i - Y_i)^2$$

Ranges from 0 (perfect) to 0.25 (worthless) for a well-behaved binary outcome model. Exactly the same formula as MSE, but specifically used for binary prediction models.

```
library(pROC)
```

```
Type 'citation("pROC")' for a citation.
```

```
Attaching package: 'pROC'
```

```
The following objects are masked from 'package:stats':

    cov, smooth, var
```

```r
compute_metrics <- function(y, pred) {
  # Guard against predicted probabilities of exactly 0 or 1,
  # which would break the logit transform below
  eps <- 1e-10
  pred <- pmin(pmax(pred, eps), 1 - eps)

  # Discrimination: c-statistic (AUC)
  c_stat <- as.numeric(pROC::auc(y, pred, quiet = TRUE))

  # Calibration slope: regress outcome on the linear predictor (logit of pred)
  lp <- log(pred / (1 - pred))
  cal_fit <- glm(y ~ lp, family = binomial())
  cal_slope <- unname(coef(cal_fit)["lp"])

  # Brier score
  brier <- mean((y - pred)^2)

  c(c_stat = c_stat, cal_slope = cal_slope, brier = brier)
}
```

Computing apparent performance for the logistic regression model on the training sample dataset

```r
get_apparent <- function(sample_data, formula) {
  fit <- glm(formula, data = sample_data, family = binomial())
  pred <- predict(fit, type = "response")
  list(fit = fit, metrics = compute_metrics(sample_data$day30, pred))
}
```

Defining the function to evaluate a given model on new data for the test performance

```r
eval_on_data <- function(fit, newdata) {
  pred <- predict(fit, newdata = newdata, type = "response")
  compute_metrics(newdata$day30, pred)
}
```

Function for the split-sample validation, essentially running a test-train split on the development sample and fitting the model.

```r
split_sample_validate <- function(sample_data, formula, train_frac = 0.6667) {
  n_train <- round(nrow(sample_data) * train_frac)
  train_idx <- sample(nrow(sample_data), n_train)

  train_set <- sample_data[train_idx, ]
  test_set  <- sample_data[-train_idx, ]

  fit <- glm(formula, data = train_set, family = binomial())
  eval_on_data(fit, test_set)
}
```

Defining function for the regular bootstrap method. Setting B=100 as the authors did in the paper. Sampling from the sample distribution with replacement, then training on that sample. Computing performance in the following way: *The performance in the bootstrap sample represents estimation of the apparent performance, and the performance in the original sample represents test performance. The difference between these performances is an estimate of the optimism in the apparent performance. This difference is averaged to obtain a stable estimate of the optimism. The optimism is subtracted from the apparent performance to estimate the internally validated performance [7]: estimated performance = apparent performance − average(bootstrap performance − test performance).* In other words, we evaluate model_b^* on the bootstrap sample itself (its own "apparent" performance), and on the original sample (its "test" performance).

$$\text{optimism}_b = \text{Perf}(\text{model}_b^*,\ \text{sample}_b^*) \ -\ \text{Perf}(\text{model}_b^*,\ \text{sample}_{\text{orig}})$$

Average the optimism across all B replicates:

$$\overline{\text{optimism}} = \frac{1}{B}\sum_{b=1}^{B} \text{optimism}_b$$

Subtract this from the apparent performance of the model fitted on the original (development) sample to get the bias-corrected estimate:

$$\text{estimated performance} = \text{apparent performance} - \overline{\text{optimism}}$$

```r
bootstrap_correct <- function(sample_data, formula, apparent_metrics, B = 100) {
  n <- nrow(sample_data)
  optimism_sum <- c(c_stat = 0, cal_slope = 0, brier = 0)

  for (b in 1:B) {
```

```
    boot_idx <- sample(n, n, replace = TRUE)
    boot_sample <- sample_data[boot_idx, ]

    boot_fit <- glm(formula, data = boot_sample, family = binomial())

    boot_apparent <- compute_metrics(
      boot_sample$day30,
      predict(boot_fit, type = "response")
    )
    boot_test <- compute_metrics(
      sample_data$day30,
      predict(boot_fit, newdata = sample_data, type = "response")
    )

    optimism_sum <- optimism_sum + (boot_apparent - boot_test)
  }

  avg_optimism <- optimism_sum / B
  apparent_metrics - avg_optimism
}
```

Function 7: run one full repetition (ties everything together)

```
# This is the "outer loop body" -- one full pass of the pipeline,
# matching one box in each boxplot of Figures 1 and 2.
run_one_repetition <- function(data, formula, sample_total = 1145, target_events = 80, B = 1

  dev_sample <- draw_dev_sample(data, sample_total, target_events)
  test_set   <- draw_test_set(data, dev_sample)

  # Apparent performance + the fitted model (reused below)
  app <- get_apparent(dev_sample, formula)
  apparent_metrics <- app$metrics

  # Test performance: evaluate the dev_sample-fitted model on truly new patients
  test_metrics <- eval_on_data(app$fit, test_set)

  # Split-sample (50/50, matching your simplified scope)
  split_metrics <- split_sample_validate(dev_sample, formula, train_frac = 0.6667)

  # Bootstrap-corrected
  boot_metrics <- bootstrap_correct(dev_sample, formula, apparent_metrics, B = B)
```

```
  # MATH NOTE: delta = method's estimate - test performance.
  # delta = 0 means the method perfectly recovered true generalization
  # performance for this repetition. This is exactly Figure 2's y-axis.
  data.frame(
    method = rep(c("apparent", "split_sample", "bootstrap"), each = 3),
    metric = rep(c("c_stat", "cal_slope", "brier"), times = 3),
    delta  = c(
      apparent_metrics - test_metrics,
      split_metrics - test_metrics,
      boot_metrics - test_metrics
    )
  )
}
```

Run the simulation loop [decreased to 10 for faster preview]

```
# NOTE: reduced from the paper's 500 repetitions to keep runtime
# reasonable for a live presentation. State this explicitly when
# presenting -- it's a deliberate, documented scope reduction, not
# an error.
n_reps <- 10

set.seed(17)
results_list <- vector("list", n_reps)

for (i in seq_len(n_reps)) {
  results_list[[i]] <- run_one_repetition(
    data = gusto_df,
    formula = model_formula,
    sample_total = 1145,
    target_events = 80,
    B = 100
  )
}

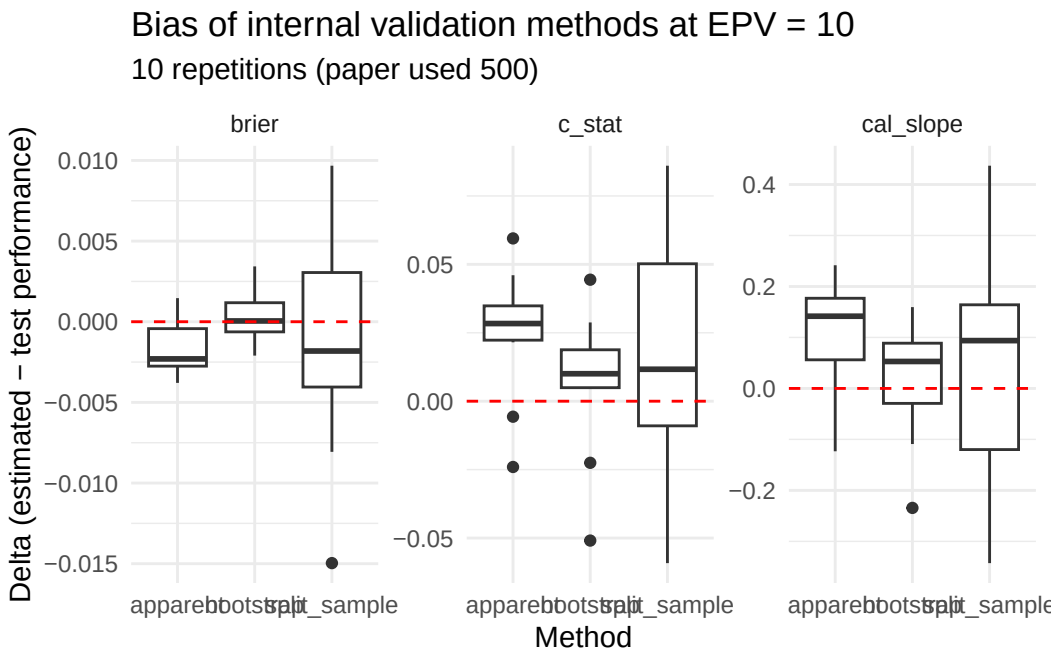
results <- do.call(rbind, results_list)
```

Visualize: recreate Figure 2

```
library(ggplot2)

ggplot(results, aes(x = method, y = delta)) +
```

```
  geom_boxplot() +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  facet_wrap(~ metric, scales = "free_y") +
  labs(
    title = "Bias of internal validation methods at EPV = 10",
    subtitle = paste0(n_reps, " repetitions (paper used 500)"),
    x = "Method",
    y = "Delta (estimated - test performance)"
  ) +
  theme_minimal()
```



Summarize bias and RMSE (reproduces Table 2's structure)

```
# MATH NOTE:
#   Bias = mean(delta)                -- average signed error
#   RMSE = sqrt(mean(delta^2))        -- root mean SQUARED error,
#                                        NOT centered around the bias.
#          RMSE^2 = Bias^2 + Var(delta)
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
summary_table <- results %>%
  group_by(method, metric) %>%
  summarise(
    bias = mean(delta),
    rmse = sqrt(mean(delta^2)),
    .groups = "drop"
  )

summary_table
```

```
# A tibble: 9 x 4
  method       metric         bias    rmse
  <chr>        <chr>         <dbl>   <dbl>
1 apparent     brier     -0.00173  0.00239
2 apparent     c_stat     0.0246   0.0335
3 apparent     cal_slope  0.102    0.147
4 bootstrap    brier      0.000324 0.00154
5 bootstrap    c_stat     0.00646  0.0260
6 bootstrap    cal_slope  0.0135   0.111
7 split_sample brier     -0.00130  0.00681
8 split_sample c_stat     0.0169   0.0438
9 split_sample cal_slope  0.0630   0.245
```

3 Week 2: Efron 1997

3.1 Student exploration

Selected Paper: Efron 1997 - Improvements on Cross-Validation: The .632+ Bootstrap Method

3.1.1 Key Ideas

3.1.1.1 Stop doing X, Start doing Y!

- Cross-validation is the standard way to estimate prediction error, but it's not good enough.
- Stop doing cross-validation, because the estimate of prediction error can be highly variable, even though it has low bias.
- Instead, use the bootstrap method with the .632+ rule, which performs better. Bootstrap procedures are smoothed versions of cross validation, with adjustments made to correct for bias.

3.1.1.2 Key Findings

- Of the 24 simulations completed, the results vary from experiment to experiment, but in terms of RMSE, the .632+ rule is an overall winner.
- The improvement in reducing variability of error rate predictions in the set of simulations is roughly equivalent to a 60% increase in the size of the training set.
- Bootstrap replications that give an estimate of prediction error can also be used to assess the variability of that estimate, which would be useful for inference, model selection, or comparing two models. Only studied on the leave-one-out boostrap method.

3.1.1.3 Quick reminder

- What is cross validation?

  - Repeated evaluation with partitions of a dataset used as test-train splits to calculate an average estimate of model performance and a sense of variance in performance across folds

- What is bootstrapping?
  - Replicates the process of sample generation from an underlying population by drawing samples with replacement from the original data set, of the same size as the original data set.
  - Used here for estimating variability / uncertainty of error rates

- How are they different?
  - Normal CV partitions the data into a fixed, small number of discrete splits (k-folds), where each data point is used for testing exactly once
  - Leave-one-out CV trains on exactly n−1 points and tests on the 1 left-out point, repeated once per observation
  - Bootstrapping resamples with replacement, generating a huge number of possible "pseudo-datasets" that vary continuously in composition, rather than a handful of fixed partitions
  - Leave-one-out bootstrap draws many bootstrap samples with replacement and, for each observation i, averages its error only over the bootstrap samples that happened to exclude i. Each observation's error estimate is itself an average over many different training-set compositions
  - How is bootstrapping a "smooth" version?
    * Because the bootstrap is averaged over many compositions, while the CV methods are dependent on cutoffs or possible variability in individual point predictions

- Why is this better?
  - Smoother estimates tend to have lower variance, which is especially useful for small datasets where a handful of CV folds can each look very different from one another

3.1.1.4 How they showed it

- Focused on the situation when the response is dichotomous (outcomes are 0, 1). Two of the explored prediction rules were Fisher's linear discriminant function (LDF), and the nearest-neighbor (NN) rule. Examples of these are shown in figure 1.

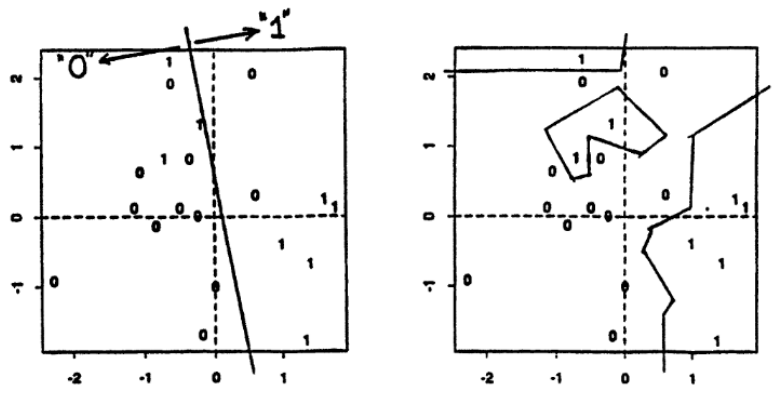


Figure 1. A Training Set Consisting of n = 20 Observations, 12 Labelled 0 and 8 Labelled 1. (a) The linear discriminant function predicts 0 to lower left of the solid line and 1 to upper right; (b) the nearest-neighbor rule predicts 1 in the three indicated islands and 0 elsewhere.

Figure 3.1: Figure 1. Showing examples of LDF and NN.

- Showed cross-validation as a nearly unbiased estimator of true error rate for both LDF and NN, but bootstrap-based estimator has RMSE 80% as large. Table 1 shows these simulation results.

Table 1. Error Rate Estimation for Situation (1)

	LDF		NN	
	Exp	*RMS*	*Exp*	*RMS*
CV1	.362	.123	.419	.123
.632+	.357	.096	.380	.099
True	.357		.418	

NOTE: CV1 is the cross-validation estimate based on omitting one observation at a time from the training set; 632+ is the bootstrap-based estimator described in Section 3. The table shows the expectation and RMS error of the two estimates for both the LDF and NN prediction rules. In both cases, RMS(.632+)/RMS(CV1) is about 80%.

Figure 3.2: Table 1. Showing comparison of CV and Bootstrap estimators.

- While the authors only studied classification problems with 0-1 loss, they claim that the results are non parametric, so they can be applied to any prediction rule. However, regression problems may have different behavior and the statistical approach may differ.
- Derived the Leave-one-out bootstrap estimator, which is a smoothed version of the leave-one-out cross-validation estimator. This is used in conjunction with the apparent

error rate to create the .632 estimator, and in turn, the .632+ rule. Further explanation in the Equations section below.

$$\widehat{\text{Err}}^{(.632+)} = \widehat{\text{Err}}^{(.632)} + \left(\widehat{\text{Err}}^{(1)'} - \overline{\text{err}}\right) \cdot \frac{0.368 \cdot 0.632 \cdot \hat{R}'}{1 - 0.368\,\hat{R}'}$$

- Performed 24 sampling experiments, to investigate the effects of different classifiers, training set sizes, signal-to-noise ration, number of classes, and number of observations.
- The results vary considerably from experiment to experiment, but in terms of RMSE, the .632+ rule is an overall winner.

3.1.2 Simulation Plan

Complete the simulation study outlined on the class notes page.

3.1.2.1 Simulation study (Efron and Tibshirani 1997)

The goal of Efron and Tibshirani 1997 was to persuade readers to use the 0.632+ bootstrap estimate of the error rate rather than alternatives. To demonstrate its superiority, the authors began with a simple simulation study involving a binary outcome (Y_i) and two predictors ($\mathbf{T}_i = (T_{i1}, T_{i2})$). The simulation study was to demonstrate that the variance of the estimator (and consequently the mean squared error) of the estimator was lower.

Data generation model

$$Y_i \sim \text{BIN}(1, 0.5)$$

$$\mathbf{T}_i | Y_i = y_i \sim N_2 \left(\begin{bmatrix} y_i - 1/2 \\ 0 \end{bmatrix}, I_2 \right)$$

where I is the identity matrix and N_2 is the bivariate normal distribution.

Methods evaluated

1. Leave-one-out cross-validation (LOO CV)
2. 0.632+ bootstrap
3. Bootstrap

Models evaluated

1. LDA

2. 1-NN
3. Regression with polynomial surface (logistic)

Simulation parameters

1. N = 10, 20, 30, 40, 50, 75, 100

Plan

1. Before beginning the simulation, consider how you will display the results. Devise a figure that will succinctly show the results and support the goal of the paper.
2. Consider modular programming
3. Start with a single simulation setting (method/model/N).

3.1.3 Definitions

- Cross Validation
  - Repeated evaluation with partitions of a dataset used as test-train splits to calculate an average estimate of model performance and a sense of variance in performance across folds
- Variance
  - Model is too sensitive to training data, so it captures noise too. Poor prediction on unseen new data.
  - High variance = model is overfit
  - Low variance = model is more stable
  - Variance = $\mathbb{E}\left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\right]$
    - Where,
    - $\hat{f}(x)$: predicted value by the model
    - $\mathbb{E}[\hat{f}(x)]$: average prediction over multiple training sets
- Bias
  - Error that occurs when a model is too simple to capture the true patterns in the data.
  - High bias = model is underfit
  - Low bias = model is better fit to patterns
  - Bias^2 = $(\mathbb{E}[\hat{f}(x)] - f(x))^2$
    - Where,
    - $\hat{f}(x)$: predicted value by the model
    - $f(x)$: true value
    - $\mathbb{E}[\hat{f}(x)]$: expected prediction over different training sets

- Error Rate
  - Apparent Error (rate): error rate calculated when evaluating the model on the data it was trained on, typically underestimates true error (optimistic)
    - also referred to as training error
  - True Error (rate): error rate calculated when evaluating the model on new / unseen data
    - also referred to as generalization error
  - Test Error (rate): error rate calculated on a held out test set, used to estimate true error
  - No-Information Error (rate): error rate that would occurr if predictors had no relationship to the response
    - simulating what error looks like under pure guessing (section 3)
    - used by .632+ estimator to determine how much to trust the apparent error and bootstrap error
- Relative Overfitting Rate ($\hat{R}$)
  - a measure of how much a model is overfitting
  - scaled between 0 (no overfitting) and 1 (overfitting equal to the no-information value)
- Mean Squared Error (MSE)
  - Average of the squared differences between predicted values and actual values
- Prediction error
  - difference between predicted value and actual value (residual)
  - aggregated across many residuals using MSE, RMSE, or other statistics.
- Bootstrap
  - Replicates the process of sample generation from an underlying population by drawing samples with replacement from the original data set, of the same size as the original data set.
  - used here for estimating variability / uncertainty of error rates
- Point Estimate
  - A single value calculated from sample data used as the best guess for an unknown population parameter
  - Example: a computed error rate, such as apparent error, as a point estimate of the true error rate
- Parametric

  - Fixed number of parameters estimated form the training data and used to make predictions
  - Examples: Linear regression, logistic regression

- Nonparametric
  - Make few assumptions about functional form and relationships between variables
  - number of parameters grows with the size of the data and are flexible for modeling complex relationships
  - Example: k-NN, Decision Trees

- 0-1 loss
  - assignment of a 0 for a correct classification and a 1 for an incorrect classification
  - also called "indicator loss"

- Fisher's linear discriminant
  - classification method used as a prediction rule for separating data into classes or groups based on a linear boundary
  - smooth, because there is a single line dividing groups

- Nearest neighbors
  - classification method used for separating data into classes or groups based on the closest points
  - unsmooth, because the decision boundary is local to closest points

3.1.4 Equations

Equation 8: leave-one-out cross-validation estimate of prediction error

$$\widehat{\text{Err}}^{(cv1)} = \frac{1}{n}\sum_{i=1}^{n} Q[y_i, r_{\mathbf{x}_{(i)}}(\mathbf{t}_i)] = \frac{1}{n}\sum_{i=1}^{n} Q(x_i, \mathbf{x}_{(i)})$$

where $\mathbf{x}_{(i)}$ is the training set with the ith observation removed.

$Q[y, r]$ is the discrepancy between a predicted value r and the actual response y.

Each observation is $x_i = (\mathbf{t}_i, y_i)$, where $\mathbf{t}_i$ is the predictor/feature vector and y_i is the response.

$r_{\mathbf{x}}(\mathbf{t})$ denotes the prediction rule, fit on training set $\mathbf{x}$, evaluated at feature vector $\mathbf{t}$.

Equation 14: leave-one-out bootstrap estimator (smoothed version of cross-validation)

$$\widehat{\text{Err}}^{(1)} = \frac{1}{n}\sum_{i=1}^{n} E_{\hat{F}_{(i)}}\{Q(x_i, \mathbf{x}^*)\}$$

where $\mathbf{x}^*$ is a bootstrap sample drawn from $\hat{F}_{(i)}$, the empirical distribution on $\mathbf{x}_{(i)}$.

Equation 24: The equation for the .632 estimator from Efron (1983):

$$\widehat{\text{Err}}^{(.632)} = 0.368 \cdot \overline{\text{err}} + 0.632 \cdot \widehat{\text{Err}}^{(1)}$$

This estimator is designed to correct upward bias in $\widehat{\text{Err}}^{(1)}$ by averaging it with the downwardly biased estimate $\widehat{\text{Err}}^{(.632)}$.

The coefficients $.368 = e^{-1}$ and .632 were suggested by an argument based on the fact that bootstrap samples are supported on approcimately .632n of the original data points.

The new equation for the .632+ rule, introduced by this paper, puts greater weight on $\widehat{\text{Err}}^{(1)}$ in situations where the amount of overfitting $\widehat{\text{Err}}^{(1)} - \overline{\text{err}}$ is large.

Equation 32: This equation is used in the simulation experiments of section 4.

$$\widehat{\text{Err}}^{(.632+)} = \widehat{\text{Err}}^{(.632)} + \left(\widehat{\text{Err}}^{(1)'} - \overline{\text{err}}\right) \cdot \frac{0.368 \cdot 0.632 \cdot \hat{R}'}{1 - 0.368\,\hat{R}'}$$

Equation 28: The relative overfitting rate

$$\hat{R} = \frac{\widehat{\text{Err}}^{(1)} - \overline{\text{err}}}{\hat{\gamma} - \overline{\text{err}}}$$

$\hat{R}$ ranges from 0, with no overfitting ($\widehat{\text{Err}}^{(1)} = \overline{\text{err}}$), to 1, with overfitting equal to the no-information value $\hat{\gamma} - \overline{\text{err}}$.

3.1.5 Simulation

What actually took place: - Generated data according to them model - Implemented functions for running simulation with LDF Classifier - Replicated resultts from Table 3, Experiment 3 for the LDF Classifier

1. Data generation model

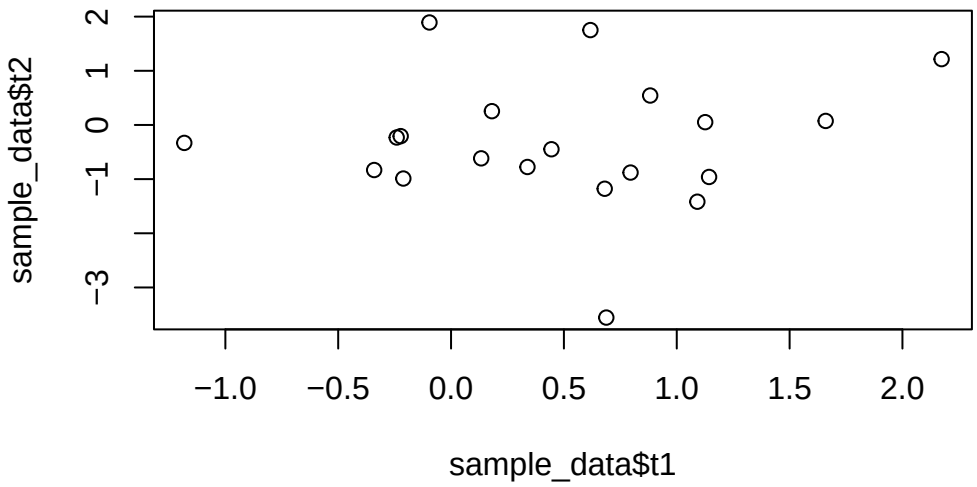
$$Y_i \sim \text{BIN}(1, 0.5)$$

$$\mathbf{T}_i | Y_i = y_i \sim N_2\left(\begin{bmatrix} y_i - 1/2 \\ 0 \end{bmatrix}, I_2\right)$$

where I is the identity matrix and N_2 is the bivariate normal distribution.

```r
## Data generating function
## n: number of observations to simulate
## Returns a data frame with columns y, t1, t2 matching Equation (1)
generate_data <- function(n) {
  y  <- rbinom(n, size = 1, prob = 0.5)
  t1 <- rnorm(n, mean = y - 0.5, sd = 1)
  t2 <- rnorm(n, mean = 0, sd = 1)
  data.frame(y = y, t1 = t1, t2 = t2)
}

## Quick sanity check
set.seed(17)
sample_data <- generate_data(20)
#table(sample_data$y)
#head(sample_data)
plot(sample_data$t1, sample_data$t2)
```



2. Set up supporting functions

   - Q Loss Function
   - LDF Model
   - True Error Function
   - LOO-CV Function

```r
## Equation 2 Q[y, r]
## y is the response, r is the predicted value
## Vectorized: y and r can be vectors of the same length,
## returning a vector of 0/1 losses (mean() of this gives an error rate)
qloss_function <- function(y, r) {
  ifelse(r == y, 0, 1)
}
```

```r
# LDA function comes from MASS library
library(MASS)

## Fit LDF on a training set and predict on given data
ldf_fit <- function(train_data) {
  lda(y ~ t1 + t2, data = train_data)
}

## Returns numeric 0/1 predictions (not the raw factor from predict())
ldf_predict <- function(fit, newdata) {
  pred <- predict(fit, newdata = newdata)
  as.numeric(as.character(pred$class))
}

## Sanity check: apparent error rate for one training sample (n = 20)
set.seed(17)
sample_data <- generate_data(20)
ldf_model <- ldf_fit(sample_data)
preds <- ldf_predict(ldf_model, sample_data)

apparent_error <- mean(qloss_function(sample_data$y, preds))
cat(sprintf("Apparent error for LDF model with n = %d observations: %.3f\n",
            nrow(sample_data), apparent_error))
```

```
Apparent error for LDF model with n = 20 observations: 0.350
```

```r
## Equation 5: true error rate for a fitted model
## fit:        a fitted model object (e.g., from ldf_fit())
## predict_fn: the matching predict function (e.g., ldf_predict())
## n_test:     size of the large independent test set used to
##             approximate Err(x, F) -- the bigger n_test, the less
##             noise in this estimate itself
true_error <- function(fit, predict_fn, n_test = 10000) {
```

```r
  test_data <- generate_data(n_test)
  preds <- predict_fn(fit, test_data)
  mean(qloss_function(test_data$y, preds))
}

## Sanity check: true error for the LDF model fit earlier (n = 20 training sample)
true_err <- true_error(ldf_model, ldf_predict)
cat(sprintf("True error for LDF model (n_test = %d): %.3f\n", 10000, true_err))
```

```
True error for LDF model (n_test = 10000): 0.319
```

```r
## Equation 8: leave-one-out cross-validation estimate of prediction error
## data:       the full training sample (data frame with columns y, t1, t2)
## fit_fn:     model fitting function (e.g., ldf_fit())
## predict_fn: matching predict function (e.g., ldf_predict())
##
## For each observation i: fit on all OTHER n-1 points (x_(i)),
## predict on the held-out point i, and score that one prediction.
## Averaging these n held-out losses gives Err^(cv1).
loo_cv_error <- function(data, fit_fn, predict_fn) {
  n <- nrow(data)
  losses <- numeric(n)

  for (i in seq_len(n)) {
    train_data <- data[-i, ]
    test_point <- data[i, , drop = FALSE]

    fit  <- fit_fn(train_data)
    pred <- predict_fn(fit, test_point)

    losses[i] <- qloss_function(test_point$y, pred)
  }

  mean(losses)
}

## Sanity check: LOO-CV estimate for the same n = 20 sample used above
loo_cv_err <- loo_cv_error(sample_data, ldf_fit, ldf_predict)
cat(sprintf("LOO-CV error for LDF model (n = %d): %.3f\n",
            nrow(sample_data), loo_cv_err))
```

```
LOO-CV error for LDF model (n = 20): 0.450
```

```r
## First define bootstrap function
## Includes note to save the index of the sample data, so that the
## leave-one-out bootstrap can utilize that information
##
## NOTE: redraws if the resample happens to contain only one class --
## rare, but possible for small/imbalanced n, and LDA cannot fit
## without both classes present. This is a small, standard guard
## against a degenerate resample, not a change to the bootstrap logic.
draw_bootstrap_sample <- function(data) {
  repeat {
    idx <- sample(nrow(data), nrow(data), replace = TRUE)
    if (length(unique(data$y[idx])) > 1) break
  }
  list(sample_data = data[idx, ], included_idx = idx)
}
```

```r
## Equations 14-17: leave-one-out bootstrap estimate Err^(1)
## data:       the full training sample
## fit_fn:     model fitting function (e.g., ldf_fit())
## predict_fn: matching predict function (e.g., ldf_predict())
## B:          number of bootstrap replications (paper uses B = 50)
##
## For each of the B bootstrap resamples: fit on the resample, then
## predict on ALL n original points at once. For each original point i,
## keep that prediction's loss ONLY if i happened to be excluded from
## this particular resample (Equation 15's I_i^b). Averaging point i's
## kept losses over just those "excluded" replicates gives E_i
## (Equation 17); averaging E_i over all i gives Err^(1) (Equation 14).
loo_bootstrap_error <- function(data, fit_fn, predict_fn, B = 50) {
  n <- nrow(data)

  ## Running totals per original observation i:
  ##   loss_sum[i]  = sum of Q_i^b over replicates b where i was excluded
  ##   exclude_n[i] = number of replicates where i was excluded
  loss_sum  <- numeric(n)
  exclude_n <- numeric(n)

  for (b in seq_len(B)) {
    boot <- draw_bootstrap_sample(data)
    fit  <- fit_fn(boot$sample_data)

    ## Predict on every original point at once -- we need Q_i^b for
```

```
    ## all i, then we'll discard the ones where i WASN'T excluded.
    preds  <- predict_fn(fit, data)
    losses <- qloss_function(data$y, preds)

    ## TRUE for observations NOT drawn into this bootstrap resample
    excluded <- !(seq_len(n) %in% boot$included_idx)

    loss_sum[excluded]  <- loss_sum[excluded] + losses[excluded]
    exclude_n[excluded] <- exclude_n[excluded] + 1
  }

  E_i <- loss_sum / exclude_n
  mean(E_i)
}

## Sanity check: leave-one-out bootstrap error for the same n = 20 sample
set.seed(17)
loo_boot_err <- loo_bootstrap_error(sample_data, ldf_fit, ldf_predict, B = 50)
cat(sprintf("Leave-one-out bootstrap error for LDF model (n = %d, B = %d): %.3f\n",
            nrow(sample_data), 50, loo_boot_err))
```

```
Leave-one-out bootstrap error for LDF model (n = 20, B = 50): 0.450
```

```
## Equation 24: the .632 estimator
## apparent_error: err-bar, the resubstitution/training error
## err1:           Err^(1), the leave-one-out bootstrap error
estimator_632 <- function(apparent_error, err1){
    (.368*apparent_error) + (.632*err1)
}

## Sanity check: .632 estimate for the same n = 20 sample, using the
## apparent_error and loo_boot_err already computed above
err_632 <- estimator_632(apparent_error, loo_boot_err)
cat(sprintf(".632 error for LDF model (n = %d): %.3f\n",
            nrow(sample_data), err_632))
```

```
.632 error for LDF model (n = 20): 0.413
```

```
## Equations 27-32: the .632+ estimator

## Equation 27: No information error rate (gamma-hat)
```

```r
## ps: observed proportion of y == 1 (p-hat-1)
## qs: observed proportion of predictions == 1 (q-hat-1)
gamma_hat <- function(ps, qs){
    (ps*(1-qs)) + ((1-ps)*qs)
}

## Equation 31: Relative overfitting rate
## NOTE: uses the raw err1 (not the truncated err1'), per the paper's definition
## NOTE: the paper describes R-hat as ranging from 0 to 1, but the raw
## formula has no upper cap -- if gamma_hat lands only slightly above
## apparent_error, the ratio can explode (confirmed via simulation: some
## replicates produced R_hat_prime in the thousands/billions). Capping
## at 1 enforces the paper's stated range and avoids a near-zero or
## negative denominator later in the .632+ formula.
R_hat_prime <- function(err1, apparent_error, gamma_hat_val){

    cond <- (err1>apparent_error) && (gamma_hat_val>apparent_error)
    raw <- ifelse(cond, (err1-apparent_error)/(gamma_hat_val-apparent_error), 0)
    #pmin(raw, 1)

}

## Equation 32: .632+ Rule used for simulations
estimator_632_plus <- function(err_632, err1, gamma_hat_val, apparent_error, r_hat_prime_val)

    ## Equation 31's truncated leave-one-out bootstrap error, Err^(1)'
    err1prime <- pmin(err1, gamma_hat_val)

    ## Calculate estimator
    numerator <- (.368*.632*r_hat_prime_val)
    denominator <- (1-(.368*r_hat_prime_val))

    err_632+((err1prime - apparent_error)*(numerator/denominator))
}


## Sanity check: .632+ estimate for the same n = 20 sample
set.seed(17)
sample_data <- generate_data(20)
ldf_model <- ldf_fit(sample_data)
preds <- ldf_predict(ldf_model, sample_data)
apparent_error <- mean(qloss_function(sample_data$y, preds))
```

```r
loo_boot_err <- loo_bootstrap_error(sample_data, ldf_fit, ldf_predict, B = 50)
err_632 <- estimator_632(apparent_error, loo_boot_err)
gamhat <- gamma_hat(mean(sample_data$y), mean(preds))
rhatprime <- R_hat_prime(loo_boot_err, apparent_error, gamhat)
err_632_plus <- estimator_632_plus(err_632, loo_boot_err, gamhat, apparent_error, rhatprime)
cat(sprintf(".632+ error for LDF model (n = %d): %.3f\n",
            nrow(sample_data), err_632_plus))
```

```
.632+ error for LDF model (n = 20): 0.466
```

```r
## Run every estimator once, on one freshly generated training sample.
## Returns a single ONE-ROW data frame -- one named column per estimator.
## Stacking many of these rows (one per replicate) is how "one experiment"
## becomes Table 3's Exp/SD/RMS summaries.
one_rep <- function(n, fit_fn, predict_fn, B=50, n_test=10000){

    ## All of the functions we just defined
    sample_data <- generate_data(n)
    model <- fit_fn(sample_data)
    preds <- predict_fn(model, sample_data)

    apparent_error <- mean(qloss_function(sample_data$y, preds))
    true_err <- true_error(model, predict_fn, n_test = n_test)
    cv1 <- loo_cv_error(sample_data, fit_fn, predict_fn)
    loo_boot_err <- loo_bootstrap_error(sample_data, fit_fn, predict_fn, B = B)
    err_632 <- estimator_632(apparent_error, loo_boot_err)
    gamhat <- gamma_hat(mean(sample_data$y), mean(preds))
    rhatprime <- R_hat_prime(loo_boot_err, apparent_error, gamhat)
    err_632_plus <- estimator_632_plus(err_632, loo_boot_err, gamhat, apparent_error, rhatpr

    ## Put the results together! One row, one column per quantity.
    data.frame(
      true_err = true_err,
      apparent = apparent_error,
      cv1 = cv1,
      err1 = loo_boot_err,
      err_632 = err_632,
      err_632_plus = err_632_plus
    )
}
```

```r
## Repeat one_rep() n_reps times and stack the one-row results into a
## single data frame: one row per replicate, one column per estimator.
montecarlo <- function(n_reps, n, fit_fn, predict_fn, B = 50, n_test = 10000){
  results_list <- vector("list", n_reps)

  for (r in seq_len(n_reps)) {
    results_list[[r]] <- one_rep(n, fit_fn, predict_fn, B = B, n_test = n_test)
  }

  do.call(rbind, results_list)
}

## Run the simulation for LDF, n = 20 (experiment #3), matching the
## paper's 200 Monte Carlo replications for this experiment
set.seed(17)
sim_results <- montecarlo(n_reps = 200, n = 20, fit_fn = ldf_fit, predict_fn = ldf_predict)

## Summarize: Exp (mean) and SD for each estimator column, and RMS
## measured against EACH replicate's own true_err (not the overall
## average true error) -- matching how Table 3 defines RMS
summary_table <- data.frame(
  estimator = names(sim_results),
  Exp = sapply(sim_results, mean),
  SD  = sapply(sim_results, sd),
  RMS = sapply(sim_results, function(col) sqrt(mean((col - sim_results$true_err)^2)))
)
rownames(summary_table) <- NULL
summary_table
```

```
     estimator       Exp         SD       RMS
1     true_err 0.3545585 0.05683561 0.0000000
2     apparent 0.2507500 0.09208330 0.1412507
3          cv1 0.3462500 0.11935100 0.1187273
4         err1 0.3733764 0.10796744 0.1065094
5      err_632 0.3282499 0.09860228 0.1012579
6 err_632_plus 0.3571477 0.18377678 0.1817677
```

The table above replicates experiment 3 from Table 3 in the paper.

This demonstrates that the .362 estimator has the best overall performance for the LDF Classifier for n = 20, p = 2.

Sources:

- https://www.geeksforgeeks.org/machine-learning/bias-vs-variance-in-machine-learning/
- https://www.geeksforgeeks.org/machine-learning/parametric-vs-non-parametric-models-in-machine-learning/
- https://www.statology.org/prediction-error-statistics/
- Claude Sonnet 5

Part II

02: Visualizing the Model

4 In class exploration and discussion

Review of Master's Content: https://tgstewart.cloud/ipm/visualizing-the-model.html

Partial Effects Plot

- Hold all but one of the features constant, and plot the relationship between the variable feature and the outcome variable (ex. x1, x3 are held constant, x2 varies, y is the outcome. plot y vs x2)

Individual Conditional Expectation (ICE) plot

- Use some information about the distribution of the features to investigate the relationship between a separate feature and the outcome variable
- vary x1 and x3 based on their distributions. vary x2 in the same way it was before for the PE plot. For each iteration of the variation in x1 and x3, plot y vs x2, and investigate the relationship.

Partial Dependence Plot (PDP)

- the average of each line on the ICE plot
- a marginal estimate of the PE plot

Accumulated Local Effect (ALE) plots

- needed when features are NOT independent (ex. BMI and weight are not independent, but could both be predictors in a model)
- uses partial derivatives to figure out the direction of the slope at different locations
- main take home message is that correlated predictors require special visualization techniques to tease apart the relationship that exists between the predictors and outcome

5 Partial Dependence Plots (PDP)

5.1 Student Exploration

Resources

1. Elements of Statistical Learning (ESL) - Section 10.13.2
2. Molnar 2022 - Section 19

5.2 Motivating Idea for the PDP

The Partial Effects plot and Independent Conditional Expectation (ICE) plot help with visualizing a model when we don't know the inner workings, specifically the relationships between features and outcomes. Partial Effects plots are used to investigate a single feature while holding all other features constant. ICE plots investigate a single feature while allowing the other features to vary across every individual value in the dataset. You can see how ICE plots may provide more information than the Partial Effects plot, but also provide too many lines on a given plot to be useful.

Enter the Partial Dependence Plot (PDP). We'll take the average line across all of the ICE plot lines, to investigate the average relationship of a feature to the predicted outcome.

Partial Dependence Plot (PDP)

A partial dependence plot shows the marginal effect one or two features have on the predicted outcome of an ML model. It can show whether the relationship between target and feature is linear, monotonic, or more complex.

The partial dependence function is defined as

$$\hat{f}_S(\mathbf{x}_S) = \mathbb{E}_{\mathbf{X}_C}[\hat{f}_S(\mathbf{x}_S, X_C)]$$

- $\mathbf{x_s}$ are the features which are plotted on the x axis of the PDP. Typically only one feature is in the set S. This is the feature we are investigating the effect of on the prediction.
- X_C are the other features used in the machine learning model $\hat{f}$
- If the feature vectors $\mathbf{x_s}$ and $\mathbf{x_C}$ are combined, they make up the total data $\mathbf{X}$

- The partial function $\hat{f}_S$ is estimated by calculating averages in the training data (aka. Monte Carlo):

$$\hat{f}_S(\mathbf{x}_S) = (1/n)\sum_{i=1}^{n}\hat{f}(\mathbf{x}_S, \mathbf{x}_C^{(i)})$$

- This step is equivalent to averaging all of the ICE curves of the dataset. For each of the points along the x axis for $\mathbf{x_s}$, you take the mean across the vertical direction ($\mathbf{x}_C^{(i)}$).
- So in this equation,
  - n is the number of instances in the dataset.
  - $\mathbf{x}_C^{(i)}$ are actual feature values from the dataset for features we are not interested in

5.3 Exploration

Let's explore how to create and interpret the PDP.

First, we'll generate some data from the following distribution.

$$\begin{aligned} E[Y \mid X] &= \sqrt{5}\,\text{sigmoid}(X_1 + X_3) + \sqrt{5}\,\text{sigmoid}(X_2)\,X_3 \\ V[Y \mid X] &= 1 \\ \text{where} \quad X_1, X_2 &\sim N(0,1) \\ X_3 &\sim \text{Bernoulli}(p = 0.4) \\ \text{sigmoid}(x) &= \frac{1}{1+e^{-x}} \end{aligned}$$

```
sigmoid <- function(x) 1 / (1 + exp(-x))

generate_data <- function(n) {
  X1 <- rnorm(n, mean = 0, sd = 1)
  X2 <- rnorm(n, mean = 0, sd = 1)
  X3 <- rbinom(n, size = 1, prob = 0.4)
  mu <- sqrt(5) * sigmoid(X1 + X3) + sqrt(5) * sigmoid(X2) * X3
  Y  <- mu + rnorm(n, mean = 0, sd = 1)
  data.frame(Y, X1, X2, X3)
}

set.seed(17)
df <- generate_data(1000)
#head(df)
```

Now we'll fit some models on the data. First, a random forest.

```r
library(randomForest)

train_data <- generate_data(1000)
test_data <- generate_data(1000)

model_formula <- Y ~ X1 + X2 + X3
rf <- randomForest(model_formula, data = train_data, ntree = 500)

preds <- predict(rf, newdata = test_data)
```

We have a fitted model that can generate predictions (now our Y values or outcomes), and we want to investigate the effect of say X_2 on Y.

Let's walk through building the PDP to see this.

First, here's what our data looks like

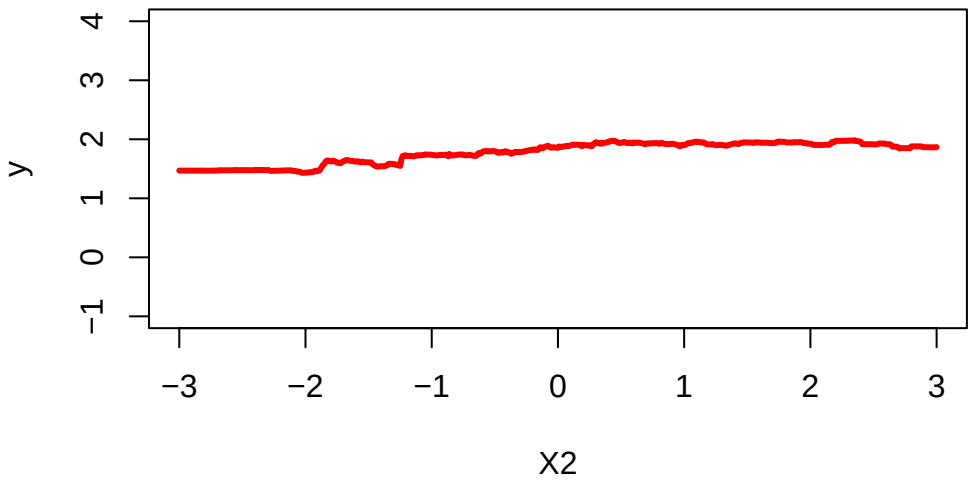
```r
df <- generate_data(1000) # generate some data
preds <- predict(rf, newdata = df) # use the RF model to create predicted outcomes
df$preds <- preds # save predicted values to df
head(df)
```

```
           Y          X1         X2 X3     preds
1  2.1390719  0.05183928 -0.8736403  1 2.6070967
2  5.5903463  1.21341246  1.2258274  1 3.3018672
3  1.4677137 -0.98338726 -0.1526766  0 0.7781615
4 -0.7025197 -0.39636582 -0.8310759  0 1.1085454
5  2.1489275 -0.04981333  0.7814832  1 2.9208151
6 -1.1496933 -1.44647693 -1.2100006  0 0.7678897
```

Then, we'll want to visualize the PDP plot. Recall that we'll have our Predictions on the y-axis, X_2 on the x-axis. X_1 and X_3 vary across all of their real values in the dataset, and we get the average line to put on the plot.

```r
# Create a partial effect plot for each draw, store it in O
O <- list()
for(i in 1:nrow(df)){
    R <- expand.grid(X1 = df[i,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[i,"X3"])
    O[[i]] <- predict(rf, newdata = R)
}

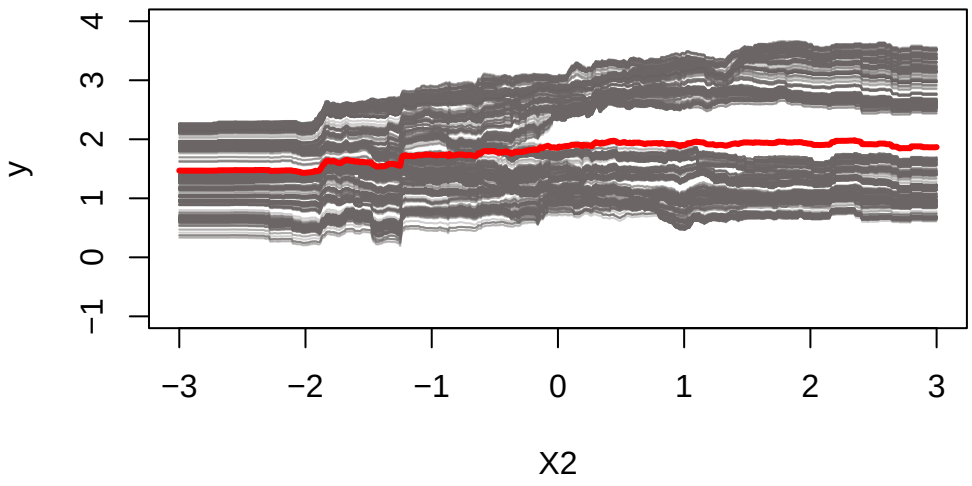
P <- do.call("cbind", O) |> rowMeans()
plot(R[,2], P, xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4), lwd = 3, col = "red")
```



To contextualize this, it can be helpful to see the ICE plots with the PDP plot.

```
# Partial effect plot for the first random x
R <- expand.grid(X1 = df[1,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[1,"X3"])
plot(R[,2], predict(rf, newdata = R), xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4))

# Overlay the plot for all the xs
for(i in 2:nrow(df)){
    R <- expand.grid(X1 = df[i,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[i,"X3"])
    lines(R[,2], predict(rf, newdata = R), xlab = "X2", ylab = "y", type = "l", col = "#6b650
}
lines(R[,2], P, xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4), lwd = 3, col = "red")
```



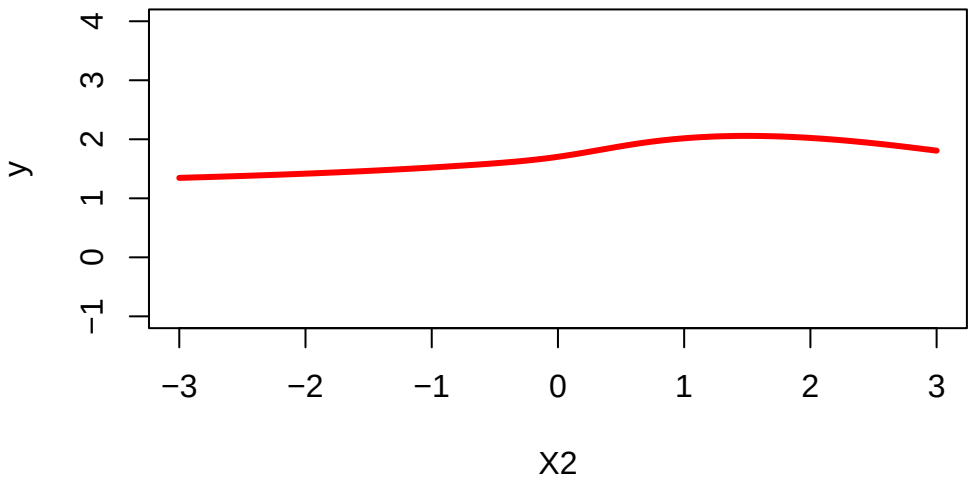
Next, let's repeat for the linear regression with splines.

```
library(splines)

lm_spline <- lm(Y ~ ns(X1, df = 4) + ns(X2, df = 4) * X3, data = df)

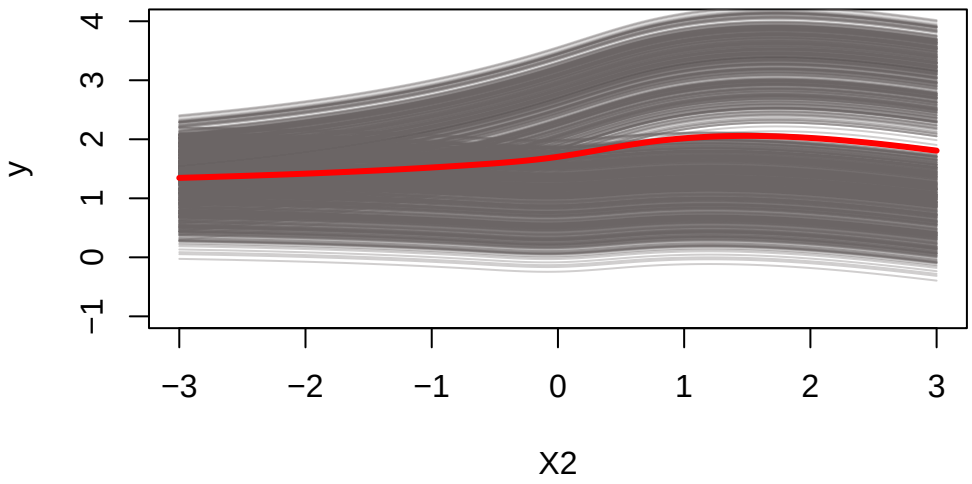
O <- list()
for(i in 1:nrow(df)){
    R <- expand.grid(X1 = df[i,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[i,"X3"])
    O[[i]] <- predict(lm_spline, newdata = R)
}

P <- do.call("cbind", O) |> rowMeans()
plot(R[,2], P, xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4), lwd = 3, col = "red")
```




```
# Partial effect plot for the first random x
R <- expand.grid(X1 = df[1,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[1,"X3"])
plot(R[,2], predict(lm_spline, newdata = R), xlab = "X2", ylab = "y", type = "l", ylim = c(-1

# Overlay the plot for all the xs
for(i in 2:nrow(df)){
    R <- expand.grid(X1 = df[i,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[i,"X3"])
    lines(R[,2], predict(lm_spline, newdata = R), xlab = "X2", ylab = "y", type = "l", col =
}
lines(R[,2], P, xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4), lwd = 3, col = "red")
```


y
4
3
2
1
0
−1
−3
−2
−1
0
1
2
3
X2

Part III

03: 4 models

6 In class exploration and discussion

Review of Master's Content: https://tgstewart.cloud/ml1/03-four-models.html

6.0.1 Background notes

- **non-parametric**: no coefficients are estimated, no functional form is assumed
  - example: KNN
- **hyperparameter**: a tuning parameter that is set before training the model. It is NOT dependent upon the training data, and does not change during the training process. It may be tuned or adjusted outside of the training loops through grid search processes, but that requires training many individual models according to the hyperparameter selections
  - example: k in KNN
- ***parameter**: a parameter that is learned during the model training, based on the training data
  - example: β in regression
- **standardization**: (also called feature scaling, and also helpfully also called Z-score normalization) rescales data so that every feature has a mean of 0 and a standard deviation of 1. This is especially helpful for KNN, because magnitude of features has a significant effect on the model.
- **curse of dimensionality**: as the dimension increases, meaning the number of features used for prediction grows, the nearness of training data to a new points decreases, because things are just further away in high dimensional space. Therefore, KNN works well in low dimensions, but not so well in high dimensions.

6.0.2 K-Nearest Neighbors (KNN)

- Given a new observation, return the average outcome value based on the k nearest values in the training set.
  - "Nearest" or **closeness** is determined by a distance function, most commonly the Euclidean distance.

- This model is non-parametric, because no coefficients are estimated and no functional form is assumed
- k is the hyperparameter determining how many points should be included in the predicted outcome for a new point provided to the model
  - if k is small, there are few local points used to make the prediction
  - as k grows, more local to global points are used in the prediction average
  - increasing k makes the model more smooth and trades variance for bias (lower variance, higher bias)
- Standard KNN workflow:
  1. standardize the predictors
  2. fit on the training data
  3. generate predictions on a grid
  4. plot a partial effect plot for one predictor

Key Callout - "The bias-variance tradeoff. At $k = 1$, KNN memorizes the training data — zero bias on training observations, high variance across different training sets. At $k = n$, it ignores everything except the mean — high bias, low variance. The optimal k sits somewhere in between, and finding it requires the cross-validation tools from Module 3. This tradeoff — and the challenge of navigating it — is the central tension in all of predictive modeling."

6.0.3 Random Forests

- Built on the decision tree
  - continually splitting the predictor space into partitions through binary splits
  - at each step, determine the splita cross the space that reduces within-node MSE
  - REPEAT until a termination condition is met
  - also called *"recursive binary partitioning"*
- **dendrogram**: the branching diagram that gives decision trees their name, and also the way to visualize the splits made in the decision tree
- "The random forest averages many trees, each trained on a different bootstrap sample of the training data. Averaging over many unstable but roughly unbiased models reduces variance without increasing bias — the central maneuver behind ensemble methods."
  - Because each tree is different when different bootstrap samples are used, we want to average over them to get the "best" model.
- The "random" in random forests is from the feature randomization that happens at each split. For every split, you only get to choose from a subset of the overall features, so that not every tree ends up being dominated by the same feature splits. Typically you use $p/3$ at each step to determine the number of features available.

  - This reduces correlation among the trees so we have a model with lower variance.

- Key hyperparameters:
  - B: the number of trees
    - more stable predictions, more reliable error estimate
    - more computation required
  - $mtry$: the number of predictors considered at each split
    - smaller mtry leads to less correlation between trees
    - larger mtry leads to less feature randomization

6.0.4 Neural Networks

From Tom's content: "it constructs a prediction surface from the predictors using a particular functional form, which is often called a network."

What does that mean? It is a continuous prediction surface that is smooth, not rectangular or stair-step like you would see in K-Nearest Neighbors or Random Forests. All points in the training set contribute to all weights in the network, so information sharing is **global.**

Neuron: basic building block of a neural network - receives linear combination of inputs, passes result through a non-linear function, produces a single number Layers: arrangement of neurons - first layer receives predictors $(x_1, x_2, ..., x_p)$ - hidden layers transform the output of a previous layer into a new representation - final layer produces the prediction $\hat{y}$

In the diagram 5.21, x1 and x2 are the individual neurons receiving predictors, the first layer. It passes through no hidden layers, so the predictors are directly related to the prediction $\hat{y}$. There are weights on the lines connecting the neurons, which we can consider the betas in a linear regression. Therefore, this representaiton is equivalent to a linear regression.

We need nonlinear activation functions in the hidden layers, because otherwise we'd just be modeling linear relationships, since our network is just doing multiplication and addition throughout the layers. - From Tom: "Without nonlinear activation, the forward pass simplifies to … linear regression. Every additional hidden layer without activation collapses in the same way. The nonlinear activation is the essential ingredient."

7 Neural Net

7.0.1 Motivating ideas and chronology

Our brains are incredible recognition and prediction machines. How do they work? Can we replicate the behavior artificially in a computer?

The provided papers give an chronology of the ideas in published literature.

1. McColloch & Pitts (1943): A Logical Calculus of the Ideas Immanent in Nervous Activity
    - Turning neuroscience into calculus.
    - The first mathematical model of a neural network using binary neurons for computation.
2. Rosenblatt (1958): The Perceptron: A probabilistic Model for Information Storage and Organization in the Brain
    - Tackles the questions of information storage and memory for how we regognize things.
    - Introducing the perceptron: the theory for a hpyothetical nervous system, or machine.
3. Rumelhart, Hinton & Williams (1986): Learning representations by back-propagating errors
    - Introducing back-propagation for "networks of neurone-like units".
    - New way to train networks with gradient descent on the errors, so that features can be discovered in hidden layers.

7.0.2 Math behind the model

Let's jump into the notation and formulas with some visuals along the way to help out.

7.0.2.1 Some definitions

Neuron: The building block of a neural network.

- A function that takes inputs and produces a single number output (between 0 and 1)
- Inputs in the first set of neurons (first layer) are the predictors.

- Inputs in future sets of neurons (hidden layers) are weighted sums of outputs from the prior layer.

Layers: Arrangements of neurons.

- Input layer: the first layer of neurons that receives the predictors.
- Output layer: the final layer of neurons that produces the predictions $\hat{y}$
- Hidden layer: layers inbetween the input and output. Can be any number of layers we choose.
  - transforms output of the previous layer
  - able to distingush features and their relationships to the predictions

Simplest example: No hidden layers.

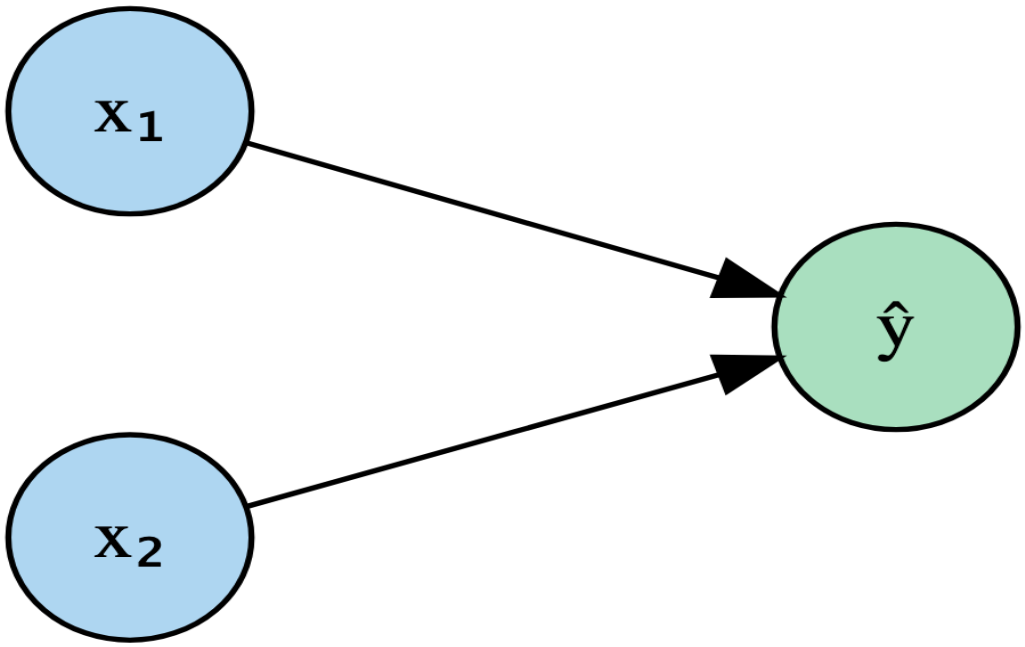


Figure 7.1: A network with no hidden layers is equivalent to linear regression: the output is a linear combination of the inputs.

Next example: One hidden layer.

7.0.2.2 Some equations

Let's start with the example for the single hidden layer. The full equation for going from input $\mathbf{x}$ to predicted $\hat{y}$ is

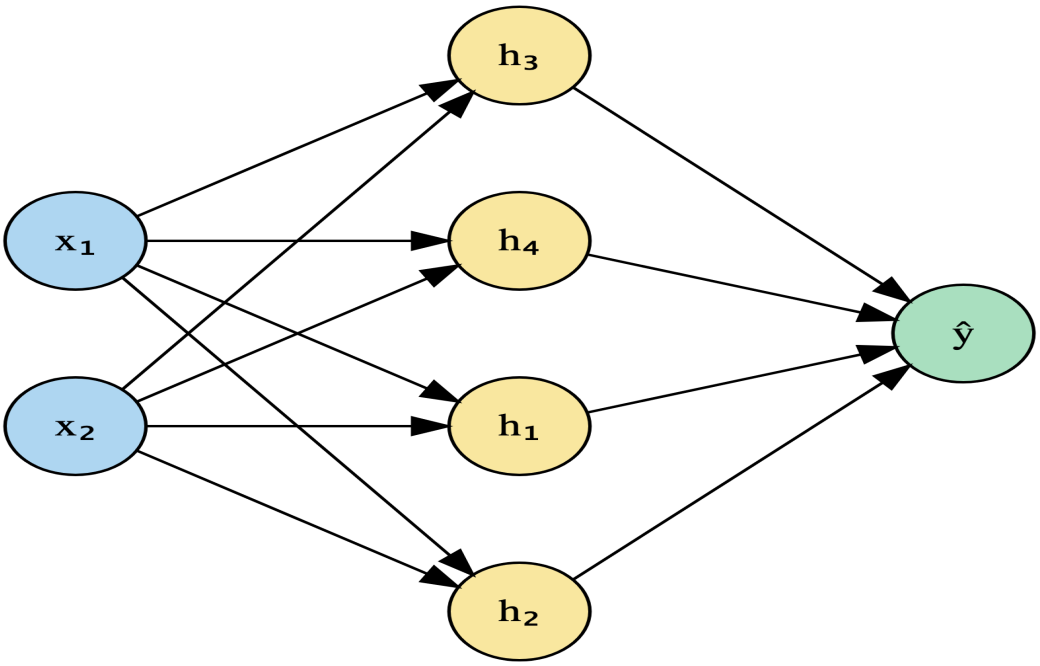


Figure 7.2: A network with one hidden layer. Each hidden neuron applies a nonlinear activation function to a weighted sum of the inputs.

$$\hat{y}(\mathbf{x}) = \mathbf{w}_{\text{out}}^t \, \sigma(W_1 \mathbf{x} + \mathbf{b}_1) + b_{\text{out}}$$

This is also called the "forward pass", to go from input $\mathbf{x}$ to a prediction $\hat{y}$.

- W_1 is the $m \times p$ **weight** matrix for the hidden layer, given a network with p predictors and m hidden neurons.
- $\mathbf{b}_1 \in \mathbb{R}^m$ is the hidden layer's **bias** vector
- $\sigma(\cdot)$ is a nonlinear activation function applied element-wise
- The hidden layer's outputs are then combined linearly using the output-layer weights $\mathbf{w}_{\text{out}} \in \mathbb{R}^m$ and bias $b_{\text{out}} \in \mathbb{R}$.
- In Figure 7.2 there are $p = 2$ inputs and $m = 4$ hidden neurons, so W_1 is a 4×2 matrix, $\mathbf{b}_1$ and $\mathbf{w}_{\text{out}}$ each have 4 entries, and each arrow in the figure is one weight.

Then we can consider the generalization of this function for multiple layers.

For this network, the output of each layer becomes the input to the next:

$$\mathbf{h}^{(1)} = \sigma(W_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{h}^{(2)} = \sigma(W_2 \mathbf{h}^{(1)} + \mathbf{b}_2), \quad \dots, \quad \hat{y} = \mathbf{w}_{\text{out}}^t \mathbf{h}^{(L)} + b_{\text{out}}$$

where L is the number of hidden layers. Figure 7.3 has $L = 2$: W_1 is 4×2 and W_2 is 3×4.

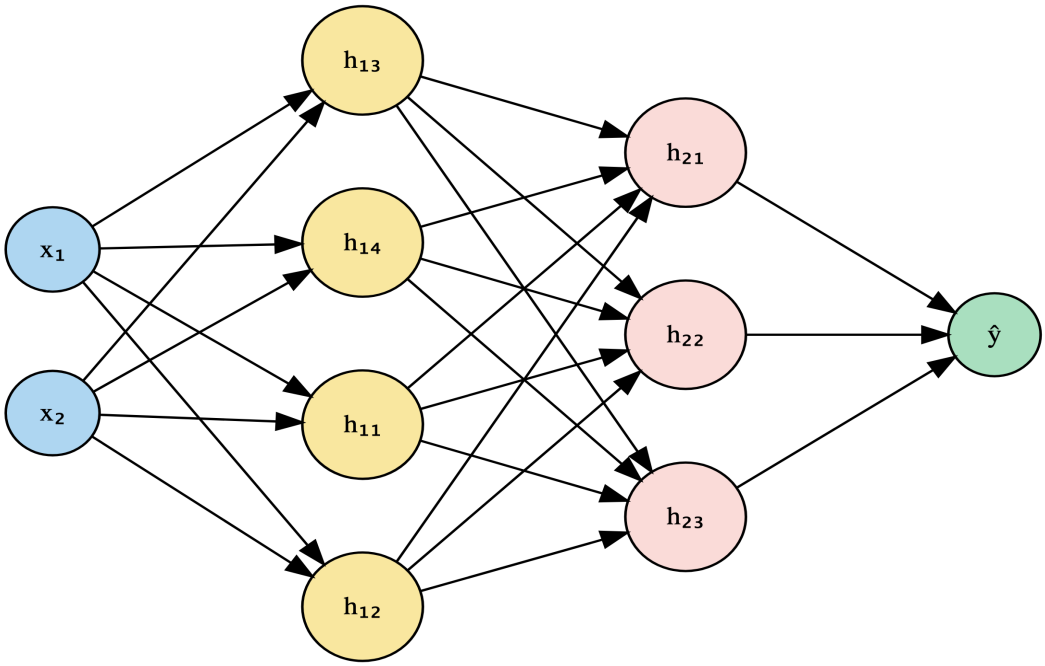


Figure 7.3: A network with two hidden layers. Each layer refines the representation of the previous layer, enabling hierarchical feature construction.

Those sigmas represent activation functions. These make the network nonlinear, so that it isn't just linear regression.

Function	Formula
Sigmoid	$\sigma(z) = \frac{1}{1+e^{-z}}$
ReLU	$\sigma(z) = \max(0, z)$
tanh	$\sigma(z) = \tanh(z)$

7.0.2.3 A larger visualization of the matrix

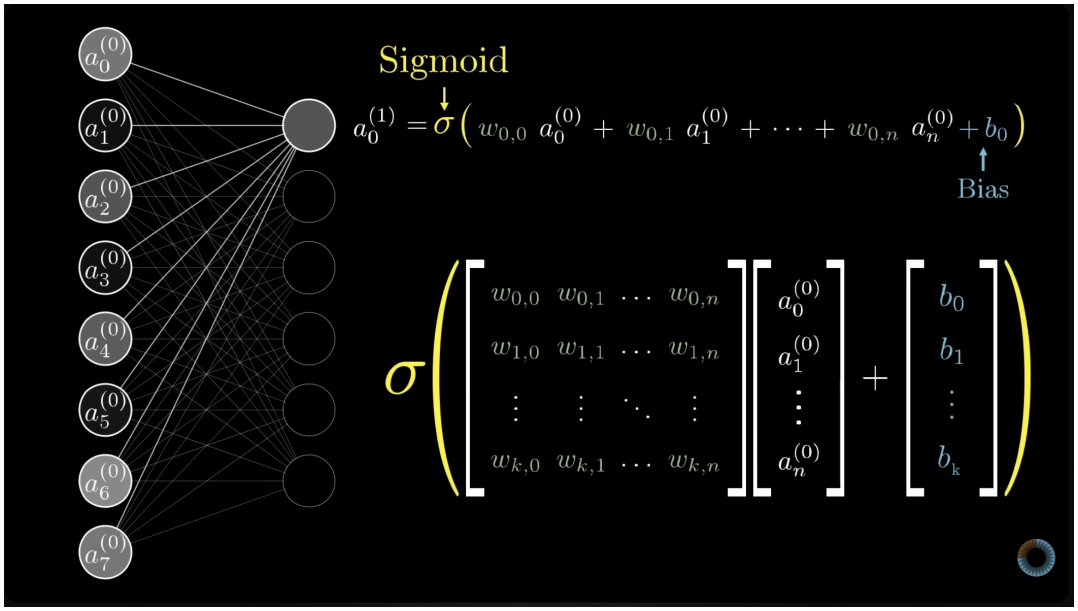


Figure 7.4: One layer of the forward pass written out in full: the weighted sum for a single neuron (top) and for every neuron in the next layer at once (bottom). Source:

7.0.2.4 Training the model

The network is trained by minimizing a loss function over the training data. For a continuous outcome, the loss is mean squared error:

$$\mathcal{L}(\mathbf{w}) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}(\mathbf{x}_i)\right)^2$$

We minimize the loss function through **gradient descent**, where the weights are updated iteratively in the direction that reduces the loss most steeply. At each step,

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$$

$\eta > 0$ is the **learning rate**, which controls the size of each step. If it is too large, the optimizer diverges; if it is too small, convergence is slow. Other key training hyperparameters are the number of **epochs** (passes over the training data) and the **batch size** (how many observations are used to estimate each gradient update).

The gradients $\nabla_{\mathbf{w}} \mathcal{L}$ are computed efficiently by applying the chain rule backwards through the network, from the output layer to the first hidden layer. This procedure is called **backpropagation**.

7.0.2.5 Really helpful resource for explaining this

https://youtu.be/aircAruvnKk?si=rcepTJBVQbB5APTH

Watch 3Blue1Brown Playlist for Deep Learning on Youtube

7.0.3 Computational Exploration

Below is an exploration of the neural networks math above using the Run Club Marathon Performance Dataset from Kaggle. This is a synthetic dataset created to simulate marathon training data for machine learning practice. It was generated with some realistic correlations based on sports science principles.

```
import kagglehub
import pandas as pd

path = kagglehub.dataset_download("likithagedipudi/run-club-marathon-performance-dataset")

train_data = pd.read_csv(path + "/train.csv")
```

```
test_data = pd.read_csv(path + "/test.csv")

train_data.head()
```

	runner_id	age	gender	running_experience_months	previous_marathon_count	training_program
0	R009432	25	Male	48	0	Intermediate
1	R077741	43	Female	151	2	Advanced
2	R034376	32	Female	82	3	Advanced
3	R020754	57	Male	6	0	Beginner
4	R001301	54	Female	11	0	Beginner

7.0.3.0.1 Preparing the data

We'll predict a runner's `actual_finish_time_minutes` from two predictors, `running_experience_months` and `weekly_mileage_km`. With two inputs, the networks below match the figures and situation covered above (Figure 7.1, Figure 7.2, and Figure 7.3).

The networks are trained on `train.csv` and evaluated on `test.csv`.

```
from sklearn.preprocessing import StandardScaler

predictors = ["running_experience_months", "weekly_mileage_km"]
outcome = "actual_finish_time_minutes"

# drop the runners with no recorded finish time
train = train_data.dropna(subset=[outcome])
test = test_data.dropna(subset=[outcome])

X_train, y_train = train[predictors], train[outcome]
X_test, y_test = test[predictors], test[outcome]

# Standardize to mean 0, sd 1. Gradient descent struggles when values are on
# very different scales (months vs. km vs. ~300 minutes).
# The scalers are fit on the training data only.
x_scaler = StandardScaler().fit(X_train)
y_scaler = StandardScaler().fit(y_train.to_frame())

X_train_s = x_scaler.transform(X_train)
X_test_s = x_scaler.transform(X_test)
y_train_s = y_scaler.transform(y_train.to_frame()).ravel()
```

7.0.3.0.2 Training settings

Using scikit-learn's `MLPRegressor` (multi-layer perceptron) to set up the network definition function. In subsequent tests, we will only change the hidden layers argument.

```
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import root_mean_squared_error

def make_network(hidden_layers, seed=1717):
    return MLPRegressor(
        hidden_layer_sizes=hidden_layers,  # neurons in each hidden layer
        activation="relu",                 # sigma(z) = max(0, z)
        solver="sgd",                      # (stochastic) gradient descent
        learning_rate_init=0.01,           # eta, the learning rate
        max_iter=100,                      # maximum number of epochs
        batch_size=64,                     # observations per gradient update
        alpha=0,                           # no penalty, so the loss is just MSE
        random_state=seed,                 # which random starting weights to use
    )

def predict_minutes(model, X_s):
    # the network predicts standardized y, so convert back to minutes
    y_s = model.predict(X_s)
    return y_scaler.inverse_transform(y_s.reshape(-1, 1)).ravel()
```

7.0.3.0.3 No hidden layers

If we set up the no hidden layers neural network, we simply have a linear regression: $\hat{y} = \mathbf{w}_{\text{out}}^t \mathbf{x} + b_{\text{out}}$. An empty tuple `()` in the make_network function gives us zero hidden layers.

```
nn0 = make_network(hidden_layers=()).fit(X_train_s, y_train_s)

print("w_out:", nn0.coefs_[0].ravel(), " b_out:", nn0.intercepts_[0])

# compare to ordinary linear regression on the same data
from sklearn.linear_model import LinearRegression
lm = LinearRegression().fit(X_train_s, y_train_s)
print("linear regression coefficients:", lm.coef_)
print("linear regression intercept:", lm.intercept_)
print("\n")
print("Neural Network test RMSE (minutes):", root_mean_squared_error(y_test, predict_minutes
print("Linear Regression test RMSE (minutes):", root_mean_squared_error(y_test, predict_minu
```

```
w_out: [-0.62354756  0.0126559 ]  b_out: [-0.0067104]
linear regression coefficients: [-0.64664405 -0.02330685]
linear regression intercept: -1.9109609522985936e-17


Neural Network test RMSE (minutes): 25.614727707768825
Linear Regression test RMSE (minutes): 25.54888984904703
```

We can compare the two models and see that the coefficients are very close between them, and the RMSE is nearly the same, too.

7.0.3.0.4 One hidden layer

With one hidden layer, we now introduce the non-linearity of the activation function in between the input and output. We'll use 4 neurons on our one hidden layer, like we had in the image Figure 7.2.

```
nn1 = make_network(hidden_layers=(4,)).fit(X_train_s, y_train_s)

# coefs_ holds the weight matrices, intercepts_ holds the bias vectors
print("weight shapes:", [w.shape for w in nn1.coefs_])
print("bias shapes:  ", [b.shape for b in nn1.intercepts_])

print("Test RMSE (minutes):", root_mean_squared_error(y_test, predict_minutes(nn1, X_test_s))
```

```
weight shapes: [(2, 4), (4, 1)]
bias shapes:   [(4,), (1,)]
Test RMSE (minutes): 23.522162138666413
```

We recall the equation from above:

$$\hat{y}(\mathbf{x}) = \mathbf{w}_{\text{out}}^{t}\, \sigma(W_1 \mathbf{x} + \mathbf{b}_1) + b_{\text{out}}$$

The first weight matrix is 2×4. scikit-learn stores it as $p \times m$, the transpose of our $m \times p$ matrix W_1, but it holds the same 8 weights (one per input-to-hidden arrow in Figure 7.2). The second is $\mathbf{w}_{\text{out}}$ with 4 entries.

7.0.3.0.5 Two hidden layers

Finally, let's set up the two hidden layers network. We use 4 neurons in the first hidden layer and 3 in the second, as in Figure 7.3.

```
nn2 = make_network(hidden_layers=(4, 3)).fit(X_train_s, y_train_s)

print("weight shapes:", [w.shape for w in nn2.coefs_])
print("Test RMSE (minutes):", root_mean_squared_error(y_test, predict_minutes(nn2, X_test_s)))
```

```
weight shapes: [(2, 4), (4, 3), (3, 1)]
Test RMSE (minutes): 22.886675111689467
```

Now there are three weight matrices: W_1^t (2×4), W_2^t (4×3), and $\mathbf{w}_{\text{out}}$ (3 entries).

7.0.3.0.6 Partial effects

Let's take a look at the partial effects plots for both of these varialbles. A partial effect plot shows how the prediction changes as one predictor varies while the other is held constant (here at its median). Each variable runs from its 1st to 99th percentile, so we aren't plotting where there's almost no data.

```
import numpy as np
import matplotlib.pyplot as plt

models = {
    "No hidden layers": nn0,
    "One hidden layer (4)": nn1,
    "Two hidden layers (4, 3)": nn2,
}
colors = ["#2a78d6", "#eb6834", "#1baf7a"]
labels = {
    "running_experience_months": "Running experience (months)",
    "weekly_mileage_km": "Weekly mileage (km)",
}

def partial_effect(model, variable):
    # every row starts with each predictor at its median...
    grid = pd.DataFrame({p: [X_train[p].median()] * 100 for p in predictors})
    # ...then the variable of interest sweeps across its range
    grid[variable] = np.linspace(X_train[variable].quantile(0.01),
                                 X_train[variable].quantile(0.99), 100)
```

```
    return grid[variable], predict_minutes(model, x_scaler.transform(grid))

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5), sharey=True)
for ax, variable in zip(axes, predictors):
    for (name, model), color in zip(models.items(), colors):
        x, pred = partial_effect(model, variable)
        ax.plot(x, pred, color=color, linewidth=2, label=name)
    other = [labels[p] for p in predictors if p != variable][0]
    ax.set(xlabel=labels[variable], title=f"{other} held at its median")
    ax.spines[["top", "right"]].set_visible(False)
    ax.grid(alpha=0.3)
axes[0].set_ylabel("Predicted finish time (minutes)")
axes[0].legend(frameon=False)
fig.tight_layout()
plt.show()
```

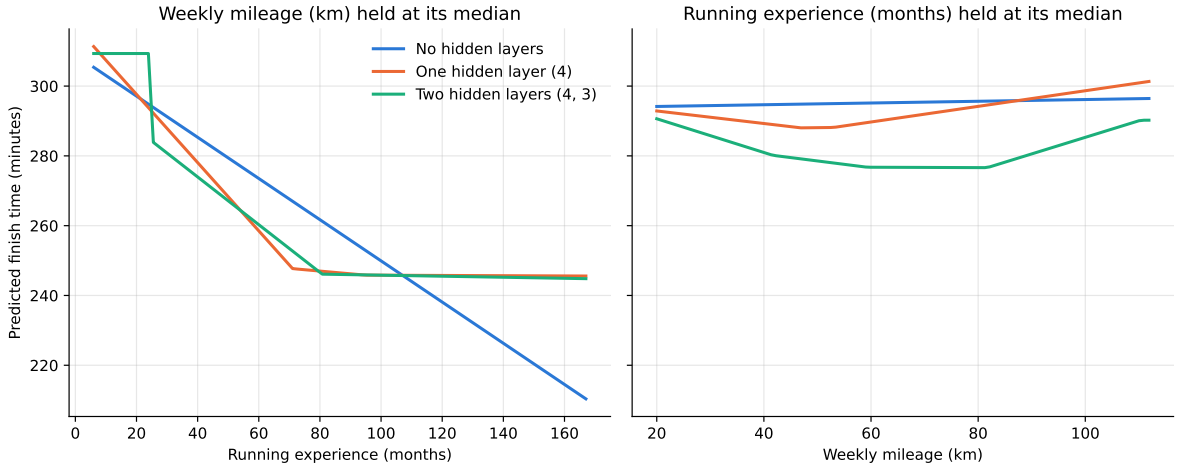


Figure 7.6: Partial effects of each predictor for the three networks. Left: running experience varies with weekly mileage held at its median. Right: weekly mileage varies with running experience held at its median.

From the partial effects plots we can observe a handful of things.

1. No Hidden Layers model only shows linear relationships.
2. Across all 3 models, Running Experience is the more influential feature in prediction.
3. The two hidden layers model starts to uncover progressively more information about the relationships.

7.0.3.0.7 Comparing the models

There was a little bit of weirdness when applying the random seed value for the gradient descent starting weights. In one case, where I used my favorite number 17, the RMSE for the third model (2 hidden layers) was significantly higher than the first two models and the partial effects plots were totally flat for both lines. So in the code below, we're going to train the networks with 5 different randomized starting positions to check how the RMSE may converge over multiple runs.

```
seed_results = []
for name, layers in [("No hidden layers", ()),
                     ("One hidden layer (4)", (4,)),
                     ("Two hidden layers (4, 3)", (4, 3))]:
    for seed in range(5):
        model = make_network(layers, seed=seed).fit(X_train_s, y_train_s)
        seed_results.append({
            "Model": name,
            "Test RMSE": root_mean_squared_error(y_test, predict_minutes(model, X_test_s)),
        })

pd.DataFrame(seed_results).groupby("Model", sort=False)["Test RMSE"].agg(["mean", "min", "ma
```

Model	mean	min	max
No hidden layers	25.58	25.55	25.61
One hidden layer (4)	23.34	23.21	23.44
Two hidden layers (4, 3)	22.98	22.51	23.29

We see that the RMSE is smaller on average for the more complex model. That's not necessarily saying we must use the more complex model, but it is the behavior we expect from these models.

7.0.3.1 The curious case of seed 17…

Check this out. Maybe gradient descent doesn't like my favorite number :(

```
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import root_mean_squared_error

def make_network(hidden_layers, seed=17):
```

```
    return MLPRegressor(
        hidden_layer_sizes=hidden_layers,  # neurons in each hidden layer
        activation="relu",                 # sigma(z) = max(0, z)
        solver="sgd",                      # (stochastic) gradient descent
        learning_rate_init=0.01,           # eta, the learning rate
        max_iter=100,                      # maximum number of epochs
        batch_size=64,                     # observations per gradient update
        alpha=0,                           # no penalty, so the loss is just MSE
        random_state=seed,                 # which random starting weights to use
    )

def predict_minutes(model, X_s):
    # the network predicts standardized y, so convert back to minutes
    y_s = model.predict(X_s)
    return y_scaler.inverse_transform(y_s.reshape(-1, 1)).ravel()
```

```
nn0 = make_network(hidden_layers=()).fit(X_train_s, y_train_s)

print("w_out:", nn0.coefs_[0].ravel(), " b_out:", nn0.intercepts_[0])

# compare to ordinary linear regression on the same data
from sklearn.linear_model import LinearRegression
lm = LinearRegression().fit(X_train_s, y_train_s)
print("linear regression coefficients:", lm.coef_)
print("linear regression intercept:", lm.intercept_)

print("Neural Network test RMSE (minutes):", root_mean_squared_error(y_test, predict_minutes
```

```
w_out: [-0.64672594  0.01073962]  b_out: [-0.00929721]
linear regression coefficients: [-0.64664405 -0.02330685]
linear regression intercept: -1.9109609522985936e-17
Neural Network test RMSE (minutes): 25.587313814563995
```

```
nn1 = make_network(hidden_layers=(4,)).fit(X_train_s, y_train_s)

# coefs_ holds the weight matrices, intercepts_ holds the bias vectors
print("weight shapes:", [w.shape for w in nn1.coefs_])
print("bias shapes:  ", [b.shape for b in nn1.intercepts_])

print("Test RMSE (minutes):", root_mean_squared_error(y_test, predict_minutes(nn1, X_test_s)
```

```
weight shapes: [(2, 4), (4, 1)]
bias shapes:   [(4,), (1,)]
Test RMSE (minutes): 23.309913328762505
```

```
nn2 = make_network(hidden_layers=(4, 3)).fit(X_train_s, y_train_s)

print("weight shapes:", [w.shape for w in nn2.coefs_])
print("Test RMSE (minutes):", root_mean_squared_error(y_test, predict_minutes(nn2, X_test_s)))
```

```
weight shapes: [(2, 4), (4, 3), (3, 1)]
Test RMSE (minutes): 33.69433433220708
```

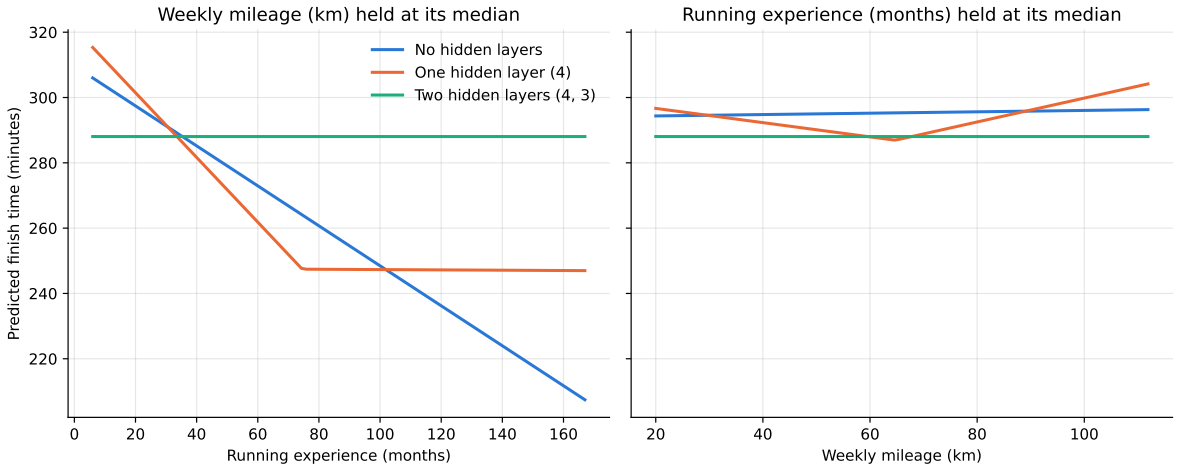
```
import numpy as np
import matplotlib.pyplot as plt

models = {
    "No hidden layers": nn0,
    "One hidden layer (4)": nn1,
    "Two hidden layers (4, 3)": nn2,
}
colors = ["#2a78d6", "#eb6834", "#1baf7a"]
labels = {
    "running_experience_months": "Running experience (months)",
    "weekly_mileage_km": "Weekly mileage (km)",
}

def partial_effect(model, variable):
    # every row starts with each predictor at its median...
    grid = pd.DataFrame({p: [X_train[p].median()] * 100 for p in predictors})
    # ...then the variable of interest sweeps across its range
    grid[variable] = np.linspace(X_train[variable].quantile(0.01),
                                 X_train[variable].quantile(0.99), 100)
    return grid[variable], predict_minutes(model, x_scaler.transform(grid))

fig, axes = plt.subplots(1, 2, figsize=(11, 4.5), sharey=True)
for ax, variable in zip(axes, predictors):
    for (name, model), color in zip(models.items(), colors):
        x, pred = partial_effect(model, variable)
        ax.plot(x, pred, color=color, linewidth=2, label=name)
    other = [labels[p] for p in predictors if p != variable][0]
    ax.set(xlabel=labels[variable], title=f"{other} held at its median")
    ax.spines[["top", "right"]].set_visible(False)
```

```
    ax.grid(alpha=0.3)
axes[0].set_ylabel("Predicted finish time (minutes)")
axes[0].legend(frameon=False)
fig.tight_layout()
plt.show()
```



7.0.4 Sources:

1. Tom Stewart, Intro to Prediction Modeling: https://tgstewart.cloud/ipm/models-overview.html#sec-nnet
2. 3Blue1Brown, Neural Networks: https://youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_3pi&si=eTLONIqIN-zzii3t
3. McCulloch, W. S. & Pitts, W. (1943), A Logical Calculus of the Ideas Immanent in Nervous Activity, *Bulletin of Mathematical Biophysics* 5: 115-133
4. Rosenblatt, F. (1958), The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain, *Psychological Review* 65(6): 386-408
5. Rumelhart, D. E., Hinton, G. E. & Williams, R. J. (1986), Learning Representations by Back-Propagating Errors, *Nature* 323: 533-536
6. Kaggle Run Club Marathon Performance Dataset: https://www.kaggle.com/datasets/likithagedipudi/run-club-marathon-performance-dataset/data

7.0.5 Future Reference:

1. http://neuralnetworksanddeeplearning.com/

Part IV

04: Linear Algebra Review

8 Appendix A

Review the material in Appendix A of *Plane Answers to Complex Questions: The Theory of Linear Models* by Ronald Christensen.

8.1 Self-assessment

To help focus class time, rate each topic below. Use no stars for no familiarity. Use for high familiarity.

Topic	Comfort level ( )
Vector spaces	
Subspaces	
Linear combinations	
Span	
Column space $C(A)$	
Null space $N(A)$	
Linear dependence and independence	
Bases	
Rank	
Uniqueness in basis	
Sums of subspaces (direct sums)	
Euclidean inner product	
Orthogonality	
Norm and distance	
Orthogonal and orthonormal bases	
Gram–Schmidt	
Orthogonal complements	
Subspace plus orthogonal complement	
Orthogonal complement of an intersection	

8.2 Overall assessment:

The last time I did any problems like this was freshman year of college (9 years ago), so I am not initially confident in my ability to solve any of the problems. However, with some practice and refreshers, I believe I could relearn the concepts in due time. Many of the words and concepts are familiar in words, but not so much in application and thorough understanding.

9 Appendix B

Review the material in Appendix B of *Plane Answers to Complex Questions: The Theory of Linear Models* by Ronald Christensen.

9.1 Self-assessment

To help focus class time, rate each topic below. Use no stars for no familiarity. Use for high familiarity.

Topic	Comfort level ( )
Square, symmetric, diagonal, and identity matrices	
Transpose	
Partitioned (block) matrices	
Kronecker products	
The Vec operator	
Nonsingular and singular matrices	
Matrix inverse	
Rank of null space $N(A)$	
Eigenvalues, eigenvectors, and multiplicity	
Eigenspaces as subspaces	
For symmetric matrices, eigenvectors of different eigenvalues are orthogonal	
A basis for $C(A)$ made of eigenvectors; rank as the number of nonzero eigenvalues	
Orthonormal eigenvector basis for symmetric matrices (spectral theorem)	
Orthonormal (orthogonal) matrices	
Spectral or singular value decomposition: $A = PD(\lambda_i)P'$	
Positive definite and nonnegative definite matrices	
$A = QQ'$	
Trace, and the cyclic property $\mathrm{tr}(AB) = \mathrm{tr}(BA)$	
Trace as the sum of eigenvalues	
Determinant as the product of eigenvalues	
Complex eigenvalues	
Perpendicular projection operators (ppo)	

Topic	Comfort level ( )
Characterizing a ppo: idempotent and symmetric ($MM = M$, $M' = M$)	
Why a ppo is unique	
Building a ppo from an orthonormal basis: $M = OO'$	
Generalized inverses	
Projection matrix $X(X'X)^{-}X'$	
Sums and differences of projection operators; orthogonal complements within a subspace	
$I - M$ as the projection onto the orthogonal complement	
How projections relate to Gram–Schmidt	
Eigenvalues of projection matrices (0 and 1), and $\mathrm{tr}(M) = r(M)$	
Idempotent matrices	
$C(XX') = C(X)$ and $r(XX') = r(X)$	

9.2 Overall assessment:

Like with the Appendix A material on Vector Spaces, the last time I did any problems like this was during college (6-9 years ago depending on the class), so I am not initially confident in my ability to solve any of the problems. However, with some practice and refreshers, I believe I could relearn the concepts in due time. Many of the words and concepts are familiar in words, but not so much in application and thorough understanding.

Part V

05: Nonlinearity

10 Explainer

10.1 The Linear Model

Let's go to where it all starts for many of us, y = mx+b.

This is probably what initially pops into most people's head when they think of a linear model.

We put in some value for x that we are given or observe, then multiply that by a value of m, which we commonly call the slope, and then we add an intercept term, b, in order to get our output value of y.

If we repeatedly do this, we can get a bunch of values of y, and we could plot a general line for this relationship between x and y, or inputs and outputs.

This y=mx+b is a model, with the parameters of m and b. In the following discussion, we will talk about what it means to use different parameters for our models with the same inputs and outputs along with a slightly different and much expanded idea of a linear model than the one our teachers illustrated in school.

The linear model that we are exploring here is the linear regression model with the following components. Y is the scalar outcome, and x is a vector of predictor values for that outcome.

It has a conditional mean component, which can be expressed as

$$E[Y \mid \mathbf{x}] = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = \beta_0 + \sum_{j=1}^{p} x_j \beta_j = \sum_{j=0}^{p} x_j \beta_j = \mathbf{x}'$$

where we set x0=1 to absorb the intercept into the dot product.

It also has a conditional variance component

$$\text{Var}(Y \mid \mathbf{x}) = \sigma^2$$

Which leads to the following conditional distribution

$$Y \mid \mathbf{x} \sim \text{Normal}(\mathbf{x}',\ \sigma^2)$$

Now in contrast to the super simple model we started with and then this scalar model we outlined above, we typically work with n observations simultaneously, so it is better for us to

work with matrix notation instead. We let Y be the nx1 vector of outcomes and X be the nx(p+1) design matrix.

Now our notation for the model is

$$Y = X\beta + \varepsilon, \qquad \varepsilon \sim \text{Normal}(\mathbf{0}_n,\ \sigma^2 I_n)$$

Y is an nx1 vector of outcomes, where Yi is the outcome for observation i X is the nx(p+1) design matrix, where the first column is all ones for the intercept; Xij is the j-th predictor value for observation i B is the p+1 x 1 coefficient vector, B0 is the intercept, Bj is the coefficient for the j-th predictor sigma^2 is a scalar for the conditional variance epsilon is an nx1 error vector 0n is a vector of all zeroes of length n In is the identity matrix of size n

In the scalar form, you might write

$$y_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} + \varepsilon_i, \qquad \varepsilon_i \overset{\text{iid}}{\sim} \text{Normal}(0,\ \sigma^2)$$

We have four important assumptions to track here 1. Linearity in parameters: E[Y|x] is linear in B. 2. Independence of errors: epsilon_i are independent across observations 3. Homoscedasticity: Var(epsilon_i) = sigma^2 for all i 4. Normality of errors; epsilon_i are normally distributed with mean 0 and variance sigma^2

10.2 Nonlinearity

We just worked through how we might approach a model if the relationship between our input and output were linear. However, we can think of plenty of situations where the relationship we're trying to model is not linear.

The question then is, can we apply the model we've outlined in some way to model this other relationship? And lucky for us the answer is yes.

We get to do some math that changes how we set the parameters for our model, so that we can model the nonlinear relationship. The general idea of how we do this is called basis expansion.

One key note to point out is that the linear model means that we have linearity in the parameters, not necessarily the predictors. In the examples below, we have a linear relationship in B in both equations, but a non linear relationship with x in the second equation. However, if we were to treat x and x^2 as two separate predictors, then the model is a standard linear model wiht two columns in X, the design matrix.

That idea of treating things as separate predictors and replacing it with a set of derived features is called basis expansion.

10.2.1 Basis expansion

The generalized formula for basis expansion is below,

$$E[Y \mid x] = \beta_0 + \sum_{k=1}^{K} \beta_k \, b_k(x)$$

where $[b_0(x), \ldots, b_K(x)]$ represent the set of derived features or basis expansion of predictor x. These are chosen so that a linear combination of these can approximate nonlinear functions of x. The model is still linear in the beta parameters, so we can still solve the normal equations.

In the following sections, we will get an overview of different types of basis expansion models. First, let's generate and load in some data to work with.

10.3 Make and read in some data

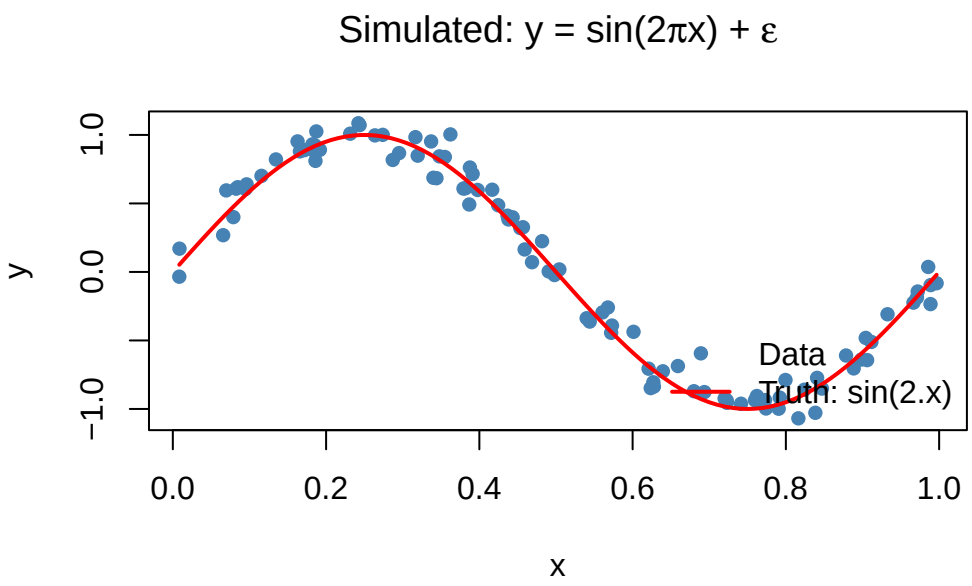
For the generated data, we'll make 100 observations from the known function $f(x) = \sin(2\pi x)$, adding small Gaussian noise. Because we know the truth, we can directly evaluate how well each basis expansion recovers it.

```
`%||%` <- function(a,b) paste0(a,b)

set.seed(7)
n    <- 100
x    <- runif(n)
y    <- sin(2 * pi * x) + rnorm(n, 0, 0.1)
sim  <- data.frame(x = x, y = y)

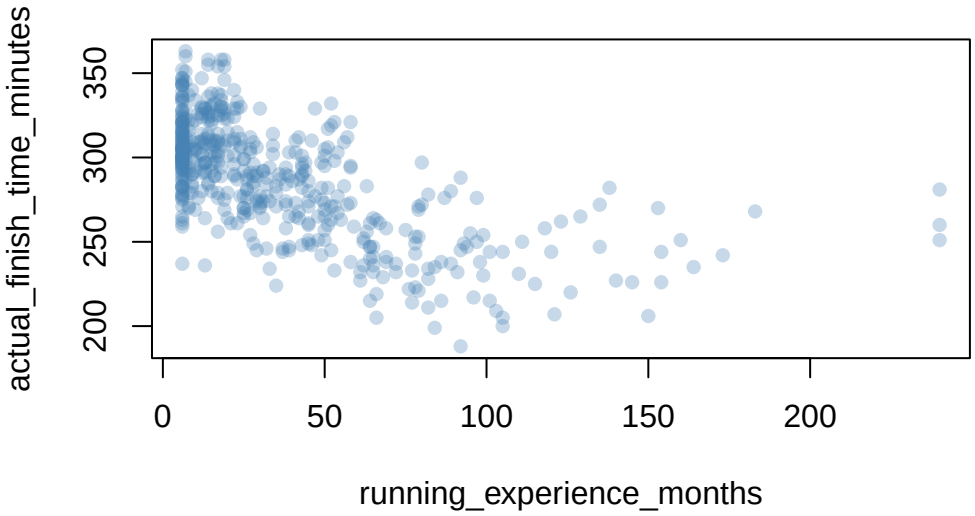
aq <- na.omit(airquality[, c("Ozone", "Temp", "Wind", "Solar.R")])
```

```
plot(sim$x, sim$y, pch = 16, col = "steelblue",
     xlab = "x", ylab = "y",
     main = expression("Simulated: y = sin(2" * pi * "x) + " * epsilon))
curve(sin(2 * pi * x), add = TRUE, col = "red", lwd = 2)
legend("bottomright",
       legend = c("Data", "Truth: sin(2 x)"),
       col    = c("steelblue", "red"),
       pch    = c(16, NA), lty = c(NA, 1), lwd = c(NA, 2),
       bty    = "n")
```



For the "real" data, we'll explore a simulated Running Performance dataset from Kaggle, and explore an outcome of `actual_finish_time_minutes` with predictor `running_experience_months`.

```
plot(actual_finish_time_minutes ~ running_experience_months, data = demo_data,
     pch = 16, col = adjustcolor("steelblue", 0.3))
```



10.3.0.1 Polynomials

As we mentioned briefly earlier when we introduced x^2 to the linear model, the easiest way to conceive of basis expansion models is with the polynomial.

Let's consider the general form with polynomial of degree d. The basis expansion follows this form:

which produces a model that looks like this:

We maintain linearity in our beta parameters.

10.3.0.2 A separate exploration (in-class live code)

```
## need to drop the NAs to make sure the plots work
d1 <- demo_data %>% select(actual_finish_time_minutes, running_experience_months) %>% drop_na

plot(d1[,2:1])

lm1 <- lm(actual_finish_time_minutes ~ running_experience_months, data = d1)
yhat <- predict(lm1)
```

```
points(d1$running_experience_months, yhat, col="red")

lm2 <- lm(actual_finish_time_minutes ~ poly(running_experience_months, 2), data = d1)
yhat2 <- predict(lm2)

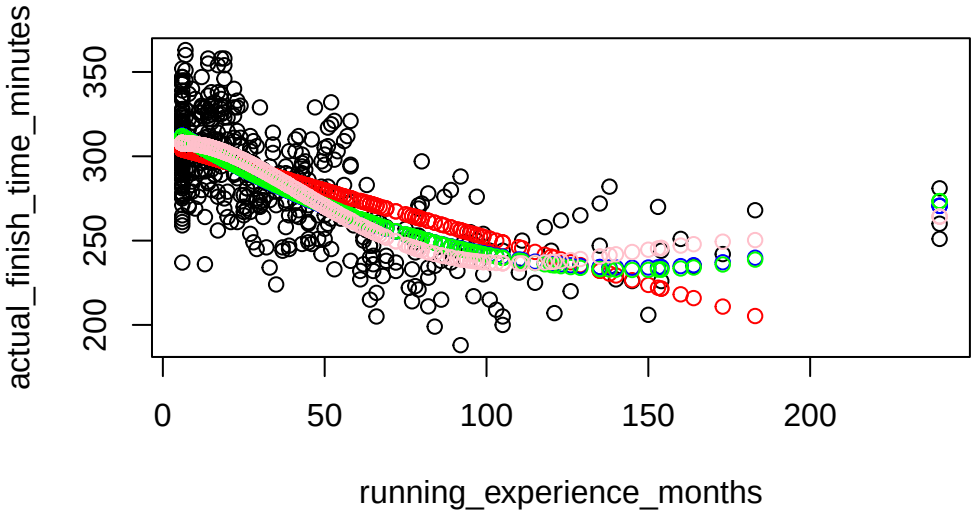
points(d1$running_experience_months, yhat2, col="blue")

lm3 <- lm(actual_finish_time_minutes ~ poly(running_experience_months, 3), data = d1)
yhat3 <- predict(lm3)

points(d1$running_experience_months, yhat3, col="green")


lmN <- lm(actual_finish_time_minutes ~ poly(running_experience_months, 5), data = d1)
yhatN <- predict(lmN)

points(d1$running_experience_months, yhatN, col="pink")
```



Part VI

06: Regularization

11 Regularization

11.1 Homework Problems

From https://tgstewart.cloud/ipm/regularization.html#module-8-homework

In this homework you will work with the riboflavin dataset from the `hdi` package: 71 samples of *B. subtilis*, each with 4,088 gene expression features. The outcome is riboflavin (vitamin B2) production rate. Because $p \gg n$, ordinary least squares fails — this is the setting where regularization is not optional.

11.1.1 Preliminary: install required packages

This only installs packages that are missing, so it is safe to leave in the book. Note that `hdi` (and its dependency `scalreg`) were archived from CRAN in 2026, so they are installed from the CRAN archive as source packages.

```
cran <- "https://cloud.r-project.org"

for (p in c("glmnet", "pls", "lars", "linprog", "MASS")) {
  if (!requireNamespace(p, quietly = TRUE)) install.packages(p, repos = cran)
}

archive <- paste0(cran, "/src/contrib/Archive/")
if (!requireNamespace("scalreg", quietly = TRUE)) {
  install.packages(paste0(archive, "scalreg/scalreg_1.0.1.tar.gz"),
                   repos = NULL, type = "source")
}
if (!requireNamespace("hdi", quietly = TRUE)) {
  install.packages(paste0(archive, "hdi/hdi_0.1-10.tar.gz"),
                   repos = NULL, type = "source")
}
```

11.1.2 Setup

```
library(hdi)
library(glmnet)
library(pls)

data(riboflavin)
X <- riboflavin$x   # 71 × 4088 matrix
y <- riboflavin$y   # length-71 vector
```

11.1.3 Problem 0

Create `HW7.qmd` in your Quarto book and add it to `_quarto.yml`.

Done: this chapter (`06_Homework.qmd`) is listed in `_quarto.yml` under "06: Regularization".

11.1.4 Problem 1: The Problem

Confirm that OLS cannot create a unique solution for this dataset. Attempt to fit a linear model using `lm()` and report what happens. How many coefficients does R return as `NA`, and why?

```
set.seed(17)

model1 <- lm(y ~ X)
#summary(model1)
sum(is.na(coef(model1)))
```

```
[1] 4018
```

Answer:

In the output above, we see that 4018 coefficients are returned as `NA` and the summary cites "because of singularities". In order to still achieve a solution, R drops those 4018 predictors where it didn't return a coefficient. Because we don't have uniqueness in the solution and our standard matrix inverse for X^TX does not exist, we have a model produced here but we may not want to proceed with it.

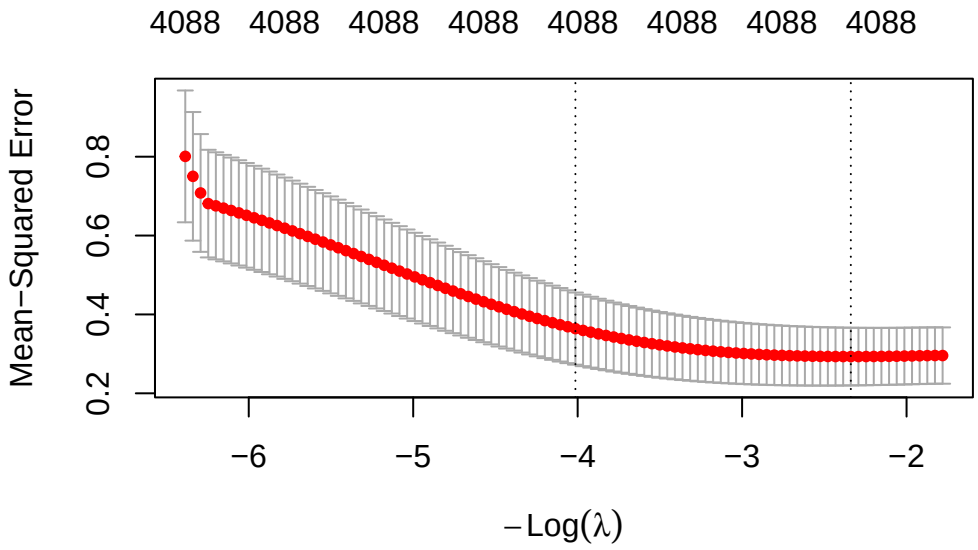
CORRECTION: We used some generalized inverse to get an answer here! Under the hood, R has picked some kind of approach in order to get a solution, maybe the QR with pivots or the SVD.

11.1.5 Problem 2: Ridge

Fit ridge regression using `glmnet` (`alpha = 0`). Select λ via 10-fold cross-validation. Report λ_{min} and λ_{1se}. Plot the CV error curve with the two vertical lines. How does CV-MSE change as λ increases?

```
set.seed(17)
#Xs <- scale(X)
#ys <- scale(y)

cv_ridge <- cv.glmnet(X, y, alpha = 0, nfolds = 10)
plot(cv_ridge)
```




```
c(`lambda.min` = round(cv_ridge$lambda.min,4), `lambda.1se` = round(cv_ridge$lambda.1se, 4))
```

```
lambda.min lambda.1se
   10.3701    55.3422
```

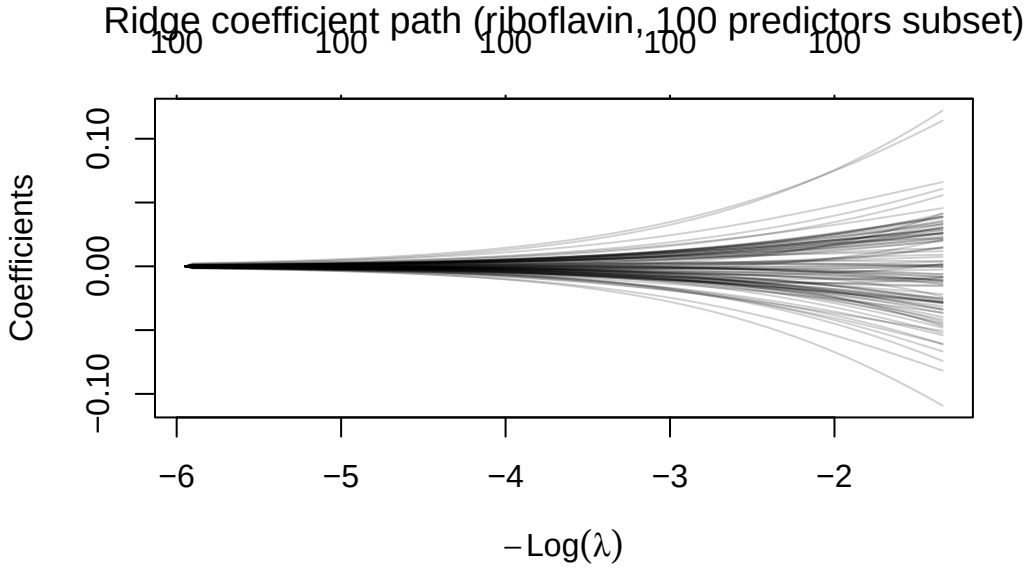
Answer:

As λ increases, the CV-MSE also increases. We can see on the plot that λ_{min} does not provide a significant improvement in our model error compared to λ_{1se}. For that reason, we'll use the λ_{1se} as our selected value.

11.1.6 Problem 3: Ridge

Plot the ridge coefficient path ($-\log(\lambda)$ on the x-axis, coefficient values on the y-axis). Describe qualitatively what happens to the coefficients as λ increases. Does any coefficient reach exactly zero?

```
fit_ridge <- glmnet(riboflavin$x[,1:100], riboflavin$y, alpha = 0)
plot(fit_ridge, xvar = "lambda", label = FALSE,
     main = "Ridge coefficient path (riboflavin, 100 predictors subset)", col = "#00000030",
```



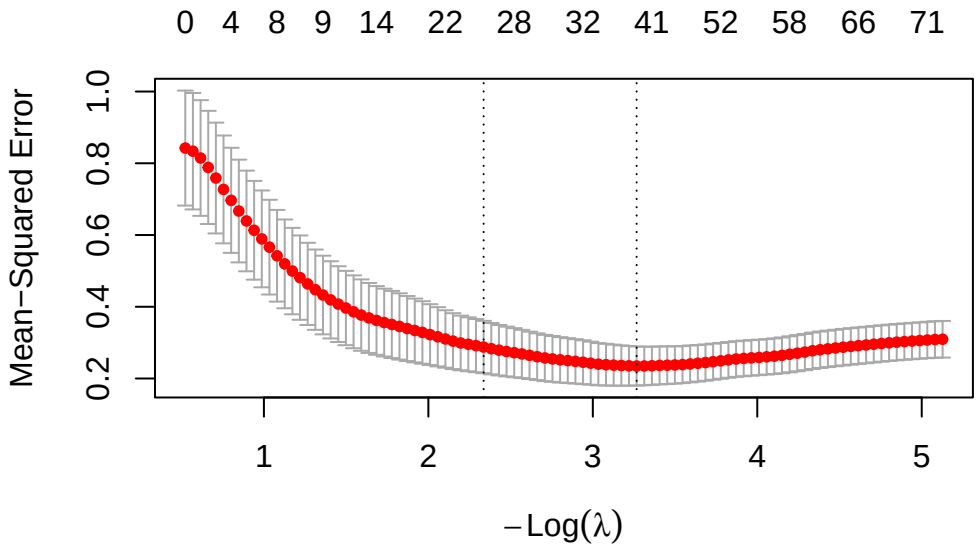
Answer:

Note the x-axis is $-Log(\lambda)$ again, so large values of λ are on the left. So we can see that as λ increases, all the coefficients shrink toward zero, but none of them ever actually reach exactly zero. This is an important property to note that Ridge Regression shrinks the coefficients, but it does not actually perform variable selection.

11.1.7 Problem 4: Lasso

Fit lasso regression (`alpha = 1`) and select λ via 10-fold CV. At λ_{1se}, how many gene expression features are retained (non-zero coefficients)? Plot the coefficient path and the CV error curve.

```r
set.seed(17)
cv_lasso <- cv.glmnet(riboflavin$x, riboflavin$y, alpha = 1, nfolds = 10)
plot(cv_lasso)
```




```r
c(`lambda.min` = round(cv_lasso$lambda.min,4), `lambda.1se` = round(cv_lasso$lambda.1se, 4))
```

```
lambda.min lambda.1se
    0.0381     0.0967
```

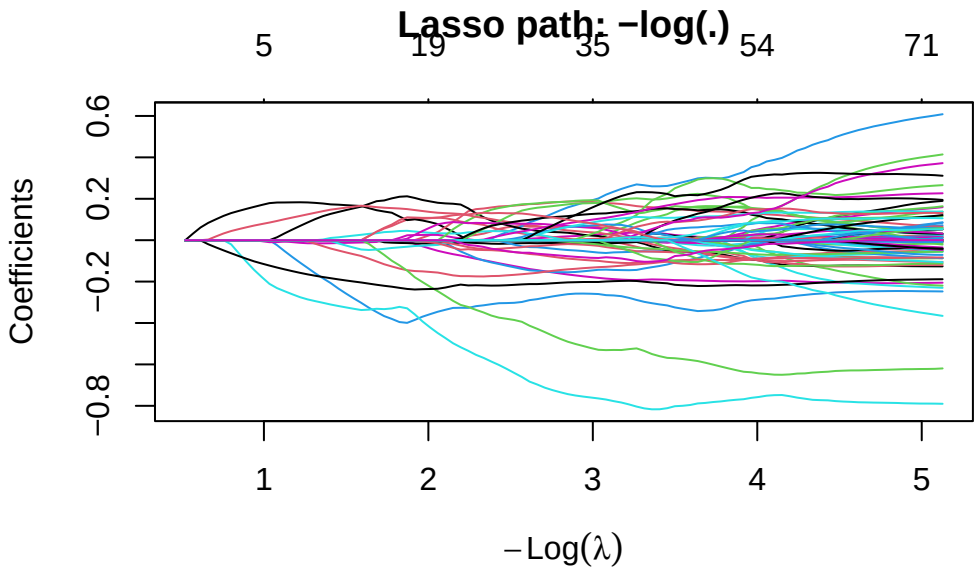
We again select the λ_{1se} based on how it is not significantly different from λ_{min} in reducing the error.

```r
n_selected <- sum(coef(cv_lasso, s = "lambda.1se") != 0) - 1  # subtract intercept
c(`Non-zero gene expression features of 4,088 at lambda.1se` = n_selected)
```

```
Non-zero gene expression features of 4,088 at lambda.1se
                                                      24
```

This selection of λ keeps only a small fraction of the features we had originally. We are now working with just 24 features of the original 4,088.

```r
fit_lasso <- glmnet(riboflavin$x, riboflavin$y, alpha = 1)
plot(fit_lasso, xvar = "lambda", label = FALSE,
     main = "Lasso path: -log( )")
```



Answer:

Finally, we see the coefficient path for Lasso with our selected λ. Note that Lasso does perform variable selection, so all coefficients are equal to zero initially, and they are introduced to the model as λ is reduced (moving to the right on our graph).

11.1.8 Problem 5: Elastic Net

Fit an elastic net with $\alpha \in \{0.25, 0.5, 0.75\}$. For each α, record the minimum CV-MSE. Create a small table showing (α, $\lambda_{\min}$, min CV-MSE). Which α yields the lowest CV-MSE? How does the number of selected features compare to lasso?

```r
set.seed(17)
alphas <- c(0.25, 0.5, 0.75)
cv_results <- lapply(alphas, function(a) {
  set.seed(17)
  cv <- cv.glmnet(riboflavin$x, riboflavin$y, alpha = a, nfolds = 10)
  n_selected <- sum(coef(cv, s = "lambda.min") != 0) - 1  # subtract intercept
```

```
  list(alpha = a, lambda.min = cv$lambda.min, cvm.min = min(cv$cvm), features = n_selected)
})

result_df <- do.call(rbind, lapply(cv_results, function(r)
  data.frame(alpha = r$alpha,
             lambda.min = round(r$lambda.min, 4),
             min_CV_MSE = round(r$cvm.min, 4),
             num_features = r$features)))
print(result_df)
```

```
  alpha lambda.min min_CV_MSE num_features
1  0.25     0.1838     0.2552           72
2  0.50     0.0837     0.2488           55
3  0.75     0.0485     0.2421           47
```

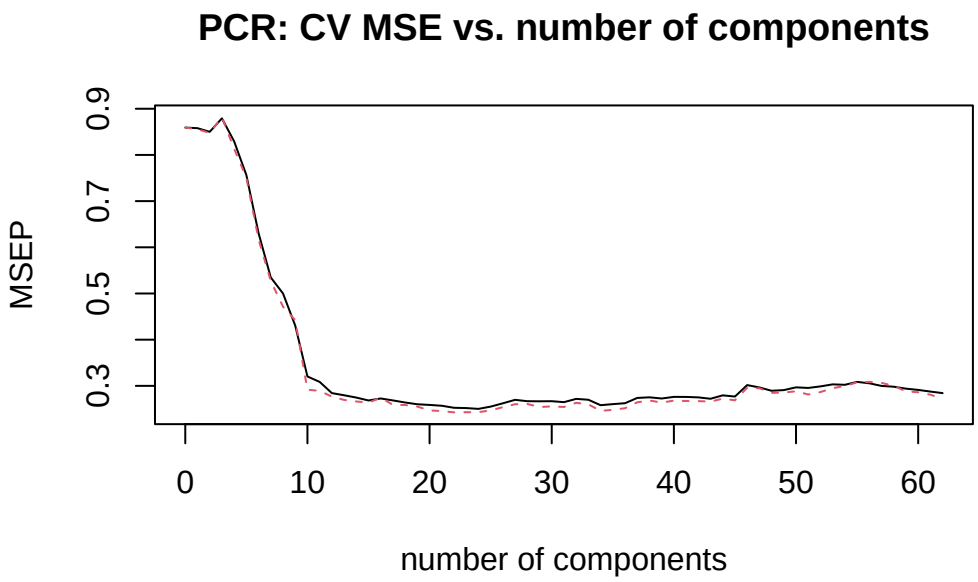
Answer:

We see in the table above that the α value of 0.75 produces the lowest CV-MSE. We also see that the number of selected features decreases as α increases. That's the expected behavior, because a value of $\alpha = 1$ would be pure lasso, and we would expect a model with fewer features as above.

11.1.9 Problem 6: PCR

Fit PCR using the `pls` package (or manual SVD as shown in the chapter). Select the number of components k via cross-validation (use `validationplot`). Report the optimal k.

```
fit_pcr <- pcr(y ~ x, data = riboflavin, scale = TRUE, validation = "CV")
validationplot(fit_pcr, val.type = "MSE",
               main = "PCR: CV MSE vs. number of components")
```




```r
# Identify optimal k
msep_vals <- MSEP(fit_pcr)$val[1, 1, ]
k_opt <- which.min(msep_vals) - 1  # subtract 1 for intercept-only model at position 1
c(`Optimal number of components` = k_opt)
```

```
Optimal number of components.24 comps
                                   24
```

Answer:

The optimal k is shown above.

a. Back-transform the PCR fit to obtain the full 4,088-dimensional coefficient vector $\hat{\beta}_{\text{PCR}}$.

b. Use $\hat{\beta}_{\text{PCR}}$ to generate a prediction for the first observation in the dataset (`X[1, ]`). Show your R code and the predicted value.

Answer:

c. In one sentence, explain why you needed all 4,088 gene expression values to generate this prediction even though you only fit k principal components.

Answer:

We need all 4,088 gene expression values to generate this prediction because each of the k principal components is a linear combination of all of the features. PCR is not a data reduction technique, it just reduces the complexity of the model, not the dimensionality of the data required for prediction.

11.1.10 Problem 7: Comparison

Comparing across methods:

a. Report the CV-MSE at the selected tuning parameter for each of the four methods (ridge, lasso, elastic net, PCR).

Answer:

b. Which method achieves the lowest CV-MSE on this dataset?

Answer:

c. Based on the guidance in the chapter, would you expect ridge or lasso to work better here? Does the data agree with your expectation? Why or why not?

Answer:

I expect that ridge would reduce the CV-MSE further than lasso, but the lasso will provide a more interpretable and smaller model in terms of the number of predictors retained. I think the CV-MSE will be further reduced when you keep all the coefficients from ridge, but lasso will ultimately be a better approach for us.

12 Paper: Lasso's Victory Lap

Paper: Tibshirani, R. (2011). "Regression shrinkage and selection via the lasso: a retrospective." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 73(3), 273–282. link

A victory lap, reflection, and some additional ideas about the original Lasso Paper 1996.

Key points to highlight: 1. Sparsity - And the extended applications from the table 2. L1 and Convex idea - contrast with Ridge 3. Why do you do relaxed lasso? - 'S' stands for screening, not selection! 4. Critiques - Bayesian - Correlated Predictors 5. Response - Known gap is inference

12.1 1. Sparsity

A model is **sparse** when most of its coefficients are exactly zero. Write the *active set* as $S = \{j : \beta_j \neq 0\}$ with size $s = |S|$. Sparsity means $s \ll p$: only a few predictors actually matter.

The lasso is least squares with an ℓ_1 penalty (details in Section 2), and its defining feature is that it returns sparse solutions. It sets some coefficients **exactly to zero**, so shrinkage and variable selection happen in one step. For a single standardized predictor whose least-squares coefficient is z, the lasso estimate is

$$\hat{\beta} = \text{sign}(z)\,(|z| - \lambda)_+,$$

so any coefficient with $|z| \leq \lambda$ is set to 0, not just made small.

Why sparsity matters. One of Tibshirani's explanations for the slow uptake of the 1996 paper is that "the statistical and numerical advantages of sparsity were not immediately apparent." Those advantages are:

- **Statistical:** a sparse model is easier to interpret (you can name the predictors that matter) and estimates fewer parameters, so it has lower variance. Sparsity is also what makes $p \gg n$ workable. Bühlmann's discussion shows the lasso's prediction error grows only like $\sqrt{\log p/n}$ when the true β is sparse.
- **Numerical:** algorithms can exploit sparsity. In coordinate descent, a coefficient that is 0 and stays 0 costs almost nothing to update (see the Computation notes below).

12.1.1 Extended applications (Table 1)

The paper's Table 1 shows that the ℓ_1 idea carries over to other problems. Each method below puts an ℓ_1-type penalty on a *different object*, so a different thing becomes sparse:

Method	Reference	Penalty / problem	What gets set to zero
Group lasso	Yuan & Lin (2007a)	$\sum_g \|\beta_g\|_2$	Whole groups of coefficients (e.g., all dummies for one factor)
Elastic net	Zou & Hastie (2005)	$\lambda_1 \sum_j \|\beta_j\| + \lambda_2 \sum_j \beta_j^2$	Individual coefficients, while correlated predictors share weight
Fused lasso	Tibshirani et al. (2005)	$\lambda \sum_j \|\beta_{j+1} - \beta_j\|$	*Differences* between neighboring coefficients, giving piecewise-constant fits
Adaptive lasso	Zou (2006)	$\lambda \sum_j w_j \|\beta_j\|$	Individual coefficients, with data-driven weights (less penalty on large effects)
Graphical lasso	Yuan & Lin (2007b); Friedman et al. (2007)	$\text{loglik} - \lambda \|\Sigma^{-1}\|_1$	Entries of the inverse covariance, i.e., *edges* in a graph
Dantzig selector	Candès & Tao (2007)	$\min \|X^\top (y - X\beta)\|_\infty$ s.t. $\|\beta\|_1 \le t$	Individual coefficients (an alternative ℓ_1 estimator)
Near-isotonic regression	Tibshirani et al. (2011)	$\sum_j (\beta_j - \beta_{j+1})_+$	*Decreases* in a sequence (relaxing monotonicity)
Matrix completion	Candès & Tao (2009); Mazumder et al. (2010)	$\|X - \hat{X}\|^2 + \lambda \|\hat{X}\|_*$	*Singular values*, giving a low-rank matrix (Netflix-style missing entries)
Compressive sensing	Donoho (2004); Candès (2006)	$\min \|\beta\|_1$ s.t. $y = X\beta$	Signal coefficients, recovered from few measurements
Sparse multivariate methods	Jolliffe et al. (2003); Witten et al. (2009)	ℓ_1 on loadings	Loadings in PCA, LDA, CCA

12.2 2. The ℓ_1 penalty, convexity, and the contrast with ridge

Assume standardized predictors x_{ij} and a centered response y_i, for $i = 1, \dots, N$ and $j = 1, \dots, p$. The lasso solves

$$\hat{\beta}^{\text{lasso}} = \arg\min_{\beta} \sum_{i=1}^{N} \Big(y_i - \sum_{j=1}^{p} x_{ij}\beta_j \Big)^2 + \lambda \sum_{j=1}^{p} |\beta_j|.$$

The same problem can be written as a constraint:

$$\min_{\beta} \sum_{i=1}^{N} \Big(y_i - \sum_{j} x_{ij}\beta_j \Big)^2 \quad \text{subject to} \quad \sum_{j} |\beta_j| \le t.$$

Each λ matches some budget t.

Contrast with ridge. Ridge regression has the same form with a squared (ℓ_2) penalty, $\lambda \sum_j \beta_j^2$, or equivalently the constraint $\sum_j \beta_j^2 \le t$. That one change in the penalty changes the behavior:

	Ridge (ℓ_2)	Lasso (ℓ_1)
Penalty	$\lambda \sum_j \beta_j^2$	$\lambda \sum_j \lvert\beta_j\rvert$
Closed form?	yes: $(X^\top X + \lambda I)^{-1} X^\top y$	no (needs an algorithm; see Computation notes)
Shrinks coefficients?	yes	yes
Sets coefficients exactly to 0?	**no**, only *shrinks*	**yes**, *shrinks and selects*
Correlated predictors	spreads weight across them	tends to pick one (see Section 4)

Bridge family and convexity. Tibshirani puts both methods into the **bridge** family of penalties $\sum_j |\beta_j|^q$:

q	Method	Shrinks?	Selects (exact zeros)?	Convex?
0	Best-subset selection (penalty counts the nonzero coefficients)	no	yes	no
1	**Lasso**	yes	**yes**	**yes**

q	Method	Shrinks?	Selects (exact zeros)?	Convex?
2	Ridge regression	yes	no	yes

Key point: $q = 1$ is the smallest value of q, so the one closest to subset selection, that still gives a *convex* problem. Convexity is what makes the lasso cheap to compute: any local minimum is the global minimum, and fast algorithms are guaranteed to find it. Best subset is the "ideal" selector, but it is a combinatorial search over 2^p models. The lasso is the closest you can get to it while keeping the problem convex.

The figure below shows why the corners matter. The constraint regions $\sum_j |\beta_j|^q \leq 1$ have corners on the axes only when $q \leq 1$. A corner on an axis is a point where one coefficient is exactly zero. The RSS contours usually touch the lasso diamond first at a corner, which is where sparsity comes from. They touch the smooth ridge circle at a point where both coefficients are nonzero.

```
theta <- seq(0, 2 * pi, length.out = 800)
lq_ball <- function(q) {
  # points on the boundary of |b1|^q + |b2|^q = 1
  r <- (abs(cos(theta))^q + abs(sin(theta))^q)^(-1 / q)
  cbind(r * cos(theta), r * sin(theta))
}
qs   <- c(0.5, 1, 2, 4)
cols <- c("grey50", "firebrick", "steelblue", "darkgreen")
plot(NA, xlim = c(-1.1, 1.1), ylim = c(-1.1, 1.1), asp = 1,
     xlab = expression(beta[1]), ylab = expression(beta[2]))
abline(h = 0, v = 0, col = "grey85")
for (k in seq_along(qs)) lines(lq_ball(qs[k]), col = cols[k], lwd = 2)
legend("topright", legend = paste("q =", qs), col = cols, lwd = 2, bty = "n")
```

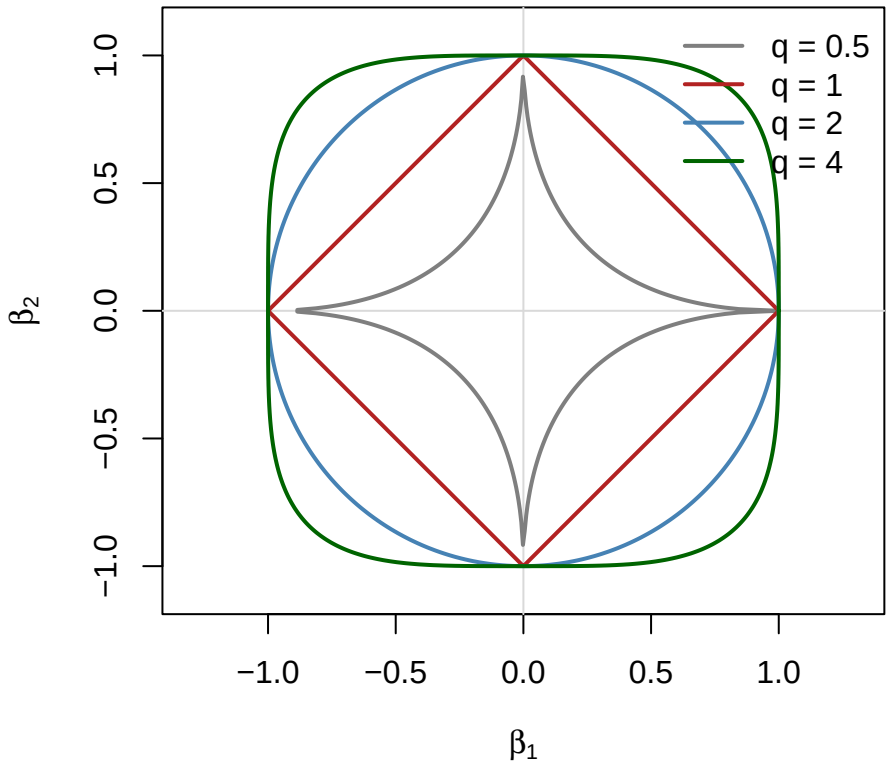


Figure 12.1: Unit 'balls' of the bridge penalty for several q. The lasso (q = 1) is the only convex region with corners on the axes, and the corners are what produce exact zeros.

12.3 3. Why do the relaxed lasso? The 'S' stands for screening, not selection

Bühlmann's point. Bühlmann's discussion summarizes the theory for $p \gg n$, with true active set $S_0 = \{j : \beta_{0j} \neq 0\}$. Each result needs stronger assumptions than the one before:

1. **Good prediction** needs almost no conditions on X, only that the true β is sparse.
2. **Accurate estimates of** β need a "restricted eigenvalue" or "compatibility" condition. Roughly, small groups of columns of X can't be too collinear.
3. **Recovering exactly the right variables**, $P(\hat{S} = S_0) \to 1$, needs much more: the *irrepresentable condition* (irrelevant predictors can't be too correlated with relevant ones)

plus a *"beta-min" condition* (every true nonzero effect must be large enough to detect). Bühlmann calls this "rather unlikely" in practice.

What *does* hold under the weaker conditions is the **screening property**:

$$\hat{S} \supseteq S_0 \quad \text{with high probability.}$$

The lasso keeps all the important variables, *plus some false positives*. So Bühlmann suggests reading the second "s" in lasso (least absolute shrinkage and **selection** operator) as **screening**. When the lasso picks 24 genes in the riboflavin homework, the safe reading is "the true genes are probably among these 24", not "these are the true genes".

Why that leads to a second stage. The lasso uses one λ for two jobs at once:

- **Selection:** a larger λ sets more coefficients to zero, so fewer false positives.
- **Shrinkage:** a larger λ also shrinks the coefficients it keeps, so more bias toward zero.

Cross-validation chooses λ for prediction. That usually means a fairly small λ, which lets in extra variables. You can't get a sparser model without also over-shrinking the real effects. Two-stage methods separate the two jobs.

The relaxed lasso (Meinshausen, 2007; formula not in this paper) first runs the lasso at λ to get the selected set $\mathcal{M}_\lambda$. It then refits *only those variables* with a smaller penalty $\phi\lambda$:

$$\hat{\beta}^{\lambda,\phi} = \arg\min_{\beta} \frac{1}{n} \sum_{i=1}^{n} \Big(y_i - \sum_{j \in \mathcal{M}_\lambda} x_{ij}\beta_j \Big)^2 + \phi\lambda \|\beta\|_1, \qquad 0 \le \phi \le 1.$$

$\phi = 1$ is the ordinary lasso, and $\phi = 0$ is OLS on the selected variables. This removes the shrinkage bias from the variables that survive the screen. (In R: `glmnet(..., relax = TRUE)`.)

Tibshirani's response: the relaxed lasso only re-estimates the nonzero coefficients. It **does not prune the model**, so it fixes the *bias* but not the *false positives* from screening. To get a sparser model he points to the **adaptive lasso** (weights $w_j = 1/|\hat{\beta}_j^{\text{init}}|$, which penalize small effects more), or non-convex penalties such as SCAD and SparseNet. Bühlmann's own approach is **stability selection**: rerun the lasso on many subsamples and keep only the variables chosen consistently, which controls the expected number of false positives.

12.4 4. Critiques

12.4.1 Bayesian (Holmes)

Holmes notes that the lasso is a **MAP (posterior mode) estimate** under independent Laplace (double-exponential) priors on the β_j:

$$\hat{\beta} = \arg\max_{\beta} \Big\{ \ell(\beta) - \lambda \sum_j |\beta_j| \Big\}.$$

He also writes the lasso as a **generalized ridge regression** with a different variance for each coefficient:

$$\{\hat{\beta}, \hat{\eta}\} = \arg\max_{\beta,\eta} \Big\{ \ell(\beta) - \sum_j \eta_j^{-1} \beta_j^2 \Big\} \quad \text{subject to} \quad \sum_j \eta_j = t.$$

This view adds to the ridge contrast in Section 2:

- **Ridge:** every coefficient gets the *same, fixed* prior variance.
- **Lasso:** a fixed total variance budget is split up *based on the data*. Irrelevant predictors get $\eta_j \to 0$, which pushes them to exactly zero and leaves more variance, and so less shrinkage, for the important ones.

His critiques:

- **The tails are too light.** Unbiased estimates of large effects would need $\eta_j \to \infty$, which the fixed budget allows only if $t \to \infty$. So the Laplace prior always biases large true effects toward zero, and the lasso doesn't have "oracle" properties. This is the same bias the relaxed lasso tries to fix (Section 3). It also motivates heavier-tailed priors like the horseshoe.
- **Reporting the posterior mode is hard to justify.** The posterior mode is only the optimal answer under a contrived 0–1 loss. A single point estimate also **hides uncertainty**, where a full Bayesian analysis would average over the unknowns.
- His conclusion: if you are going to add subjective penalties, it is "simpler and more natural to do so within a Bayesian paradigm."

12.4.2 Correlated predictors

Holmes's second concern, which he illustrates with genomics, is correlation:

- In a group of **strongly correlated predictors**, the lasso tends to keep **one** and set the rest to zero, and *which one* it keeps can change from sample to sample.
- Weak but real predictors that are correlated with stronger ones may be dropped completely.

- A Bayesian analysis can instead report *joint* posterior inclusion probabilities that show which predictors trade off with each other (Clyde & George, 2004; Park & Casella, 2008).

This links to Section 3. Bühlmann's irrepresentable condition, which is needed for exact selection, is a condition on exactly this kind of correlation between relevant and irrelevant predictors.

12.5 5. Response: the known gap is inference

Tibshirani's rejoinder:

- **On the Bayesian critique:** under the Laplace prior, the posterior **mean and median are not sparse**. The Bayesian lasso (Park & Casella, 2008) gives up the selection property. Bayesian sparsity needs a prior with **point mass at zero**, such as spike-and-slab (George & McCulloch, 1993), which leads to non-convex, computationally daunting problems. That is the convexity tradeoff from Section 2 again. He also questions Holmes's constrained-variance prior: "is this prior with a constraint something which a statistician might think is reasonable on its own?"
- **On correlated predictors:** he agrees the problem is important, but prefers **better penalties** to full Bayes: the group lasso, the **elastic net**, or covariance-regularized regression ("scout").
- **On screening:** see Section 3. The adaptive lasso is promising, while the relaxed lasso doesn't prune the model.

The known gap: inference. Tibshirani closes by saying "we need better tools for inference with the lasso":

- We need reliable ways to assess the **sampling variability** of $\widehat{\beta}^{\text{lasso}}$.
- Ordinary standard errors would be a start, but they are not enough. Because the lasso sets coefficients exactly to zero, the sampling distribution of each $\widehat{\beta}_j$ is a **mixture with a point mass at zero**, so it is not approximately normal.
- The **bootstrap** might be the fallback, but how to apply it here, and what its properties are, still had to be worked out.
- Bühlmann's multiple-testing and p-value work for high-dimensional models is an early step in this direction.

12.6 General notes

12.6.1 History, and why it took a while to catch on

Inspiration: Breiman's non-negative garrote (1995). First fit OLS to get $\widehat{\beta}_j$, then shrink each one by a factor $c_j \geq 0$ with $\sum_j c_j \leq t$. Because it depends on the OLS estimates, it breaks down when OLS is unstable or doesn't exist. Tibshirani merged the two stages and constrained β directly. Related work from the same period: bridge regression (Frank & Friedman, 1993) and basis pursuit (Chen, Donoho & Saunders, 1998).

Why the paper sat largely ignored for years. Tibshirani's guesses:

a. Computation in 1996 was slow.
b. The algorithms were black boxes (quadratic programming) with no statistical intuition until LARS in 2002.
c. Neither he nor the community yet saw the statistical and numerical value of sparsity.
d. Large problems (large N, large p, or both) were rare.
e. There was no R ecosystem for fast, easy sharing of software.

12.6.2 Computational advances

The history of lasso computation goes in three steps:

1. **Off-the-shelf quadratic programming** (1996). Correct, but it scales poorly and gives no insight into the problem.
2. **LARS / homotopy** (Osborne et al., 2000; Efron et al., 2002/2004). The solution path $\widehat{\beta}(\lambda)$ is *piecewise linear* in λ, so the whole path can be computed for about the cost of one least-squares fit. LARS also connects the lasso to forward stagewise regression, which helped explain older stepwise methods.
3. **Coordinate descent** (Fu, 1998; Friedman et al., 2007, 2010; Wu & Lange, 2008). Cycle through the coefficients one at a time and update each with a closed-form *soft-thresholding* step. The method is "extremely simple and fast" and works especially well when the solution is sparse, because most coefficients stay at zero and their updates cost almost nothing. This is what `glmnet` uses, and it is what we call in the homework.

13 Explainer: Regularization

Audience: first-year Master's and PhD students who know linear regression and some matrix algebra.

Goal: For each of the four approaches in this module (ridge, lasso, elastic net, and principal component regression), we

1. explain the math,
2. implement it on example data (first by hand where that's practical, then with the standard R package), and
3. walk through the key lines of code.

```
library(glmnet)  # ridge, lasso, elastic net (coordinate descent)
library(pls)     # principal component regression
library(lars)    # the diabetes data from Efron et al. (2004)
set.seed(6400)
```

13.1 Why regularize at all?

Recall the linear model from the last explainer:

$$Y = X\beta + \varepsilon, \qquad \varepsilon \sim \text{Normal}(\mathbf{0}_n, \sigma^2 I_n).$$

Ordinary least squares (OLS) chooses β to minimize the residual sum of squares,

$$\hat{\beta}^{\text{OLS}} = \arg\min_{\beta} \|Y - X\beta\|_2^2 = (X^\top X)^{-1} X^\top Y.$$

That formula requires $X^\top X$ to be invertible, and in practice it breaks down in two ways:

1. $p \geq n$ (more predictors than observations, as in the riboflavin homework). $X^\top X$ is singular, so infinitely many β fit the training data perfectly.
2. **Collinearity.** $X^\top X$ is technically invertible but *nearly* singular. Its smallest eigenvalues are close to 0, and since $\text{Var}(\hat{\beta}^{\text{OLS}}) = \sigma^2 (X^\top X)^{-1}$, the coefficient variances blow up.

Even when OLS works, it is *unbiased* but can have high *variance*. Expected prediction error at a new point x_0 splits into

$$E[(Y_0 - \hat{f}(x_0))^2] = \sigma^2 + \text{Bias}^2(\hat{f}(x_0)) + \text{Var}(\hat{f}(x_0)).$$

Regularization deliberately adds a little bias to remove a lot of variance. Every method below does this by restricting how large or how complex β can be.

13.1.1 The general recipe

Ridge, lasso, and elastic net all solve a *penalized* least-squares problem:

$$\hat{\beta}(\lambda) = \arg\min_{\beta} \underbrace{\|Y - X\beta\|_2^2}_{\text{fit the data}} + \underbrace{\lambda P(\beta)}_{\text{keep } \beta \text{ small}} .$$

- $P(\beta)$ is the **penalty**, which decides *how* "small" is measured.
- $\lambda \geq 0$ is a **tuning parameter**. $\lambda = 0$ gives back OLS, and $\lambda \to \infty$ forces $\beta \to 0$. We choose λ by cross-validation.

Each penalized problem has an equivalent **constrained** form, $\min_\beta \|Y - X\beta\|_2^2$ subject to $P(\beta) \leq t$, where each λ corresponds to some budget t. The constrained form makes the geometry easy to see.

13.1.2 Two housekeeping rules: standardize and center

1. **Standardize the predictors.** The penalty treats all coefficients the same, but a coefficient's size depends on its predictor's units. (Measuring height in mm instead of m would shrink its β by 1000 and make it almost free to keep.) So we rescale each column of X to mean 0 and variance 1 before penalizing.
2. **Don't penalize the intercept.** Shrinking β_0 toward 0 would just bias predictions toward 0. If we center Y and X, the intercept drops out (it equals $\bar{y}$).

From here on, unless stated otherwise, X is standardized and Y is centered.

13.2 The data

We use two datasets.

Simulated data, where we know the truth. This is adapted from Example 1 of Tibshirani (1996): $p = 8$ predictors with correlation $\mathrm{Cor}(x_j, x_k) = 0.5^{|j-k|}$ and a *sparse* true coefficient vector,

$$\beta = (3,\ 1.5,\ 0,\ 0,\ 2,\ 0,\ 0,\ 0), \qquad \sigma = 3.$$

Only predictors 1, 2, and 5 matter. Because we know this, we can check which methods find them.

```
n_sim  <- 60
p_sim  <- 8
beta_true <- c(3, 1.5, 0, 0, 2, 0, 0, 0)

Sigma <- 0.5^abs(outer(1:p_sim, 1:p_sim, "-"))     # AR(1) correlation matrix
Z     <- matrix(rnorm(n_sim * p_sim), n_sim, p_sim)
X_sim <- Z %*% chol(Sigma)                          # correlated predictors
colnames(X_sim) <- paste0("x", 1:p_sim)
y_sim <- drop(X_sim %*% beta_true + rnorm(n_sim, sd = 3))

# standardize X (using the 1/n variance, to match glmnet) and center y
sd_n <- function(v) sqrt(mean((v - mean(v))^2))
Xs   <- scale(X_sim, center = TRUE, scale = apply(X_sim, 2, sd_n))
yc   <- y_sim - mean(y_sim)
```

Code notes:

- `outer(1:p, 1:p, "-")` builds the matrix of $j - k$, so `0.5^abs(...)` is the AR(1) correlation matrix Σ.
- If Z has iid $N(0,1)$ entries and $\Sigma = R^\top R$ (Cholesky), then the rows of ZR have covariance $R^\top R = \Sigma$. That is why we multiply by `chol(Sigma)`.
- We standardize with the $1/n$ standard deviation (not R's default $1/(n-1)$) so that each column satisfies $\frac{1}{n}\sum_i x_{ij}^2 = 1$ exactly. This makes the formulas below cleaner and lets our hand-coded results match `glmnet` to many decimal places.

Real data: diabetes progression. This is the dataset Efron, Hastie, Johnstone & Tibshirani (2004) used to introduce LARS. There are 442 patients, and the outcome is a measure of disease progression one year after baseline. We use the expanded version `x2`, which has the 10 baseline variables (age, sex, BMI, blood pressure, six blood serum measures) plus their squares and

pairwise interactions, for **64 predictors**. Many of them are strongly correlated, which is a natural setting for regularization.

```
data(diabetes)
X_db <- unclass(diabetes$x2)       # 442 x 64 matrix
y_db <- diabetes$y
dim(X_db)
```

```
[1] 442  64
```

```
train <- sample(nrow(X_db), 342)   # hold out 100 patients for testing
X_tr <- X_db[train, ];  y_tr <- y_db[train]
X_te <- X_db[-train, ]; y_te <- y_db[-train]

# how collinear is this design?
round(kappa(scale(X_tr), exact = TRUE))
```

```
[1] 15165
```

The condition number $\kappa = \sigma_{\max}/\sigma_{\min}$ of the standardized training design is very large, so OLS coefficients here will be unstable.

13.3 Ridge regression (ℓ_2 penalty)

13.3.1 The math

Ridge regression (Hoerl & Kennard, 1970) uses the squared ℓ_2 norm as its penalty:

$$\hat{\beta}^{\text{ridge}} = \arg\min_{\beta} \|Y - X\beta\|_2^2 + \lambda \sum_{j=1}^{p} \beta_j^2.$$

This objective is a smooth quadratic, so we can set its gradient to zero:

$$-2X^\top(Y - X\beta) + 2\lambda\beta = 0 \quad \Longrightarrow \quad \boxed{\hat{\beta}^{\text{ridge}} = (X^\top X + \lambda I_p)^{-1} X^\top Y.}$$

Adding λI_p moves every eigenvalue of $X^\top X$ up by λ. For any $\lambda > 0$ the matrix is therefore invertible, **even when** $p > n$. This "add a ridge to the diagonal" step is where the name comes from.

How ridge shrinks, through the SVD. Write $X = UDV^\top$ with singular values $d_1 \geq \cdots \geq d_p$. The ridge fitted values are

$$\hat{Y}^{\text{ridge}} = X\hat{\beta}^{\text{ridge}} = \sum_{j=1}^{p} u_j \frac{d_j^2}{d_j^2 + \lambda} u_j^\top Y.$$

OLS would use a factor of 1 for every direction. Ridge multiplies the projection onto each principal direction u_j by $d_j^2/(d_j^2 + \lambda)$, which is less than 1. **Directions with small d_j (low variance, where collinearity lives) are shrunk the most.** The *effective degrees of freedom* are $\text{df}(\lambda) = \sum_j d_j^2/(d_j^2 + \lambda)$, which falls smoothly from p at $\lambda = 0$ toward 0.

Ridge as augmented OLS. Ridge is exactly OLS on a padded dataset: append p fake observations with X-rows $\sqrt{\lambda}\, I_p$ and response 0:

$$\tilde{X} = \begin{bmatrix} X \\ \sqrt{\lambda}\, I_p \end{bmatrix}, \quad \tilde{Y} = \begin{bmatrix} Y \\ \mathbf{0}_p \end{bmatrix} \implies \tilde{X}^\top \tilde{X} = X^\top X + \lambda I, \quad \tilde{X}^\top \tilde{Y} = X^\top Y.$$

The fake observations each say "this coefficient is 0", which pulls the estimates toward zero.

Bayesian view. Ridge is the posterior mean (and mode) under independent priors $\beta_j \sim N(0, \tau^2)$, with $\lambda = \sigma^2/\tau^2$.

13.3.2 Implementation from scratch

```
ridge_closed <- function(X, y, lambda) {
  p <- ncol(X)
  solve(crossprod(X) + lambda * diag(p), crossprod(X, y))
}

# check against OLS on the augmented data, for one lambda
lam  <- 25
X_aug <- rbind(Xs, sqrt(lam) * diag(p_sim))
y_aug <- c(yc, rep(0, p_sim))
cbind(closed_form = drop(ridge_closed(Xs, yc, lam)),
      augmented   = coef(lm(y_aug ~ X_aug - 1)))
```

```
   closed_form   augmented
x1  1.58410686  1.58410686
x2  0.89708971  0.89708971
x3  0.17378740  0.17378740
```

```
x4  0.41941820  0.41941820
x5  1.14121834  1.14121834
x6 -0.32424379 -0.32424379
x7  0.08301728  0.08301728
x8  0.30371878  0.30371878
```

Code notes:

- `crossprod(X)` computes $X^\top X$ and `crossprod(X, y)` computes $X^\top Y$. They are faster and more numerically careful than `t(X) %*% X`.
- `solve(A, b)` solves the linear system $A\beta = b$ directly. It is better than `solve(A) %*% b`, which forms the inverse explicitly. We never need $(X^\top X + \lambda I)^{-1}$ itself.
- `lm(y_aug ~ X_aug - 1)` fits OLS with no intercept on the padded data. The two columns agree, which confirms the augmented-data identity.

Now we compute the full **coefficient path** by solving for $\hat{\beta}$ over a grid of λ values:

```r
lambdas <- 10^seq(-2, 4, length.out = 100)
B_ridge <- sapply(lambdas, function(l) ridge_closed(Xs, yc, l))   # p x 100

d   <- svd(Xs)$d
dfs <- sapply(lambdas, function(l) sum(d^2 / (d^2 + l)))           # effective df

matplot(log10(lambdas), t(B_ridge), type = "l", lty = 1, lwd = 2,
        col = ifelse(beta_true != 0, "firebrick", "grey60"),
        xlab = expression(log[10](lambda)), ylab = "Coefficient")
abline(h = 0, lty = 3)
axis(3, at = log10(lambdas)[c(1, 40, 60, 75, 90)],
     labels = round(dfs[c(1, 40, 60, 75, 90)], 1))
mtext("effective degrees of freedom", side = 3, line = 2, cex = 0.8)
legend("topright", c("true signal (x1, x2, x5)", "true zero"),
       col = c("firebrick", "grey60"), lwd = 2, bty = "n")
```

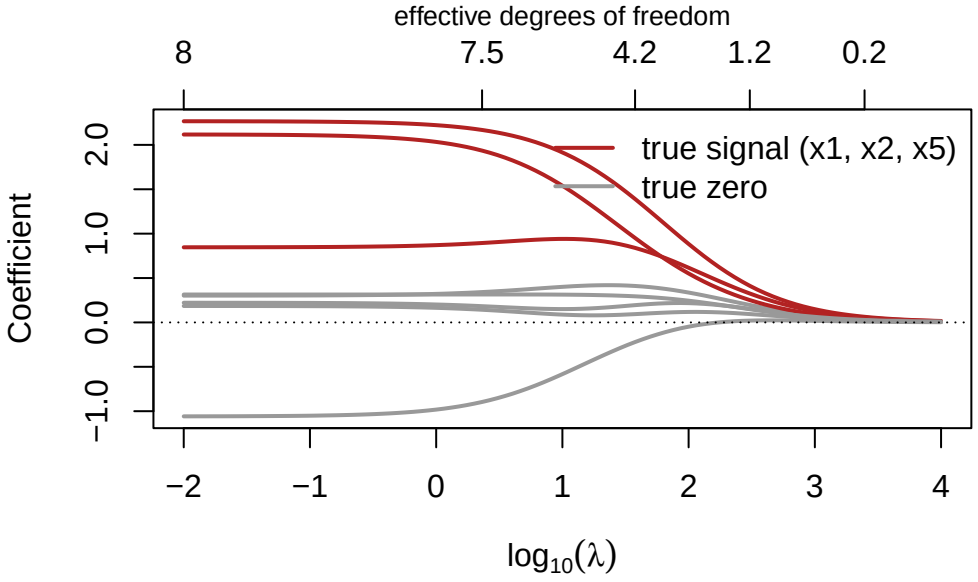


Figure 13.1: Ridge coefficient path on the simulated data, computed with the closed-form solution. Every coefficient shrinks smoothly toward zero, but none reaches exactly zero.

Code notes:

- `sapply(lambdas, function(l) ...)` returns one column per λ, giving a $p \times 100$ matrix. `matplot(..., t(B_ridge))` draws one line per coefficient.
- The top axis converts λ to effective df using the singular values from `svd()`. This is the SVD shrinkage formula above, coded directly.
- Coloring by `beta_true != 0` lets us see that the true signals (red) stay large longest. The noise coefficients (grey) shrink, but they **never become exactly 0**. Ridge *shrinks* but does not *select*.

13.3.3 Implementation with `glmnet`

In practice we use `glmnet`, which also handles cross-validation. Its objective is scaled slightly differently from ours:

$$\text{glmnet:} \quad \frac{1}{2n}\|Y - X\beta\|_2^2 + \lambda_{\text{glmnet}} \cdot \frac{1}{2}\|\beta\|_2^2.$$

Multiply through by $2n$ and you get our objective with $\lambda = n\lambda_{\text{glmnet}}$. There is one more catch. For the Gaussian family, `glmnet` internally standardizes Y to unit variance before fitting, so the λ it reports corresponds to $\lambda = n\lambda_{\text{glmnet}}/\text{sd}(Y)$. We check this:

```
lam_glmnet <- lam * sd_n(yc) / n_sim
fit_r <- glmnet(Xs, yc, alpha = 0, lambda = lam_glmnet,
                standardize = FALSE, intercept = FALSE,
                control = list(thresh = 1e-16))
cbind(closed_form = drop(ridge_closed(Xs, yc, lam)),
      glmnet      = as.numeric(coef(fit_r))[-1])
```

```
   closed_form      glmnet
x1  1.58410686  1.58410686
x2  0.89708971  0.89708972
x3  0.17378740  0.17378740
x4  0.41941820  0.41941819
x5  1.14121834  1.14121834
x6 -0.32424379 -0.32424379
x7  0.08301728  0.08301728
x8  0.30371878  0.30371878
```

Code notes:

- `alpha = 0` tells `glmnet` to use the pure ridge penalty (more on `alpha` in the elastic net section).
- We set `standardize = FALSE` and `intercept = FALSE` only because we already standardized and centered the data ourselves. With raw data, leave both at their defaults (`TRUE`). `glmnet` will then standardize internally and **report coefficients on the original scale**.
- `control = list(thresh = ...)` tightens the convergence tolerance so the match goes to many decimal places.
- The takeaway: λ **values are not comparable across software.** Don't copy a λ from `glmnet` into another package, or into a formula, without checking how each one scales the objective.

13.4 The lasso (ℓ_1 penalty)

13.4.1 The math

The lasso (Tibshirani, 1996) replaces the squared penalty with absolute values:

$$\hat{\beta}^{\text{lasso}} = \arg\min_{\beta} \frac{1}{2n}\|Y - X\beta\|_2^2 + \lambda \sum_{j=1}^{p} |\beta_j|.$$

(Here we use `glmnet`'s $\frac{1}{2n}$ scaling from the start. It only rescales λ.)

This small change has a large effect: **the lasso sets some coefficients exactly to zero**, so it performs variable selection as well as shrinkage. There are two ways to see why.

1. Geometry. In the constrained form, we look for the point in the region $\sum_j |\beta_j| \le t$ that has the smallest RSS. The RSS contours are ellipses centered at $\hat{\beta}^{\text{OLS}}$. We expand the ellipses until they first touch the region. The ℓ_1 region is a diamond with **corners on the axes**, and an ellipse usually first touches a diamond at a corner, where some $\beta_j = 0$. The ℓ_2 region (a circle) has no corners, so the contact point almost never lies on an axis.

```
set.seed(2)
X2 <- scale(MASS::mvrnorm(100, c(0, 0), matrix(c(1, 0.6, 0.6, 1), 2)))
y2 <- drop(X2 %*% c(2, 0.6) + rnorm(100))
y2 <- y2 - mean(y2)
b_ols <- coef(lm(y2 ~ X2 - 1))
rss   <- function(b) sum((y2 - X2 %*% b)^2)

# RSS on a grid, for the contour plot
g  <- seq(-0.5, 2.5, length.out = 200)
RS <- outer(g, g, Vectorize(function(a, b) rss(c(a, b))))

# brute-force the constrained solution: minimum RSS on each region's boundary
t_budget <- 1
th <- seq(0, 2 * pi, length.out = 20000)
diamond <- cbind(cos(th), sin(th)) / (abs(cos(th)) + abs(sin(th))) * t_budget
circle  <- cbind(cos(th), sin(th)) * t_budget
b_lasso <- diamond[which.min(apply(diamond, 1, rss)), ]
b_ridge <- circle[which.min(apply(circle, 1, rss)), ]

contour(g, g, RS, levels = rss(b_ols) + c(5, 20, 50, 100, 170, 260, 370),
        drawlabels = FALSE, col = "grey55", asp = 1,
        xlab = expression(beta[1]), ylab = expression(beta[2]))
abline(h = 0, v = 0, col = "grey85")
polygon(diamond, border = "firebrick", lwd = 2, col = adjustcolor("firebrick", 0.08))
polygon(circle,  border = "steelblue", lwd = 2, col = adjustcolor("steelblue", 0.08))
points(b_ols[1], b_ols[2], pch = 19)
points(b_lasso[1], b_lasso[2], pch = 19, col = "firebrick", cex = 1.4)
points(b_ridge[1], b_ridge[2], pch = 19, col = "steelblue", cex = 1.4)
```

```
text(b_ols[1], b_ols[2], "OLS", pos = 4)
legend("topleft", c("lasso: |b1| + |b2| <= 1", "ridge: b1^2 + b2^2 <= 1"),
       col = c("firebrick", "steelblue"), lwd = 2, bty = "n")
```

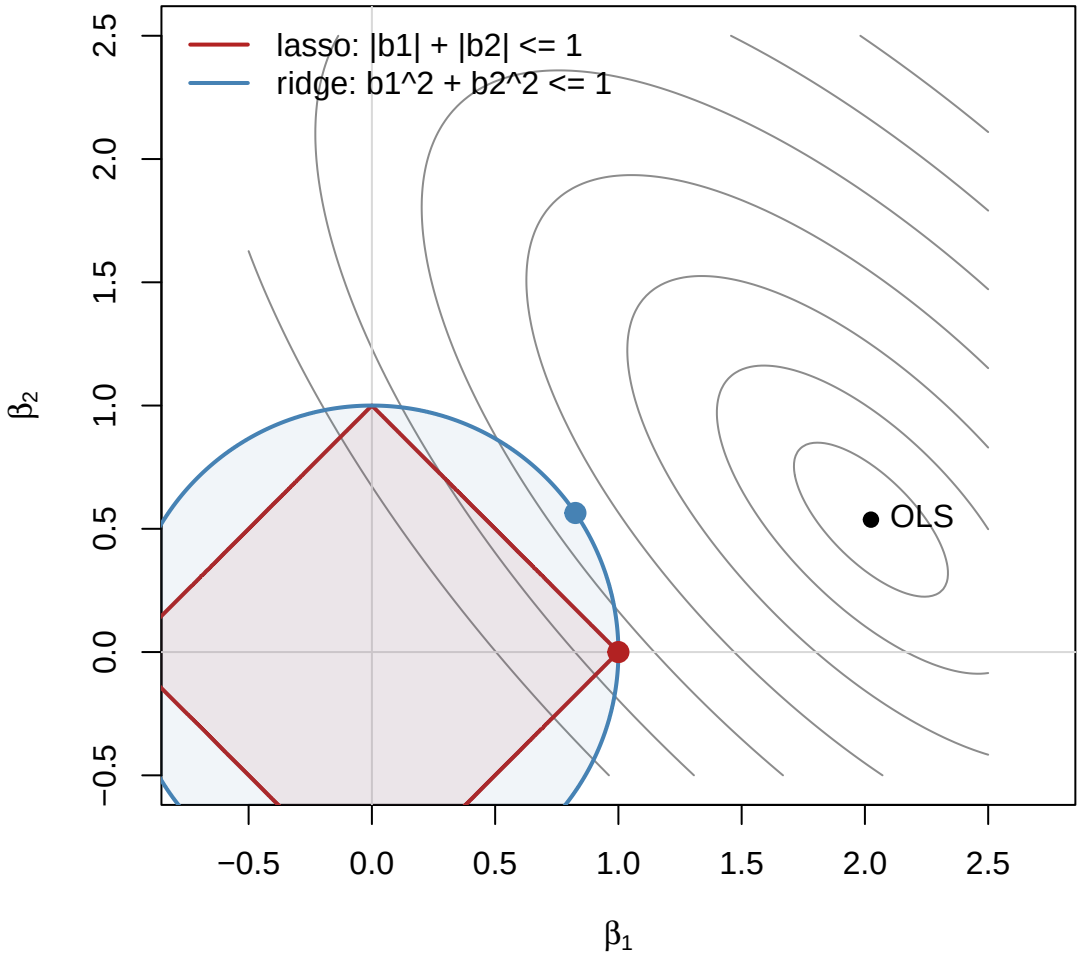


Figure 13.2: Constrained-form geometry with two correlated predictors. RSS contours (ellipses) are centered at the OLS estimate. The lasso solution (red) lands on a corner of the diamond, so beta_2 = 0 exactly. The ridge solution (blue) lands on a smooth part of the circle, where both coefficients are nonzero.

Code notes:

- `outer(g, g, Vectorize(...))` evaluates the RSS at every (β_1, β_2) grid point, and `contour()` draws the level sets.
- The OLS estimate lies outside both regions, so each constrained solution is on the region's *boundary*. That means we can find it by brute force: trace 20,000 points around each boundary and keep the one with the smallest RSS (`which.min`). Dividing the unit circle by $|\cos\theta| + |\sin\theta|$ turns it into the ℓ_1 diamond.
- Brute force is fine for a 2-D picture. It is not how the lasso is actually computed, which comes next.

2. Soft-thresholding. Consider the lasso problem for one standardized predictor (or, equivalently, an orthonormal design). Let $z = \frac{1}{n} x^\top Y$ be the least-squares coefficient. Because $|\beta|$ has a kink at 0, we use the subgradient: the optimality condition is $0 \in \beta - z + \lambda\, \partial|\beta|$, where $\partial|\beta| = \text{sign}(\beta)$ if $\beta \neq 0$ and $\partial|\beta| = [-1, 1]$ if $\beta = 0$. Solving it gives the **soft-thresholding operator**:

$$\hat{\beta} = S(z, \lambda) = \text{sign}(z)\,(|z| - \lambda)_+ = \begin{cases} z - \lambda & z > \lambda \\ 0 & |z| \le \lambda \\ z + \lambda & z < -\lambda \end{cases}$$

If the least-squares coefficient is smaller than λ in absolute value, the lasso sets it to **exactly zero**. Otherwise it subtracts λ from it (shrinkage). Compare the other methods under the same orthonormal design:

Method	Estimate as a function of the OLS estimate z
Ridge	$z/(1+\lambda)$ (proportional shrinkage, never 0)
Lasso	$\text{sign}(z)(\lvert z\rvert - \lambda)_+$ (soft threshold)
Best subset	$z \cdot \mathbb{1}(\lvert z\rvert > \lambda)$ (hard threshold: keep or kill)

```
z <- seq(-3, 3, length.out = 601); l <- 1
plot(z, z, type = "l", lty = 3, col = "grey50",
     xlab = "OLS estimate z", ylab = "regularized estimate")
lines(z, z / (1 + l), col = "steelblue", lwd = 2)
lines(z, sign(z) * pmax(abs(z) - l, 0), col = "firebrick", lwd = 2)
lines(z, z * (abs(z) > l), col = "darkgreen", lwd = 2, lty = 2)
legend("topleft", c("no shrinkage (OLS)", "ridge", "lasso (soft)", "best subset (hard)"),
       col = c("grey50", "steelblue", "firebrick", "darkgreen"),
       lty = c(3, 1, 1, 2), lwd = 2, bty = "n")
```

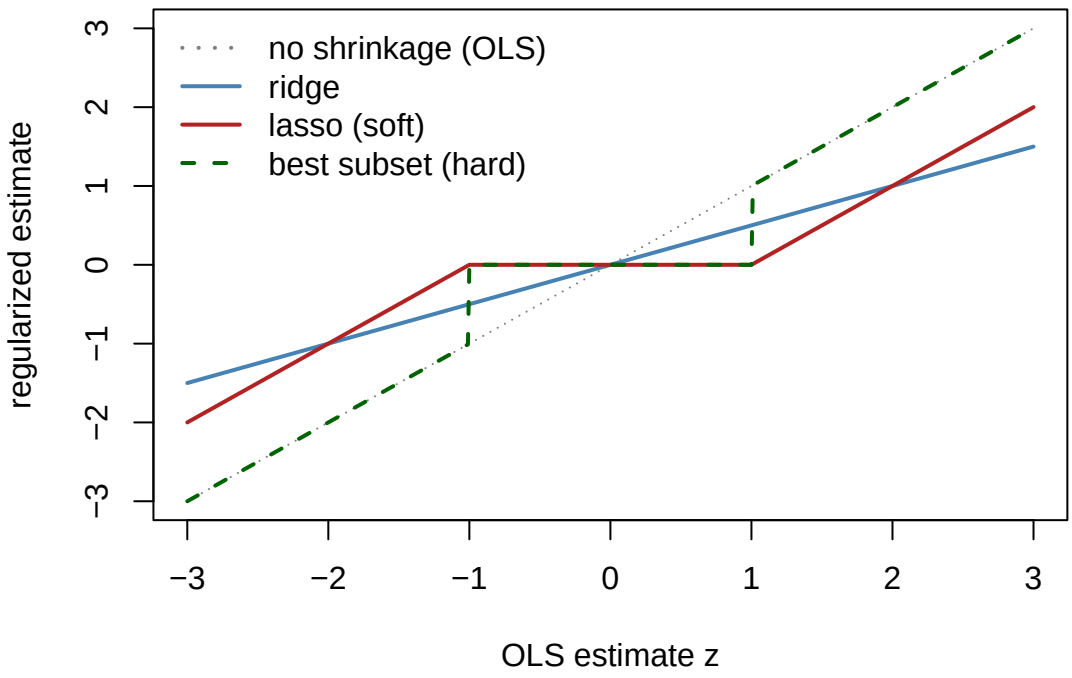


Figure 13.3: How each method maps the OLS estimate z to a regularized estimate (orthonormal design, lambda = 1). Ridge scales everything down, the lasso zeroes small values and shifts large ones, and best subset keeps or kills.

Bayesian view. The lasso estimate is the posterior *mode* (MAP) under independent Laplace priors $p(\beta_j) \propto e^{-|\beta_j|/b}$. The Laplace density has a sharp peak at 0, the probability version of the diamond's corner. Note that the posterior *mean* under this prior (the "Bayesian lasso" of Park & Casella, 2008) is **not** sparse. See the Paper chapter for the debate about this.

13.4.2 Implementation from scratch: coordinate descent

The lasso has no closed form when $p > 1$, but the one-dimensional soft-thresholding solution leads to a very simple algorithm. **Coordinate descent** cycles through the coefficients and, holding all the others fixed, updates each one with its exact one-dimensional solution:

1. Compute the **partial residual** without predictor j: $r^{(j)} = Y - \sum_{k \neq j} x_k \beta_k$.
2. Find the least-squares coefficient of $r^{(j)}$ on x_j: $z_j = \frac{1}{n} x_j^\top r^{(j)}$.

3. Soft-threshold it: $\beta_j \leftarrow S(z_j, \lambda) \,/\, (\frac{1}{n} x_j^\top x_j)$. The denominator is 1 for standardized predictors.
4. Repeat over all j until the coefficients stop changing.

The objective is convex, and the non-smooth part ($\lambda \sum |\beta_j|$) splits into separate terms for each coordinate, so this procedure converges to the global minimum (Tseng, 2001). This is the algorithm inside `glmnet` (Friedman, Hastie & Tibshirani, 2010).

```
soft_threshold <- function(z, gamma) sign(z) * pmax(abs(z) - gamma, 0)

lasso_cd <- function(X, y, lambda, beta = rep(0, ncol(X)),
                     tol = 1e-10, max_iter = 10000) {
  n  <- nrow(X)
  xx <- colSums(X^2) / n            # (1/n) x_j'x_j; equals 1 if standardized
  r  <- drop(y - X %*% beta)        # current full residual
  for (iter in seq_len(max_iter)) {
    beta_old <- beta
    for (j in seq_along(beta)) {
      r       <- r + X[, j] * beta[j]                # step 1: partial residual
      z_j     <- sum(X[, j] * r) / n                 # step 2: simple LS coef
      beta[j] <- soft_threshold(z_j, lambda) / xx[j]  # step 3: soft-threshold
      r       <- r - X[, j] * beta[j]                # put updated fit back
    }
    if (max(abs(beta - beta_old)) < tol) break       # step 4: converged?
  }
  beta
}

lam_l <- 0.5
fit_l <- glmnet(Xs, yc, alpha = 1, lambda = lam_l,
                standardize = FALSE, intercept = FALSE,
                control = list(thresh = 1e-16))
cbind(truth       = beta_true,
      scratch_cd  = round(lasso_cd(Xs, yc, lam_l), 5),
      glmnet      = round(as.numeric(coef(fit_l))[-1], 5))
```

```
     truth scratch_cd  glmnet
[1,]   3.0    1.86128 1.86128
[2,]   1.5    0.96766 0.96766
[3,]   0.0    0.00000 0.00000
[4,]   0.0    0.07911 0.07911
[5,]   2.0    1.17824 1.17824
```

```
[6,]   0.0    0.00000 0.00000
[7,]   0.0    0.00000 0.00000
[8,]   0.0    0.00000 0.00000
```

Code notes:

- **The residual trick.** Recomputing $r^{(j)} = Y - \sum_{k \neq j} x_k \beta_k$ from scratch would cost $O(np)$ per coordinate. Instead we keep the *full* residual `r`, add back predictor j's contribution (adding `X[, j] * beta[j]` to `r`), update β_j, and subtract the new contribution. Each update then costs only $O(n)$.
- **Sparsity makes it fast.** When `beta[j]` is 0 and stays 0, the residual doesn't change. `glmnet` goes further with "active set" iterations that skip predictors that keep coming out zero.
- `soft_threshold` is the boxed formula above, vectorized with `pmax`.
- No `glmnet` adjustment is needed this time. Its lasso objective is exactly $\frac{1}{2n}\text{RSS} + \lambda\|\beta\|_1$, so the two columns match. (The lasso solution scales linearly with Y, so the internal Y-standardization cancels. That is why the sd(Y) issue only showed up for ridge.)
- Compare with `truth`: the noise coefficients are mostly *exactly* 0, and the true signals are nonzero but **shrunk toward zero**. That shrinkage is the lasso's bias.

13.4.3 The lasso path and choosing λ

Warm starts. To compute the whole path, we start at $\lambda_{\max} = \max_j |\frac{1}{n} x_j^\top Y|$, the smallest λ at which *every* coefficient is 0. We then decrease λ step by step and initialize each fit at the previous solution. Neighboring solutions are close, so each fit takes only a few iterations.

```r
lambda_max <- max(abs(crossprod(Xs, yc))) / n_sim
lam_grid   <- lambda_max * 10^seq(0, -2.5, length.out = 60)

B_lasso <- matrix(0, p_sim, length(lam_grid))
b <- rep(0, p_sim)
for (k in seq_along(lam_grid)) {
  b <- lasso_cd(Xs, yc, lam_grid[k], beta = b)    # warm start from previous b
  B_lasso[, k] <- b
}

fit_path <- glmnet(Xs, yc, alpha = 1, lambda = lam_grid,
                   standardize = FALSE, intercept = FALSE)

cols <- ifelse(beta_true != 0, "firebrick", "grey60")
matplot(log(lam_grid), t(B_lasso), type = "l", lty = 1, lwd = 2, col = cols,
        xlab = expression(log(lambda)), ylab = "Coefficient")
```

```
matpoints(log(lam_grid)[seq(1, 60, 4)], t(as.matrix(fit_path$beta))[seq(1, 60, 4), ],
          pch = 1, col = cols, cex = 0.7)
abline(h = 0, lty = 3)
text(min(log(lam_grid)), B_lasso[, 60], colnames(Xs), pos = 4, cex = 0.7)
```

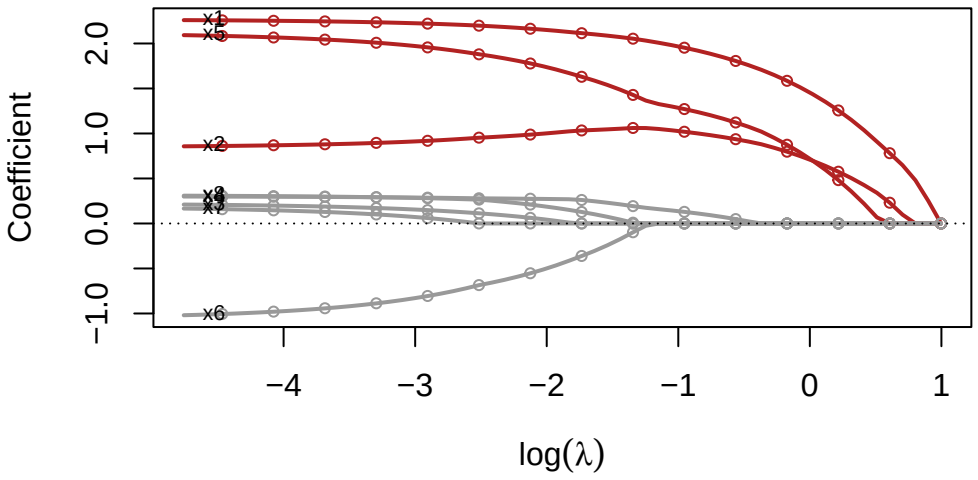


Figure 13.4: Lasso path on the simulated data. Solid lines are our coordinate-descent implementation and points are glmnet. Coefficients enter the model one at a time as lambda decreases, and the true signals (red) enter first.

Code notes:

- `b <- lasso_cd(..., beta = b)` is the warm start: each fit starts from the previous solution instead of from zero.
- Large λ is on the right. Moving left, coefficients "enter" one at a time. The path is piecewise linear in λ, which is what the LARS algorithm exploits. Unlike ridge, the curves sit **exactly on zero** until the coefficient enters.

Cross-validation. To choose λ we use K-fold CV. `cv.glmnet` reports two values:

- $\lambda_{\min}$: the value with the lowest mean CV error.
- $\lambda_{1\text{se}}$: the *largest* λ (so the sparsest model) whose CV error is within one standard error of the minimum. This "one-standard-error rule" prefers simpler models when the data can't tell the two apart.

```r
cv_l <- cv.glmnet(X_sim, y_sim, alpha = 1, nfolds = 10)
plot(cv_l)
coef_1se <- coef(cv_l, s = "lambda.1se")
cbind(truth = c(NA, beta_true), lasso_1se = round(as.numeric(coef_1se), 3))
```

```
      truth lasso_1se
 [1,]    NA    -0.072
 [2,]   3.0     1.601
 [3,]   1.5     0.860
 [4,]   0.0     0.000
 [5,]   0.0     0.000
 [6,]   2.0     0.844
 [7,]   0.0     0.000
 [8,]   0.0     0.000
 [9,]   0.0     0.000
```

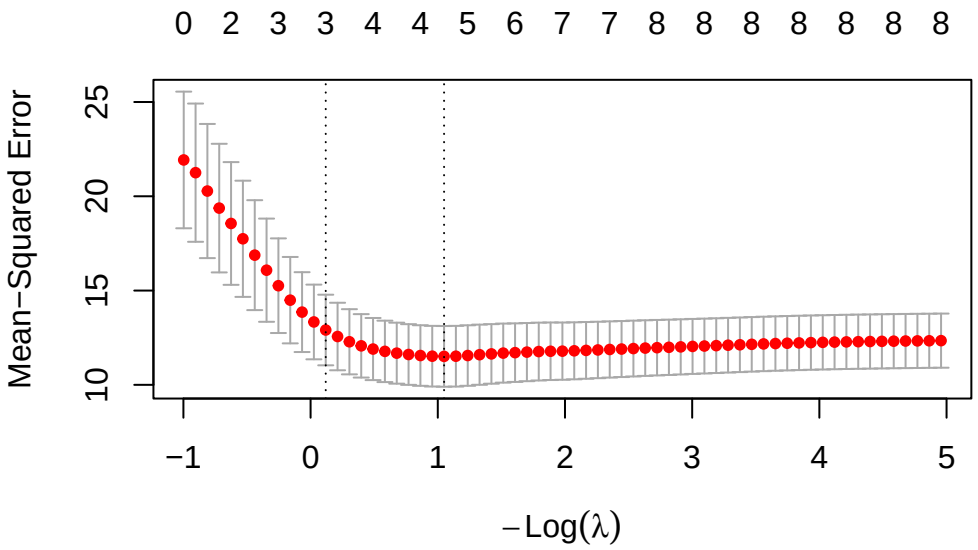


Figure 13.5: 10-fold CV for the lasso on the simulated data. The numbers on top are the count of nonzero coefficients.

Code notes:

- Here we pass the **raw** `X_sim` and `y_sim`. With the default `standardize = TRUE`, `cv.glmnet` standardizes internally, fits an unpenalized intercept, and returns coefficients on the original scale. This is the normal way to use it.
- `coef(cv_l, s = "lambda.1se")` extracts the coefficients at the chosen λ. The first row is the intercept.

13.5 Elastic net (a mix of ℓ_1 and ℓ_2)

13.5.1 The math

The lasso has two weaknesses that Zou & Hastie (2005) pointed out:

1. When $p > n$, the lasso can select **at most** n variables.
2. Given a group of **highly correlated** predictors, the lasso tends to pick one more or less arbitrarily and set the rest to zero. Which one it picks can change from sample to sample.

Ridge has neither problem: correlated predictors get similar coefficients (the "grouping effect"). But ridge never selects. The **elastic net** combines the two penalties. In `glmnet`'s parameterization:

$$\widehat{\beta}^{\mathrm{EN}} = \arg\min_{\beta} \frac{1}{2n}\|Y - X\beta\|_2^2 + \lambda\Big[\alpha\|\beta\|_1 + \frac{1-\alpha}{2}\|\beta\|_2^2\Big], \qquad 0 \le \alpha \le 1.$$

- $\alpha = 1$ gives the lasso and $\alpha = 0$ gives ridge. This is why we passed `alpha = 0` and `alpha = 1` above.
- For $0 < \alpha < 1$ the penalty region has corners on the axes (so selection still happens) and curved edges (so correlated predictors share weight).
- The coordinate-descent update needs only one change. The ridge part adds to the denominator:

$$\beta_j \leftarrow \frac{S(z_j, \lambda\alpha)}{\frac{1}{n}x_j^\top x_j + \lambda(1-\alpha)}.$$

(The original Zou & Hastie paper writes the penalty as $\lambda_1\|\beta\|_1 + \lambda_2\|\beta\|_2^2$ and recommends rescaling the result by $(1+\lambda_2)$. That "corrected" elastic net differs from what `glmnet` computes by exactly that factor.)

13.5.2 Implementation

Extending our coordinate descent is a one-line change:

```r
enet_cd <- function(X, y, lambda, alpha, tol = 1e-10, max_iter = 10000) {
  n <- nrow(X); beta <- rep(0, ncol(X))
  xx <- colSums(X^2) / n
  r  <- y
  for (iter in seq_len(max_iter)) {
    beta_old <- beta
    for (j in seq_along(beta)) {
      r       <- r + X[, j] * beta[j]
      z_j     <- sum(X[, j] * r) / n
      beta[j] <- soft_threshold(z_j, lambda * alpha) /
                 (xx[j] + lambda * (1 - alpha))          # <- the only change
      r       <- r - X[, j] * beta[j]
    }
    if (max(abs(beta - beta_old)) < tol) break
  }
  beta
}
ys <- yc / sd_n(yc)            # unit-variance y, so glmnet's internal y-scaling is a no-op
fit_en <- glmnet(Xs, ys, alpha = 0.5, lambda = 0.2, standardize = FALSE,
                 intercept = FALSE, control = list(thresh = 1e-16))
cbind(scratch = round(enet_cd(Xs, ys, lambda = 0.2, alpha = 0.5), 5),
      glmnet  = round(as.numeric(coef(fit_en))[-1], 5))
```

```
     scratch  glmnet
[1,] 0.36778 0.36778
[2,] 0.20477 0.20477
[3,] 0.00000 0.00000
[4,] 0.03543 0.03543
[5,] 0.22859 0.22859
[6,] 0.00000 0.00000
[7,] 0.00000 0.00000
[8,] 0.00000 0.00000
```

Code notes: we compare on `ys`, a version of Y scaled to unit variance. The reason is the Y-scaling issue from the ridge section. The ℓ_1 part of `glmnet`'s penalty is unaffected by its internal standardization of Y, but the ℓ_2 part effectively becomes $\lambda(1-\alpha)/\mathrm{sd}(Y)$. With $\mathrm{sd}(Y) = 1$ the two formulations are identical, and the columns match.

Choosing α and λ together. We cross-validate over a small grid of α. To compare α values fairly, we use the **same fold assignment** for every one, otherwise differences could just reflect different random splits. We do this on the diabetes data, which has strongly correlated predictors:

```r
foldid <- sample(rep(1:10, length.out = nrow(X_tr)))   # same folds for every alpha
alphas <- c(0, 0.25, 0.5, 0.75, 1)

cv_fits <- lapply(alphas, function(a)
  cv.glmnet(X_tr, y_tr, alpha = a, foldid = foldid))

data.frame(
  alpha      = alphas,
  lambda_min = signif(sapply(cv_fits, `[[`, "lambda.min"), 3),
  cv_mse_min = round(sapply(cv_fits, function(f) min(f$cvm))),
  nonzero_1se = sapply(cv_fits, function(f)
    sum(coef(f, s = "lambda.1se")[-1] != 0))
)
```

```
  alpha lambda_min cv_mse_min nonzero_1se
1  0.00      17.80       3074          64
2  0.25       8.99       2924          13
3  0.50       4.94       2916          10
4  0.75       3.61       2916           9
5  1.00       2.71       2916           9
```

Code notes:

- `sample(rep(1:10, length.out = n))` gives each training row a random fold label from 1 to 10. Passing it as `foldid` makes all five `cv.glmnet` calls use the same split.
- `sapply(cv_fits,[[, "lambda.min")` pulls one element out of each fitted object in the list.
- **Reading the table:** the CV error is usually similar across $\alpha > 0$, but the number of nonzero coefficients changes a lot. Smaller α keeps more correlated predictors together, and $\alpha = 0$ (ridge) keeps all 64. If several α values have about the same CV error, base the choice on what you need: sparsity and interpretability (larger α) or stability under correlation (smaller α).

```r
par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))
for (a in c(0, 0.5, 1)) {
  plot(glmnet(X_tr, y_tr, alpha = a), xvar = "lambda",
       col = "#00000040", main = paste("alpha =", a), font.main = 1)
}
```

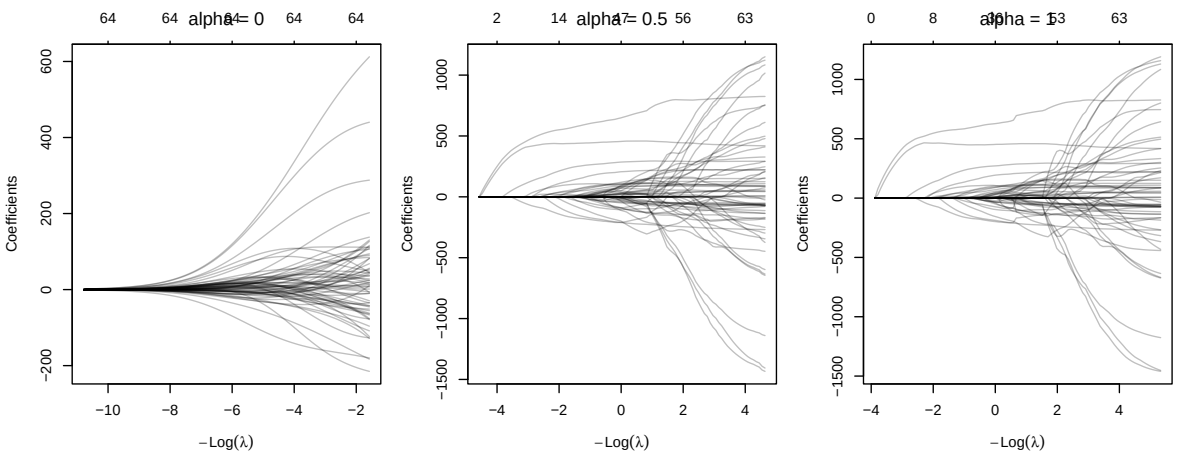


Figure 13.6: Coefficient paths on the diabetes data for ridge, elastic net (alpha = 0.5), and lasso. Moving from left to right, more coefficients sit exactly at zero over more of the path.

13.6 Principal component regression (PCR)

13.6.1 The math

PCR (Massy, 1965) regularizes differently. Instead of penalizing β, it **throws away low-variance directions of** X before regressing.

1. Center (and usually standardize) X and take its SVD: $X = UDV^\top$. The columns of V are the principal-component loadings, and the **principal components** are $Z = XV = UD$. They are uncorrelated, and z_j has variance d_j^2/n.
2. Keep the first $k < p$ components $Z_k = XV_k$ and regress Y on them by OLS. Because the z_j are orthogonal, this is just p separate simple regressions: $\hat{\theta}_j = z_j^\top Y / d_j^2$.
3. **Back-transform** to the original predictors: $X\beta = Z_k\theta = XV_k\theta$, so $\hat{\beta}^{\mathrm{PCR}} = V_k\hat{\theta}$.

PCR vs. ridge in one line each. Using the SVD shrinkage factors:

$$\hat{Y}^{\mathrm{ridge}} = \sum_{j=1}^{p} u_j \, \frac{d_j^2}{d_j^2 + \lambda} \, u_j^\top Y, \qquad \hat{Y}^{\mathrm{PCR}} = \sum_{j=1}^{p} u_j \, \mathbb{1}(j \le k) \, u_j^\top Y.$$

Ridge shrinks low-variance directions **smoothly**. PCR keeps the top k directions completely and drops the rest (**hard thresholding on the components**). Both target the same thing, the near-collinear directions where OLS variance is large, but neither does variable selection.

$\hat{\beta}^{\text{PCR}}$ is generally nonzero for *every* original predictor, because each component is a mix of all of them.

The main caveat is that PCR picks components by how much **variance in** X they explain, without looking at Y. Nothing guarantees that the high-variance directions are the ones that predict Y. (Partial least squares, `pls::plsr`, is the variant that does use Y.)

13.6.2 Implementation from scratch

```
pcr_scratch <- function(X, y, k) {
  sv <- svd(X)                                  # X = U D V'
  Zk <- X %*% sv$v[, 1:k, drop = FALSE]         # first k PC scores
  theta <- crossprod(Zk, y) / sv$d[1:k]^2       # orthogonal => separate regressions
  sv$v[, 1:k, drop = FALSE] %*% theta           # back-transform: beta = V_k theta
}

# variance explained by each component of the simulated X
pve <- svd(Xs)$d^2 / sum(svd(Xs)$d^2)
round(cumsum(pve), 3)
```

```
[1] 0.360 0.554 0.718 0.812 0.877 0.935 0.975 1.000
```

```
cbind(truth = beta_true,
      pcr_k3 = round(drop(pcr_scratch(Xs, yc, 3)), 3),
      pcr_k8 = round(drop(pcr_scratch(Xs, yc, 8)), 3),
      ols    = round(coef(lm(yc ~ Xs - 1)), 3))
```

```
     truth pcr_k3 pcr_k8    ols
Xsx1   3.0  0.920  2.267  2.267
Xsx2   1.5  1.159  0.846  0.846
Xsx3   0.0  0.840  0.222  0.222
Xsx4   0.0  0.593  0.302  0.302
Xsx5   2.0 -0.039  2.118  2.118
Xsx6   0.0 -0.099 -1.059 -1.059
Xsx7   0.0  0.378  0.186  0.186
Xsx8   0.0  0.545  0.314  0.314
```

Code notes:

- `svd(X)` returns a list with `u`, `d` (the singular values as a vector), and `v`. The PC scores are `X %*% V_k`.
- Since the components are orthogonal, $Z_k^\top Z_k = \text{diag}(d_1^2, \dots, d_k^2)$, so the OLS fit on the components is `crossprod(Zk, y) / d^2` and no matrix inverse is needed.
- `drop = FALSE` keeps a one-column result as a matrix when `k = 1`, so `%*%` still works.
- **Sanity check:** with $k = p = 8$ we keep every direction, and PCR gives exactly the OLS coefficients. With $k = 3$, every coefficient is still nonzero. That is the "no selection" point. Also look at `x5`: it is a true signal ($\beta_5 = 2$), but with $k = 3$ PCR estimates it near 0, while giving sizable weight to the noise predictors `x3` and `x8`. The top three components were chosen to explain variance in X, not to predict Y, so they happen to miss the direction that `x5` contributes.

13.6.3 Implementation with `pls`

```
pcr_fit <- pcr(y ~ X, data = data.frame(y = y_tr, X = I(X_tr)),
               scale = TRUE, validation = "CV",
               segments = lapply(1:10, function(f) which(foldid == f)))
validationplot(pcr_fit, val.type = "RMSEP", main = "", legendpos = "topright")

cv_rmsep <- RMSEP(pcr_fit, estimate = "CV")$val[1, 1, -1]   # drop the 0-component entry
k_best   <- which.min(cv_rmsep)
k_best
```

```
35 comps
      35
```

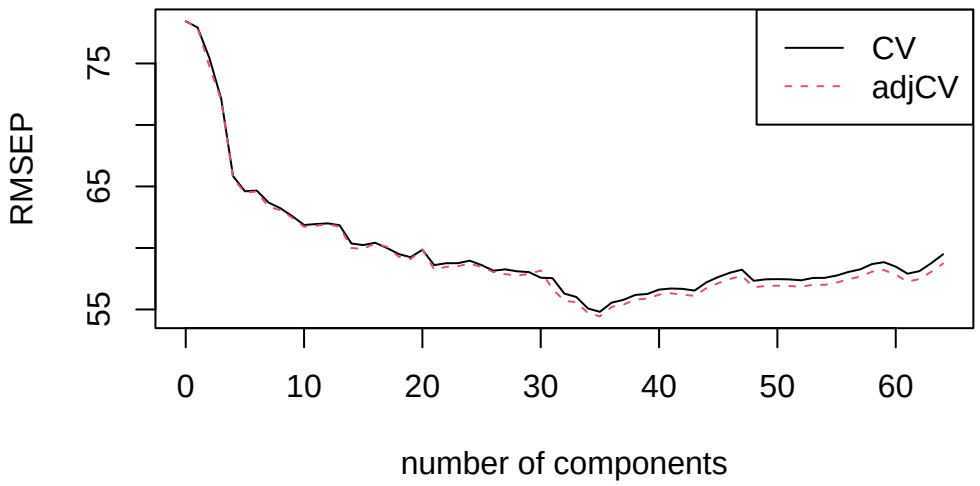


Figure 13.7: 10-fold CV error (RMSEP) for PCR on the diabetes training data, as a function of the number of components.

Code notes:

- `pcr(y ~ X, data = data.frame(y = y_tr, X = I(X_tr)))`: wrapping the matrix in `I()` stores it as a *single* matrix column named `X`, so the formula `y ~ X` uses all 64 predictors.
- `scale = TRUE` standardizes the predictors before the SVD. This matters for PCR, since otherwise high-variance *units* would dominate the components.
- `segments = ...` reuses the elastic-net fold assignment so the comparison below is fair.
- `RMSEP(...)$val` is a 3-D array (estimate, response, number of components). The first entry is the intercept-only model, so we drop it with `-1` before `which.min`.

13.7 Putting it together: test-set comparison on the diabetes data

We fit every method on the 342 training patients, choose tuning parameters by 10-fold CV with the same folds throughout, and evaluate on the 100 held-out patients.

```
test_mse <- function(pred) mean((y_te - pred)^2)
```

```r
ols_fit <- lm(y ~ ., data = data.frame(y = y_tr, X_tr))
ols_mse <- test_mse(predict(ols_fit, newdata = data.frame(X_te)))

glm_row <- function(fit, s, label) {
  c(method   = label,
    test_mse = round(test_mse(predict(fit, X_te, s = s))),
    nonzero  = sum(coef(fit, s = s)[-1] != 0))
}

res <- rbind(
  c(method = "OLS", test_mse = round(ols_mse), nonzero = ncol(X_tr)),
  glm_row(cv_fits[[1]], "lambda.min", "Ridge (alpha = 0)"),
  glm_row(cv_fits[[3]], "lambda.min", "Elastic net (alpha = 0.5)"),
  glm_row(cv_fits[[5]], "lambda.min", "Lasso (lambda.min)"),
  glm_row(cv_fits[[5]], "lambda.1se", "Lasso (lambda.1se)"),
  c(method   = paste0("PCR (k = ", k_best, ")"),
    test_mse = round(test_mse(drop(predict(pcr_fit, newdata = data.frame(X = I(X_te)),
                                           ncomp = k_best)))),
    nonzero  = ncol(X_tr))
)
knitr::kable(as.data.frame(res), align = "lrr",
             col.names = c("Method", "Test MSE", "# nonzero coefficients"))
```

Method	Test MSE	# nonzero coefficients
OLS	7728	64
Ridge (alpha = 0)	3537	64
Elastic net (alpha = 0.5)	3276	18
Lasso (lambda.min)	3280	17
Lasso (lambda.1se)	3427	9
PCR (k = 35)	3797	64

Code notes:

- `predict(fit, X_te, s = s)` on a `cv.glmnet` object predicts at the chosen λ (`"lambda.min"` or `"lambda.1se"`). `glmnet` applies its internal standardization automatically.
- `cv_fits[[1]]`, `[[3]]`, and `[[5]]` are the $\alpha = 0, 0.5, 1$ fits from the elastic-net section, so all methods share one set of CV folds.
- The `nonzero` column shows the selection behavior: OLS, ridge, and PCR use all 64 predictors, while the lasso and elastic net use only some of them.

How to read the results. With only 100 test patients, differences of a few hundred in MSE are within noise. (Try a different `set.seed` and the ranking may change.) The pattern that holds up is:

- Every regularized method **beats or matches OLS**. OLS spends 64 degrees of freedom on a design with an enormous condition number, and its variance shows up in the test error.
- The lasso and elastic net get similar accuracy to ridge with **far fewer predictors**, which is what we want when interpretability matters.
- PCR is competitive but usually slightly worse, since it picks directions without looking at y.

13.8 Summary

	Ridge	Lasso	Elastic net	PCR
Penalty / constraint	$\lambda\|\beta\|_2^2$	$\lambda\|\beta\|_1$	$\lambda[\alpha\|\beta\|_1 + \frac{1-\alpha}{2}\|\beta\|_2^2]$	keep top k PCs of X
Shrinkage pattern	smooth: $\frac{d_j^2}{d_j^2+\lambda}$ per PC direction	soft threshold per coefficient	soft threshold + proportional	hard threshold per PC direction
Exact zeros (selection)?	no	**yes**	**yes**	no
Closed form?	yes	no (coordinate descent / LARS)	no (coordinate descent)	yes (SVD)
Works for $p > n$?	yes	yes, but selects $\leq n$	yes, can select $> n$	yes ($k < n$)
Correlated predictors	shares weight (grouping)	picks one, often unstable	shares weight *and* selects	combines them into PCs
Bayesian analogue	Normal prior (posterior mean = mode)	Laplace prior (posterior mode)	Normal × Laplace prior	—
Tuning	λ	λ	λ, α	k
R	`glmnet(alpha = 0)`	`glmnet(alpha = 1)`	`glmnet(alpha = a)`	`pls::pcr()`

Rules of thumb:

- Expect **many small effects**? Ridge usually predicts best.
- Expect **a few large effects** and want an interpretable model? Use the lasso, and treat its selected set as a *screen*, not a final answer (see the Bühlmann discussion in the Paper chapter).

- **Groups of correlated predictors**, or $p \gg n$ where you want more than n variables? Use the elastic net.
- Predictors are **noisy measurements of a few underlying factors**? PCR (or PLS) matches that structure.
- Whatever you choose: **standardize, don't penalize the intercept, tune by CV with fixed folds, and evaluate on held-out data.**

13.9 References

- Efron, B., Hastie, T., Johnstone, I. & Tibshirani, R. (2004). Least angle regression. *Annals of Statistics*, 32, 407–499.
- Friedman, J., Hastie, T. & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1).
- Hastie, T., Tibshirani, R. & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.), Ch. 3. Springer.
- Hoerl, A. E. & Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12, 55–67.
- Massy, W. F. (1965). Principal components regression in exploratory statistical research. *JASA*, 60, 234–256.
- Park, T. & Casella, G. (2008). The Bayesian lasso. *JASA*, 103, 681–686.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *JRSS-B*, 58, 267–288.
- Tibshirani, R. (2011). Regression shrinkage and selection via the lasso: a retrospective. *JRSS-B*, 73, 273–282.
- Tseng, P. (2001). Convergence of a block coordinate descent method for nonsmooth separable minimization. *J. Optimization Theory and Applications*, 109, 475–494.
- Zou, H. & Hastie, T. (2005). Regularization and variable selection via the elastic net. *JRSS-B*, 67, 301–320.